



BENI-SUEF UNIVERSITY
FACULTY OF COMPUTERS & ARTIFICIAL INTELLIGENCE

Heart Guard

Cardiovascular Risk Prediction Platform

Course Integration: Senior Graduation Capstone Documentation

Academic Year: 2025 / 2026

Prepared by

Abdelmasseh Malak Israel

Kerolos Issa Fathy

Isaac Riad Isaac Riad

Marvy Ehab Kamal

Under the Academic Supervision of:

Prof. Dr. Muhammed Sayed Kayed & Dr mohamed salah — Dean
/ Senior Project Advisor

T.A. Ahmed Fayez — Technical Engineering Mentor

June 2026

Acknowledgement

We would like to express our deepest gratitude to our project advisor Dr. Mohamed salah and, Dr. Muhammed Sayed Kayed, Dean of the Faculty of Computers and Artificial Intelligence at Beni-Suef University. His academic experience, domain expertise, and meticulous guidance on system design empowered the team to refine complex data patterns and maintain a rigorous standard of scientific inquiry throughout every design iteration.

We are equally grateful to our Teaching Assistant and technical mentor, Dr. Ahmed Fayez, for his structural support, technical review sessions, and practical insight during milestone evaluations, all of which elevated the structural depth and academic alignment of this work.

Finally, our sincere appreciation goes to the administrative and academic staff of the Faculty of Computers and Artificial Intelligence for cultivating an environment conducive to building high impact software systems that address real local, national, and regional challenges in applied computing.

Purpose of the Documentation

This documentation serves as the comprehensive structural blueprint, technical requirements specification, and interface architecture foundation of the Heart Guard cardiovascular risk prediction ecosystem. It records the conceptualization, system logic, requirements engineering, and human-centered design layers of a platform explicitly built to resolve the pervasive fragmentation found in contemporary preventive cardiac healthcare and remote patient monitoring.

It is submitted to the academic examination panel at Beni-Suef University's Faculty of Computers and Artificial Intelligence as

rigorous evidence of proficiency in modern mobile UI/UX design workflows, software engineering principles, predictive healthcare modeling, and production-grade machine learning integration. This report intentionally focuses on front-end system requirements, human-centered interface logic, exhaustive clinical and competitive research, and the rigorous statistical data-processing pipelines driving the predictive Gradient Boosting subsystem. It deliberately excludes low-level backend server source code, cross-platform Flutter deployment infrastructure, and production database execution scripts, which fall outside the specific scope of this design-engineered and machine learning-led capstone thesis.

Abstract

HeartGuard is a mobile health application built to help patients understand and monitor their risk of heart disease in a simple, guided way. It offers an easy way to record vital signs and lifestyle information, submit a heart-health assessment, and instantly receive a structured, easy-to-read risk report. The app includes an interactive History dashboard with animated charts that track risk probability, BMI, and blood pressure over time, a chat assistant that answers heart-health questions and can analyze attached medical files such as ECGs, X-rays, and lab reports, and an educational articles section covering prevention and nutrition topics. Every assessment is processed by a dedicated Machine Learning service that classifies the patient's vital signs and generates a structured analysis — including risk factors, recommendations, lifestyle tips, and warning signs — which is also exportable as a shareable PDF report. HeartGuard works on Android and iOS from a single Flutter codebase, connecting to a Node.js/Express backend with MongoDB for data persistence, creating an accessible tool that keeps patients informed and encourages proactive heart-health management.

Table of Contents

Acknowledgement	1
Purpose of the Documentation	2
Abstract.....	3
Chapter 1: Introduction & System Motivation.....	7
1.1 Project Introduction	7
1.2 Problem statement	7
1.3 Project objectives.....	8
1.4 Why Machine Learning	8
1.5 Project scope.....	8
1.6 Machine Learning overview	9
1.7 Project workflow	10
Chapter 2 : Software Engineering Standards & Requirements Specification	11
2.1 Requirements	11
2.2 Functionality	12
2.3 Non-Functional Requirements	13
2.4 Product Requirements	14
2.5 Entity-Relationship Diagram	14
2.6 Use Case Diagram	17
Chapter 3: UI/UX Design & Architecture	19
3.1 UI&UX Design and software architecture overview	19
3.1.1 Home dashboard	20
3.1.2 Health Assessment step-1	21
3.1.3 Health Assessment step-2	22
3.1.4 Health Assessment step-3	23
3.1.5 Health Assessment step-4	24
3.1.6 Assessment Result Screen	25
3.1.7 Health overview / Analytics Dashboard	27
3.1.8 Find a doctor screen	28
3.1.9 AI Health chatbot.....	29

Chapter 4: Mobile Application	30
1. Overview	30
2. Advantages of flutter	30
3. Comparison with other development approaches	31
3.1 Native development	31
3.2 React Native	31
4. Why flutter for HeartGuard	31
5. conclusion	32
6. The user interface	33
7. Features of HeartGuard	49
8. Implementation of HeartGuard	49
9. Integration with API	51
10. BLOC management	53
11. ML Integration	55
12. How to use the app	57
 Chapter 5: Machine Learning Model	 58
5.1 Part1 : Dataset	58
5.2 Part2 : Data Processing	65
5.3 Part3 : Model Selection and Hyperparameter Configuration.....	73
5.4 Part4 : Model Training	78
5.5 Part5 : Evaluation Metrics and Results	85
5.6 Part6 : Feature Importance.....	90
5.7 Part7 : Model Saving and Interface Pipeline.....	94
 Chapter 6 : Backend API.....	 99
6.1 Technology Stack	99
6.2 Project Structure.....	100
6.3 Endpoints	101
6.4 Data Models.....	110
6.5 HTTP Status Code.....	112
6.6 Running the Project	113
 Chapter 7: Security	 115
7.1 XSS – Cross-site Scripting	116

7.2 NoSQL Injection.....	119
7.3 Fake Token	121
7.4 Missing Authorization Token	123
7.5 Weak Password Rejection.....	125
7.6 Mass Assignment	128
Chapter 8 : Challenges and Conclusion.....	130
8.1 Introduction	131
8.2Conclusion.....	132

Chapter 1: Introduction

1.1 Introduction

Cardiovascular disease remains one of the leading health concerns worldwide, and early awareness of personal risk factors is one of the most effective ways to encourage preventive action. HeartGuard, a mobile application developed using Flutter, addresses this need by giving patients a private, accessible tool to assess and track their heart-health risk over time. The application leverages Flutter's cross-platform capabilities to deliver a single, consistent experience across Android and iOS devices from one codebase. Its core features include a guided assessment flow for entering vital signs and lifestyle data; an ML-powered risk report with AHA-aligned classification of blood pressure, blood glucose, and cholesterol; an interactive History dashboard with animated trend charts for risk probability, BMI, and blood pressure across assessments; a chat assistant that answers heart-health questions and can analyze uploaded medical files; a PDF export feature for sharing results with a physician; and an educational articles library. By employing the BLoC pattern for state management and a multi-layered caching strategy (in-memory plus persistent local storage), HeartGuard remains responsive and usable even under unreliable network conditions. This project aims to create an accessible, informative, and encouraging companion for heart-health awareness, helping patients take an active role in monitoring and improving their cardiovascular health.

1.2 Problem Statement

Despite the well-established link between certain clinical measurements and heart disease, most healthcare settings still rely on manually applied risk criteria or general clinical judgment to flag patients who may be at risk. These traditional approaches are typically based on a small number of fixed thresholds, for example treating a single blood pressure or cholesterol reading as the main deciding factor, rather than considering how multiple indicators interact with one another.

This creates two practical limitations. First, traditional methods can miss patients whose individual readings appear normal but whose overall combination of factors still points to elevated risk. Second, they offer little consistency across different practitioners, since the interpretation of borderline cases often depends on individual experience rather than a standardized, data-driven process.

An intelligent prediction system addresses both of these gaps. By learning from patterns across a large number of patient records, a machine learning model can weigh multiple clinical factors simultaneously and produce a consistent, evidence-based risk estimate for each patient. This does not remove the need for clinical expertise, but it gives healthcare professionals an additional, data-driven reference point to support their assessment.

1.3 Project Objectives

The HeartGuard project was developed with a clear set of goals in mind, centered on using machine learning to make cardiovascular risk assessment more accessible and more consistent. The main objectives of the project are to:

- Support medical decision-making by providing healthcare professionals with an additional, data-driven reference for assessing a patient's cardiovascular risk.
- Improve early risk prediction by identifying patients who may be at risk before symptoms become clinically apparent.
- Use machine learning to analyze patient data, allowing multiple clinical and demographic factors to be considered together rather than in isolation.

Together, these objectives guided the design of the system, from how the underlying data was processed to how the final model was selected and evaluated.

1.4 Why Machine Learning?

Cardiovascular risk is rarely determined by a single factor. It emerges from the combined effect of variables such as age, blood pressure, cholesterol levels, and lifestyle indicators, and the way these factors interact can vary considerably from one patient to another. This is precisely the kind of problem machine learning is well suited to handle, since it does not rely on a fixed set of rules but instead learns patterns directly from historical patient data.

Rather than checking each clinical factor separately, a machine learning model can analyze all of a patient's relevant features simultaneously and recognize combinations that are associated with higher or lower risk. This kind of pattern recognition is difficult to replicate manually, particularly when the number of contributing factors is large. As new patient records are used to train the model, it becomes better at distinguishing between subtle patterns that separate at-risk patients from those who are not, without needing every relationship to be explicitly programmed.

In practical terms, this means the model can take in a patient's clinical profile and return a risk estimate that reflects the interaction of many variables at once, which is closer to how an experienced clinician might weigh several factors together, but done in a consistent and repeatable way.

1.5 Project Scope

HeartGuard is designed to predict a patient's risk of cardiovascular disease based on their clinical information, such as demographic details, vital measurements, and laboratory results. Given a patient's data, the system produces a risk classification that indicates the likelihood of cardiovascular disease being present.

It is important to be clear about what the system does not do. HeartGuard is intended to support healthcare professionals in their assessment process, not to replace medical diagnosis. The output of the model is a statistical risk estimate based on patterns learned from data, and any decision regarding diagnosis, treatment, or further testing remains the responsibility of a qualified medical professional. The system does not interpret medical images, does not account for factors outside the defined set of input features, and is not intended for use as a standalone diagnostic tool.

1.6 Machine Learning Overview

From a technical standpoint, the cardiovascular risk prediction task is framed as a binary classification problem, where the model assigns each patient to one of two outcomes: at risk or not at risk of cardiovascular disease.

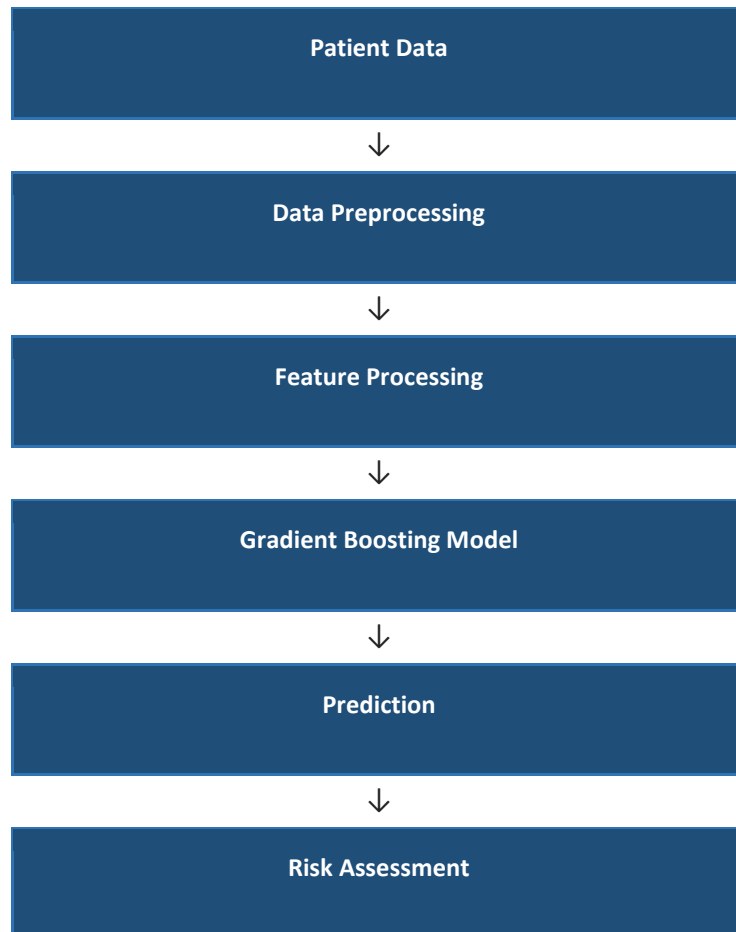
The dataset used to train and evaluate the model is HeartGuardDataset, sourced from Center for disease control and prevention in national health and nutrition examination survey, and consists of 23 clinical and demographic features per patient. The prediction target is HeartDisease.

The predictive model at the core of the system is a Gradient Boosting classifier, an ensemble learning method that builds a sequence of decision trees, where each new tree focuses on correcting the errors made by the ones before it. This approach tends to perform well on structured, tabular clinical data of the kind used in this project, and it produces predictions by combining the patient's medical features into a single risk output.

To evaluate how well the model performs, several classification metrics are used together rather than relying on a single number. Accuracy gives a general sense of overall correctness, Recall measures how well the model identifies patients who are genuinely at risk, and ROC-AUC captures the model's ability to distinguish between the two classes across different decision thresholds. These are considered alongside other classification metrics to give a fuller picture of the model's reliability before it is used to support real assessments.

1.7 Project Workflow Overview

The diagram below summarizes the end-to-end flow of the HeartGuard system, from the moment patient data is collected to the point where a risk assessment is produced.



The process begins with the collection of patient data, covering the demographic and clinical information needed for assessment. This data then passes through data preprocessing, where it is cleaned and prepared for modeling, followed by feature processing, which transforms the raw clinical values into a format the model can interpret effectively. The processed features are then passed into the Gradient Boosting model, which generates a prediction indicating the patient's likelihood of cardiovascular disease. Finally, this prediction is translated into a risk assessment that can be reviewed by healthcare professionals as part of their overall evaluation of the patient.

Chapter 2: System Requirements

2.1 Requirements

This chapter defines the function of the Heart Guard system from the client's viewpoint. The requirements establish the system's functionality, constraints, and goals, and were developed through consultation with the intended users — patients seeking to monitor their cardiovascular health, and the clinicians who receive referrals generated by the system.

Requirements for Heart Guard emerged incrementally, beginning with the core patient-facing assessment workflow and expanding to include the historical dashboard, the AI health chatbot, and the X-ray upload and discussion feature as the scope of the platform grew. These requirements form the basis for acceptance testing of the delivered system: each functional and non-functional requirement below maps to one or more test cases that must pass before a release is accepted.

The development team worked closely with the client throughout the requirements phase to ensure that the resulting specification is understandable to both the client and the development staff, and that it is expressed in a form — written descriptions, entity diagrams, and use case diagrams — that leaves little room for misinterpretation on either side.

Requirement Goals

- Understand the requirements of the Heart Guard platform in appropriate detail, covering the patient journey from login through assessment to follow-up care.
- Define requirements in a manner that is clear to the client, using plain-language descriptions supported by diagrams (ER diagram, use case diagram) rather than technical jargon alone.
- Define requirements in a manner that is clear to the people who will design, implement, and maintain the system, including precise data structures and interface behaviour.
- Ensure that the client and the development team share the same understanding of each requirement and its implications for scope, timeline, and risk (e.g. handling of sensitive health data).

Functional Requirements

Functional requirements describe the functions that the Heart Guard system must perform. They were identified by analysing the intended use of the system and are grouped below under Functionality, Data, and User Interfaces.

Functionality

ID	Requirement	Description
FR-1	Patient Registration & Login	The system shall allow a patient to create an account and authenticate using an email address and password before accessing any other feature.
FR-2	Assessment Dashboard	The system shall display a dashboard listing the patient's past health assessments in chronological order, including the date and overall risk outcome of each.
FR-3	Health Input Collection	The system shall provide a form through which the patient enters the health inputs required for an assessment (e.g. blood pressure, cholesterol, heart rate, BMI, blood sugar, smoking status).
FR-4	Assessment Processing	The system shall run the entered health inputs through a diagnostic algorithm to produce a risk score and classification for the patient.
FR-5	Results & Recommendations	The system shall present the patient with the outcome of the assessment together with a personalised recommendation, generated from the risk classification.
FR-6	Doctor Referral	Where the risk classification exceeds a defined threshold, the system shall generate a referral to a doctor and record the referral against the patient's assessment history.
FR-7	AI Health Chatbot	The system shall provide a conversational chatbot that the patient can use to ask health-related questions and receive AI-generated guidance.
FR-8	X-ray Upload	The system shall allow a patient to upload an X-ray image file for storage against their profile.
FR-9	X-ray Discussion	The system shall allow the patient to discuss an uploaded X-ray with the AI assistant, which shall reference the specific image in its responses.

Data

The system must capture, store, and retrieve the following categories of data:

- Patient account data — name, email, password (hashed), date of birth, gender, phone number.
- Health input data — the raw metrics entered for each assessment (blood pressure, cholesterol, heart rate, BMI, blood sugar, smoking status, family history).
- Assessment and recommendation data — the algorithm's risk score, risk level, generated recommendation text, and referral status for each assessment.
- Referral data — the doctor assigned, referral date, urgency level, and status of each referral.
- Conversation data — chatbot session and message history, and X-ray discussion message history, each timestamped and attributed to sender (patient or AI).
- Media data — uploaded X-ray image files and any AI-generated analysis summary associated with them.

User Interfaces

- A login / registration screen for account creation and authentication.
- A dashboard screen summarising historical assessments, with the ability to open any past assessment for detail.
- An assessment input screen for entering health metrics, with clear validation of required fields.
- A results screen presenting the risk outcome, recommendation, and (where applicable) referral details.
- A chatbot interface supporting free-text conversation with the AI health assistant.
- An X-ray upload and discussion interface allowing image upload alongside a conversational thread about that image.

2.3 Non-Functional Requirements

Non-functional requirements are requirements that are not directly related to the functions the system must perform, but instead constrain how those functions are delivered. They are grouped below into product, organizational, and external requirements.

2.4 Product Requirements

ID	Requirement	Description
NFR-1	Performance	The diagnostic algorithm shall return a risk assessment result within 3 seconds of submission under normal load.
NFR-2	Reliability	The system shall be available at least 99.5% of the time per month, excluding scheduled maintenance windows.
NFR-3	Portability	The patient-facing application shall be usable on current versions of major desktop and mobile web browsers without loss of functionality.
NFR-4	Data Security	All health data shall be encrypted in transit and at rest, and passwords shall never be stored in plain text.
NFR-5	Usability	A first-time patient shall be able to complete a full health assessment, unaided, in under 5 minutes.

Organizational Requirements

- Delivery: the system shall be delivered in incremental releases, beginning with registration, dashboard, and assessment, followed by the chatbot, and finally the X-ray feature.
- Training: the client's support staff shall receive a training session and reference guide covering the referral workflow before go-live.
- Standards: development shall follow the client's existing coding, documentation, and change-management standards.

External Requirements

- Legal & compliance: the system shall comply with applicable health-data protection regulations (e.g. HIPAA or GDPR, depending on deployment region) for the storage and handling of patient health information.
- *Interoperability: referral records shall be exportable in a format that a receiving clinic's system can import.*
- Marketing & public relations: patient-facing language (assessment results, chatbot responses) shall be reviewed to ensure it is reassuring and non-alarming in tone, consistent with the client's public messaging around the product.

2.5 Entity-Relationship Diagram

The Entity-Relationship Diagram (ERD) translates the data requirements defined in Section 4.1 into a concrete data model. It identifies the persistent entities the Heart Guard system must

store, the attributes of each entity, and the relationships that connect them, and forms the basis for the database schema.

HeartGuard Backend – Entity-Relationship Diagram

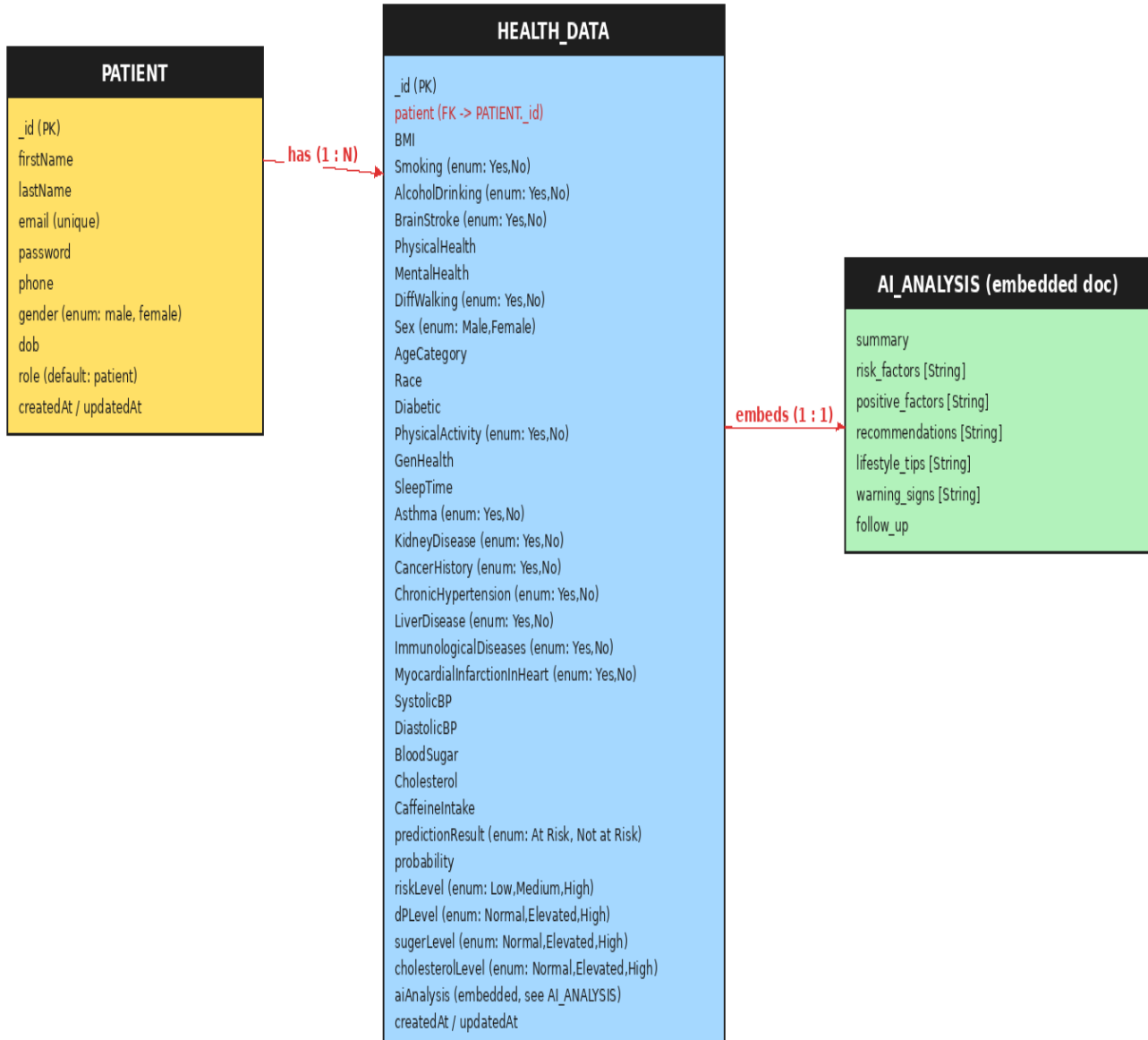


Figure 4.1 — Heart Guard Entity-Relationship Diagram

Entities

The data model is organised around ten entities, grouped into three functional clusters: the core assessment flow, the AI chatbot, and the X-ray feature.

Entity	Description
User (Patient)	The registered patient. Stores login credentials (email, password) and profile details (name, gender, date of birth, role).
Health Data	A single record containing all of a patient's health inputs (BMI, blood pressure, blood sugar, cholesterol, smoking, chronic diseases, etc.) plus the prediction outcome (prediction result, probability, risk level). Linked to Patient with a (1:N) relationship.
AI Analysis	An embedded document inside Health Data containing the AI-generated analysis: summary, risk factors, positive factors, recommendations, lifestyle tips, warning signs, and follow-up. Related to Health Data with a (1:1) relationship.

Relationships

Each relationship below is expressed with its cardinality, read from the first entity to the second:

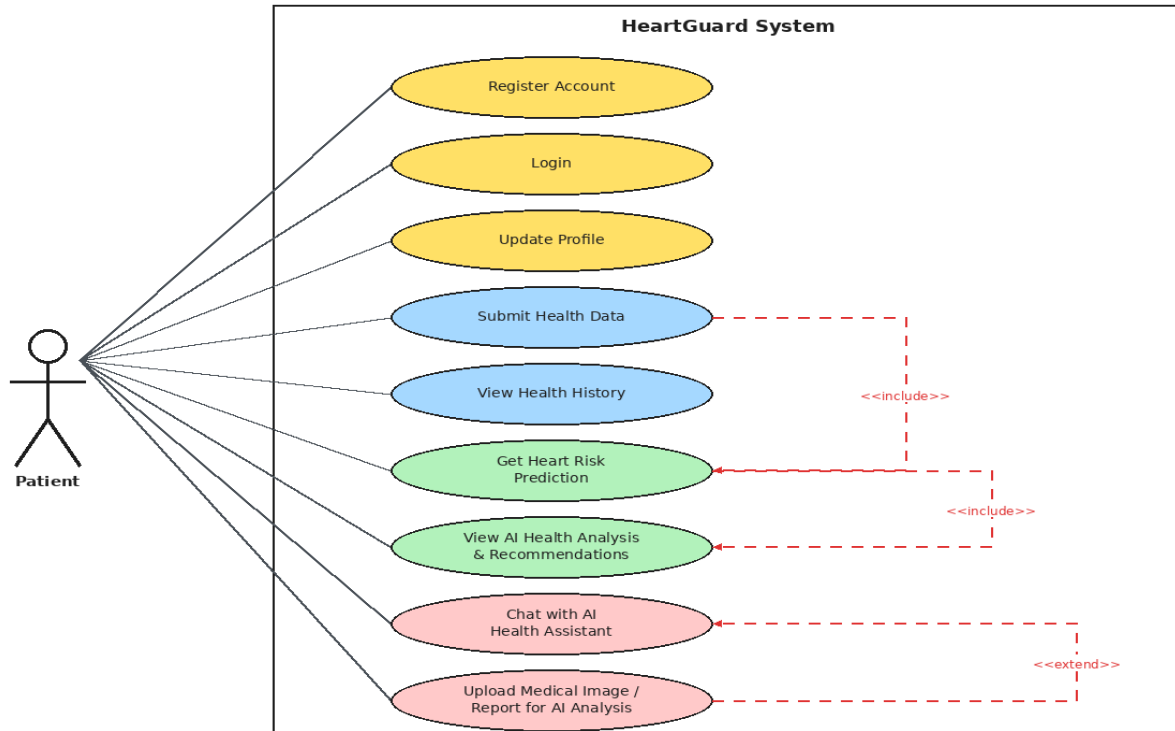
- **Patient → Health Data (1:N)** — a patient can have any number of health data records (each representing one assessment submission).
- **Health Data → AI Analysis (1:1)** — each health data record has exactly one embedded AI analysis document (summary, risk factors, recommendations, lifestyle tips, warning signs, follow-up).

2.6 Use Case Diagram

The Use Case Diagram complements the ERD by describing the system from the perspective of its actors: who interacts with Heart Guard, and what each of them is able to do. It restates the functional requirements, making the include/extend relationships between use cases explicit.

HeartGuard — Use Case Diagram

(derived from Patient.js, HealthData.js, chatController.js — chat has no DB-persisted history)



Actors

Actor	Role
Patient (primary)	The end user of the system; initiates every top-level use case (login, dashboard, assessment, chatbot).
Doctor (secondary)	The system's own diagnostic algorithm and chatbot component; participates in use cases as a non-human actor.

Use Cases

Use Case	Notes
Login / Authenticate	Entry point for all patient interaction.
View Assessment Dashboard	Displays the patient's historical assessments.
Perform Health Assessment	Includes Enter Health Inputs, Run Diagnostic Algorithm, and View Results & Recommendations.
Enter Health Inputs	<<include>> of Perform Health Assessment.
Run Diagnostic Algorithm	<<include>> of Perform Health Assessment; performed by the AI Engine actor.
View Results & Recommendations	<<include>> of Perform Health Assessment; may <<extend>> into Generate Doctor Referral.
Chat with AI Health Assistant	Direct conversation between the Patient and the AI Engine actor.

Include / Extend Relationships

- Perform Health Assessment <<include>> Enter Health Inputs — always executed as part of the assessment.
- Perform Health Assessment <<include>> Run Diagnostic Algorithm — always executed as part of the assessment.
- Perform Health Assessment <<include>> View Results & Recommendations — every assessment concludes by showing a result.

Chapter 3: UI/UX Design & Software Architecture

UI/UX Design and Software Architecture Overview

This chapter documents the system's design and software architecture. It outlines the structural framework and the user interface principles applied across Heart Guard to ensure a seamless and efficient user experience, and maps each screen back to the data model (Chapter 4.2) and use cases (Chapter 4.3) that drive it.

3.1 UI/UX Design

The user interface and user experience design for this project focus on usability, accessibility, and intuitive navigation. The complete design system, including wireframes and interactive prototypes, is maintained in Figma:

Figma project: [HEART DISEASE — Figma Design File](#)

The nine screens below cover the primary patient journey — from the home dashboard, through the four-step health assessment, to the results, historical analytics, doctor referral, and AI chatbot. Each entry gives the screen's technical implementation and key highlight.

3.1.1 Home Dashboard

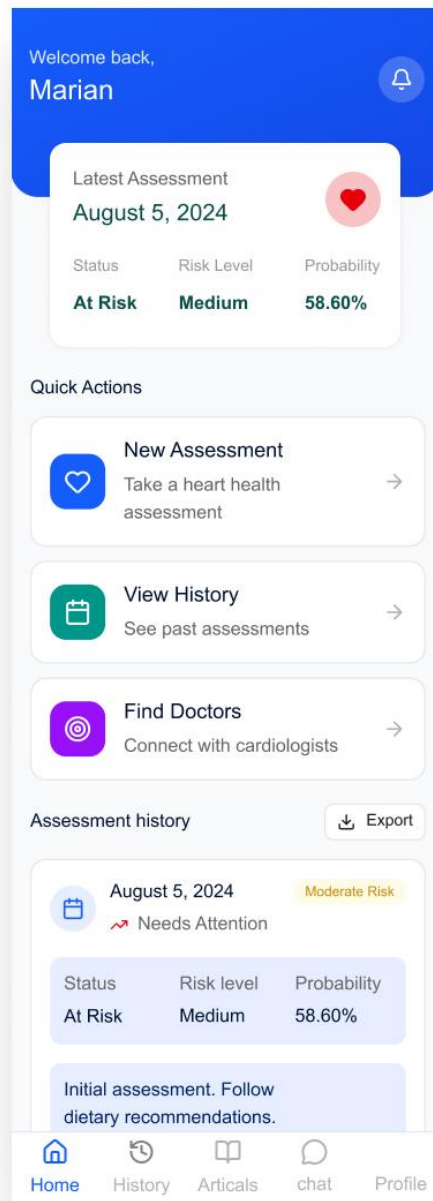


Figure 5.1 — Home Dashboard

- **Technical Implementation:** The summary card and history list are both driven by the patient's Health Assessment and Recommendation records, queried once for the most recent entry and once as a paginated history list; Quick Actions are simple navigation links to the Assessment, History, and Find-Doctor screens; the bottom tab bar is implemented as a persistent navigation shell so state is preserved when switching tabs.

- Key Highlight: The current risk status is surfaced in the very first fold of the screen, so a returning patient gets an at-a-glance read of their heart health before doing anything else.

3.1.2 Health Assessment — Step 1: Demographics & Basic Health

The screenshot shows a mobile app interface for a health assessment. At the top, there is a back arrow, the title 'Health Assessment', and 'Step 1 of 4' with a progress bar. The main content area is titled 'Demographics & Basic Health' and asks the user to provide basic information and vital signs. It contains several input fields: 'Age (years)' with the value '45', 'Sex' with a dropdown menu showing 'Select your sex', 'Height (cm)' with the value '170', 'Weight (kg)' with the value '70', and 'Race' with a dropdown menu. At the bottom, there is a blue button labeled 'Continue to Step 2' with a right arrow.

Figure 5.2 — Health Assessment, Step 1: Demographics & Basic Health

- Technical Implementation: Implemented as the first screen of a client-side multi-step wizard; each field writes into a shared in-progress Health Input draft object that persists across all four steps, with the progress bar bound to the current step index out of four.
- Key Highlight: Splitting a long clinical intake into four short, clearly-labelled steps reduces perceived effort and lowers the drop-off risk of the assessment.

3.1.3 Health Assessment — Step 2: Lifestyle & Habits

The screenshot shows a mobile app interface for a health assessment. At the top, there is a back arrow, the title 'Health Assessment', and 'Step 2 of 4'. A progress bar is partially filled. The main section is titled 'Lifestyle & Habits' with a subtitle 'Help us understand your daily lifestyle choices'. It contains five questions: 'Sleep Time (hours per day) in english?' with a text input field containing '8'; 'Caffeine Intake (cups per day) in english?' with a text input field containing '2'; 'Do you smoke?' with radio buttons for 'Yes' and 'No'; 'Do you drink alcohol?' with radio buttons for 'Yes' and 'No'; and 'Physical activity (exercise)?' with radio buttons for 'Yes' and 'No'. Below these is a question 'Difficulty walking/climbing stairs?' with radio buttons for 'Yes' and 'No'. At the bottom is a blue button labeled 'Continue to Step 3 →'.

Figure 5.3 — Health Assessment, Step 2: Lifestyle & Habits

- Technical Implementation: Yes/No answers are modelled as boolean fields on the same Health Input draft used in Step 1; the step reuses the shared wizard shell and progress-bar component, only swapping the field set and validation rules.
- Key Highlight: Lifestyle and habit questions are separated from clinical vitals, letting the downstream recommendation engine distinguish modifiable risk factors from ones that require medical attention.

3.1.4 Health Assessment — Step 3: Vital Signs & Labs

←

Health Assessment

Step 3 of 4

Vital Signs & Labs

Please provide your vital signs and lab results

Blood Pressure

Systolic	Diastolic
120	80

mmHg

Blood Sugar (mg/dL) in english?

100

Cholesterol (mg/dL) in english?

120

Continue to Step 4 →

Figure 5.4 — Health Assessment, Step 3: Vital Signs & Labs

- **Technical Implementation:** Systolic and diastolic values are captured as two related fields inside one visually grouped card, stored as separate blood-pressure attributes on the Health Input entity; numeric input types are enforced so mobile keyboards default to the numeric layout.
- **Key Highlight:** Grouping systolic and diastolic readings into a single visual unit mirrors how blood pressure is actually read and reduces the chance of transposed or mismatched entries.

3.1.5 Health Assessment — Step 4: Medical History

The screenshot shows a mobile application interface for a health assessment. At the top, there is a back arrow, the title "Health Assessment", and the subtitle "Step 4 of 4". A blue progress bar is visible. The main content area contains several questions with radio button options for "Yes" and "No". The questions are: "Are you a liver patient?", "Mental Health Days (last 30 days) in english?" (with a text input field containing "10"), "Are you diabetic?", "Are you suffering from chronic high blood pressure?", "Are you a liver patient?", "Do you have kidney disease?", "Do you have an autoimmune disease?", "Did you have a stroke?", "Did you have a heart attack?", and "Asthma?". At the bottom, there is a blue button labeled "Review Answers" with a right-pointing arrow.

←

Health Assessment

Step 4 of 4

Are you a liver patient?

Mental Health Days (last 30 days) in english?

10

Are you diabetic?

☐ Yes ☐ No

Are you suffering from chronic high blood pressure?

☐ Yes ☐ No

Are you a liver patient?

☐ Yes ☐ No

Do you have kidney disease?

☐ Yes ☐ No

Do you have an autoimmune disease?

☐ Yes ☐ No

Did you have a stroke?

☐ Yes ☐ No

Did you have a heart attack?

☐ Yes ☐ No

Asthma?

☐ Yes ☐ No

Review Answers →

Figure 5.5 — Health Assessment, Step 4: Medical History

- **Technical Implementation:** Each Yes/No answer maps to a boolean risk-modifier flag consumed directly by the diagnostic algorithm; “Review Answers” transitions the patient to a confirmation state before the completed Health Input record is finalised and submitted, triggering the Run Diagnostic Algorithm use case.

- Key Highlight: A dedicated review step before final submission gives the patient one last chance to correct answers that materially change the risk score, reducing incorrectly high or low assessment outcomes.

3.1.6 Assessment Result Screen

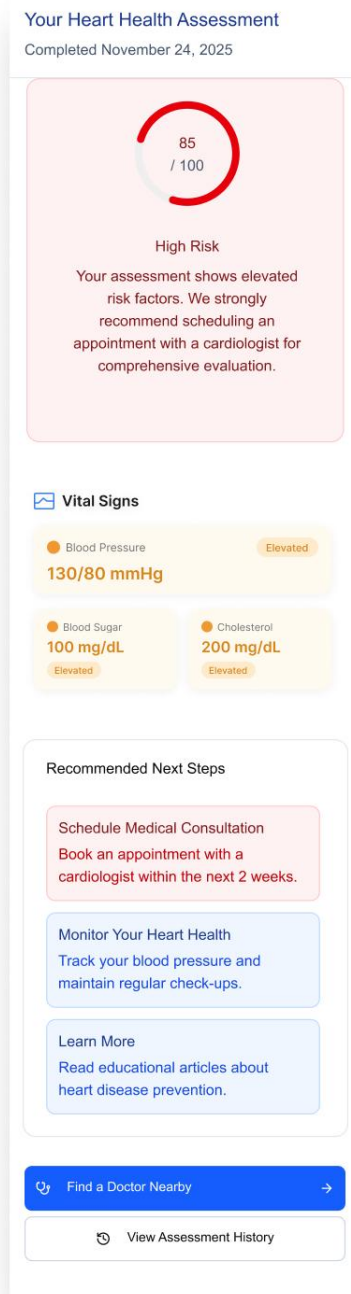


Figure 5.6 — Assessment Results

- **Technical Implementation:** The circular gauge renders the numeric risk score from the Recommendation entity; a Vital Signs section below it surfaces the key Health Input values for that assessment (Blood Pressure, Blood Sugar, Cholesterol), each tagged with a status badge (e.g. “Elevated”) computed by comparing the stored value against normal clinical ranges. Each “next step” card is conditionally displayed based on the risk level and the `requires_doctor_referral` flag, so a low-risk result would omit the consultation card; “Find a Doctor Nearby” deep-links into the Find a Doctor screen, pre-filtered to the referral's recommended specialization.
- **Key Highlight:** The screen doesn't stop at a score — it surfaces the specific vital signs driving that score and converts the result directly into a prioritised, actionable next step, closing the loop between assessment and follow-up care.

3.1.7 Health Overview / Analytics Dashboard

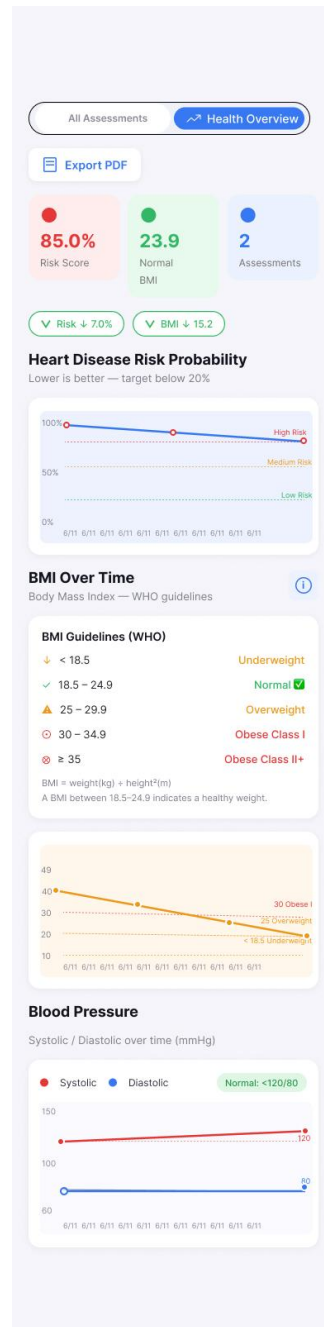


Figure 5.7 — Health Overview / Analytics Dashboard

- **Technical Implementation:** Each chart is rendered from the patient's full Health Assessment and Health Input history using a charting library, with each metric plotted chronologically by assessment date; the trend badges are computed as the percentage change between the two most recent assessments; Export PDF bundles the current charts and summary stats into a downloadable report.

- Key Highlight: Turning a list of past assessments into trend lines against clinical reference bands lets the patient see whether they are improving over time, not just what their most recent number was.

3.1.8 Find a Doctor Screen

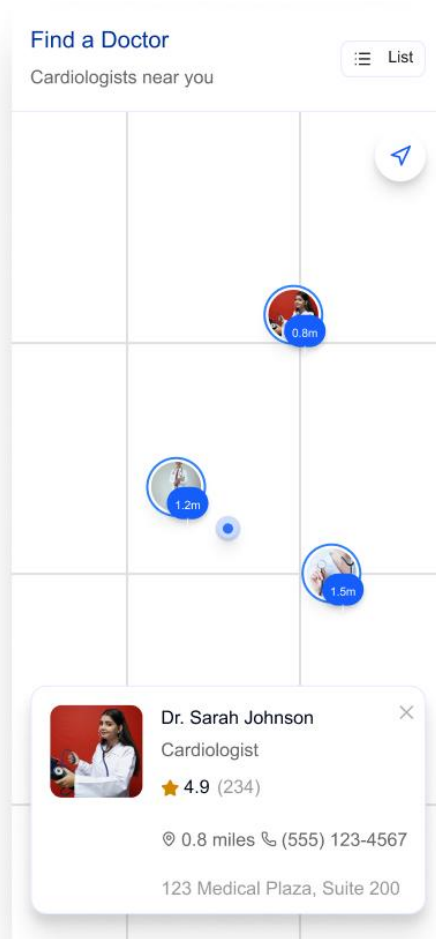


Figure 5.8 — Find a Doctor Screen

- Technical Implementation: Map markers are populated from the Doctor entity, filtered to cardiologists and sorted by geolocated distance from the patient's device; the bottom detail card binds to the selected Doctor record on marker tap; the List toggle renders the same dataset as a sortable list instead of a map.
- Key Highlight: Surfacing distance and a direct call button right on the map removes the friction between receiving a referral and actually booking an appointment.

3.1.9 AI Health Chatbot

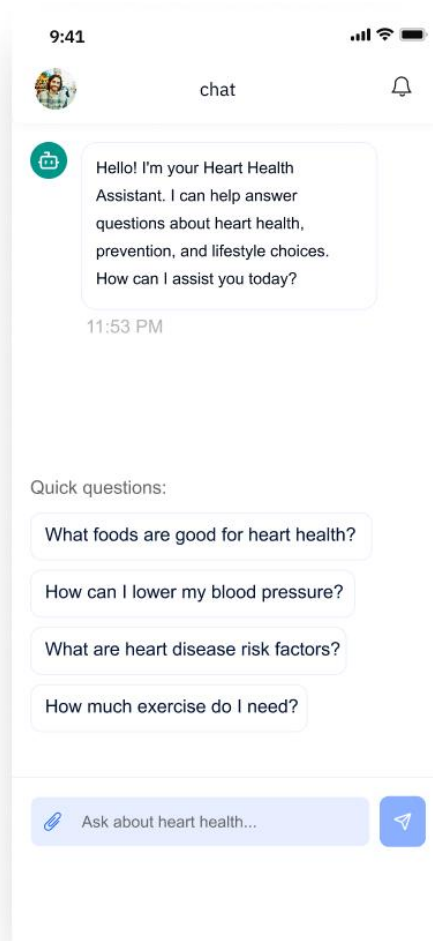


Figure 5.9 — AI Health Chatbot

- Technical Implementation: Each conversation is persisted as a Chat Session containing ordered Chat Message records, with a sender-type field distinguishing patient from AI; the quick-question chips are pre-written prompts that populate and submit the input field through the same send endpoint as free-text messages; responses are generated by the AI Engine actor, which also powers the X-ray discussion feature via the same messaging infrastructure.
- Key Highlight: The quick-question chips lower the barrier to starting a conversation, steering patients toward exactly the kind of preventive-health questions the assistant is designed to answer well.

Chapter 4 : Mobile Application

1. Overview

HeartGuard is a heart-health assessment app designed to give patients an accessible and informative experience on their own devices. It was built using Flutter, Google's toolkit for creating apps with great design and fast performance across platforms. Flutter was chosen because it is easy to work with, runs well on both Android and iOS, and makes it straightforward to build smooth, data-rich interfaces such as animated dashboards and chat screens. This section explains why Flutter was the right choice for HeartGuard and compares it with other options such as Native Development and React Native.

2. Advantages of Flutter

Flutter offers several compelling benefits that align with the requirements of HeartGuard:

- **Cross-Platform Development:** Flutter enables a single codebase to target both iOS and Android, reducing development time and cost.
 - **Impact on HeartGuard:** This allowed the same assessment, dashboard, and chat features to reach both Android and iOS patients without maintaining two separate codebases.
- **High Performance:** Flutter compiles to native machine code using Dart, delivering smooth animations and transitions.
 - **Impact on HeartGuard:** This performance supported smooth, animated `fl_chart` line charts on the History dashboard and responsive chat interactions without noticeable lag.
- **Rich and Customizable UI:** Flutter's widget-based architecture and Skia graphics engine provide pixel-perfect, highly customizable interfaces across platforms.
 - **Impact on HeartGuard:** This enabled a clean, colour-coded interface — such as green/amber/red vital-sign status badges and risk-level indicators — that communicates health information at a glance.
- **Hot Reload:** Flutter's hot reload lets developers see code changes instantly without restarting the app.
 - **Impact on HeartGuard:** Hot reload sped up iteration on the Result screen layout and the dashboard's chart styling, allowing quick visual adjustments during development.
- **Strong Ecosystem and Community:** Flutter has a mature ecosystem of packages (`flutter_bloc`, `dio`, `fl_chart`, `get_it`, `pdf`, `image_picker`) backed by an active community.
 - **Impact on HeartGuard:** This ecosystem accelerated the integration of animated charts, HTTP communication with the backend, dependency injection, and PDF report generation.

- **Responsive Layout Tools:** Widgets such as Column, Row, ListView, and MediaQuery-based sizing make it straightforward to build layouts that adapt to different screen sizes.
 - **Impact on HeartGuard:** This kept the History dashboard's charts and the chat interface legible and well-proportioned across different phone screen sizes.
- **Cost-Effectiveness:** A single codebase removes the need for separate iOS and Android teams, lowering development and maintenance costs.
 - **Impact on HeartGuard:** This let the team focus available time on the assessment logic, dashboard analytics, and chat experience rather than duplicating UI work.

3. Comparison with Other Development Approaches

3.1. Native Development (iOS: Swift, Android: Kotlin)

- **Advantages:** Best possible performance and full access to platform-specific APIs and hardware.
- **Disadvantages:** Requires two separate codebases, doubling development effort, and slower iteration without hot reload.

Comparison with Flutter: For HeartGuard, Native Development would have meant building and maintaining the assessment flow, dashboard charts, and chat screen twice. Flutter's near-native performance was sufficient for animated charts and smooth navigation, while a single codebase noticeably reduced development time.

3.2. React Native

- **Advantages:** Cross-platform with a single JavaScript codebase, large community, and hot reloading support.
- **Disadvantages:** Performance overhead from the JavaScript bridge, and UI consistency across platforms often needs platform-specific tweaks.

Comparison with Flutter: React Native was considered but Flutter's Skia-based rendering gave smoother chart animations and more predictable UI consistency across Android and iOS, which mattered for a health app where charts and risk indicators must be trustworthy and clearly legible.

4. Why Flutter for HeartGuard?

- **Cross-Platform Efficiency:** A single codebase reduced development time, freeing time for the assessment logic and dashboard analytics.
- **Performance and Visual Quality:** Flutter's rendering performance supported smooth chart animations that make risk trends easy to read.
- **Responsive Layouts:** Built-in layout tools kept the interface usable across different phone screen sizes.
- **Ecosystem Support:** Packages such as flutter_bloc, dio, fl_chart, and pdf accelerated feature development.

- Cost and Time Savings: Hot reload and a single codebase kept development efficient within the project's timeline.

5. Conclusion

Flutter was the right choice for building HeartGuard because it offers fast performance, an efficient development workflow, an attractive and consistent interface, and true cross-platform delivery. Compared to Native Development and React Native, Flutter offered the best balance of speed, visual quality, and development efficiency, allowing HeartGuard to deliver a trustworthy, data-driven heart-health companion to patients on both Android and iOS.

6. The User Interface

This section walks through HeartGuard's main Flutter screens in the order a patient typically encounters them: authentication, the home dashboard, assessment history and analytics, the ML-generated result and PDF export, the chat assistant, educational articles, and account/profile management.

Sign In Screen

The Sign In screen lets a returning patient access their heart-health dashboard using their email and password, with a link to switch to account creation for new patients.

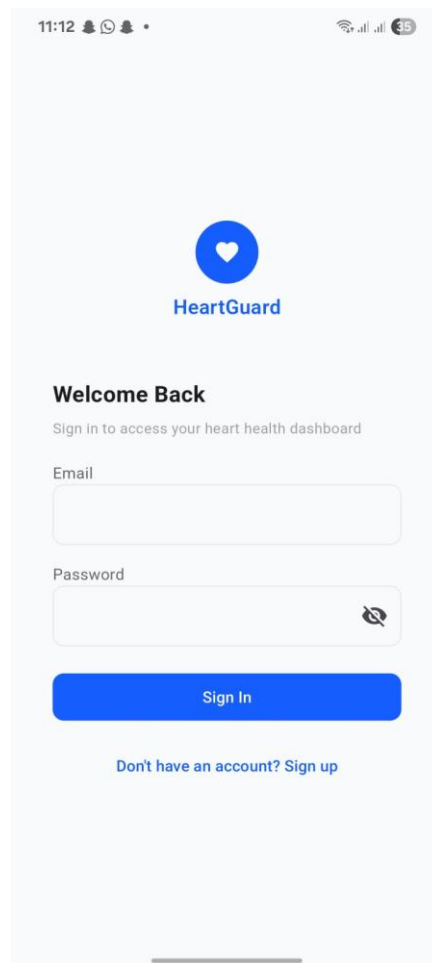


Figure: Sign In Screen screen

Create Account Screen

The Create Account screen collects the new patient's first and last name, email, and password (with confirmation), along with acceptance of the Terms of Service and Privacy Policy before registration.

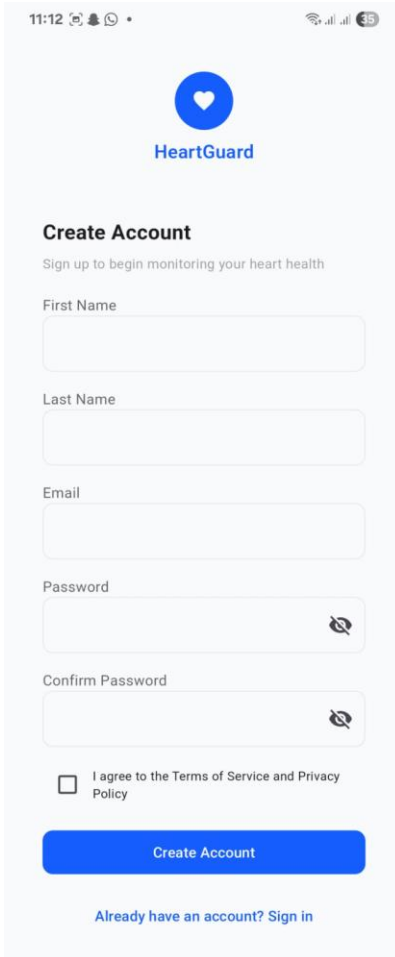
The image shows a mobile app interface for 'HeartGuard'. At the top, there's a status bar with the time '11:12', signal strength, and battery level '65'. Below the status bar is a blue circular logo with a white heart icon, and the text 'HeartGuard' underneath. The main heading is 'Create Account' in bold. Below it, a subtitle reads 'Sign up to begin monitoring your heart health'. The form consists of five input fields: 'First Name', 'Last Name', 'Email', 'Password', and 'Confirm Password'. The 'Password' and 'Confirm Password' fields have an eye icon to toggle visibility. Below the form is a checkbox labeled 'I agree to the Terms of Service and Privacy Policy'. At the bottom of the form is a blue button labeled 'Create Account'. Below the button is a link that says 'Already have an account? Sign in'.

Figure: Create Account Screen screen

Home Screen

The Home screen greets the signed-in patient by name and immediately surfaces their latest assessment (status, risk level, and probability), alongside Quick Actions for starting a new assessment, viewing history, or finding a doctor.

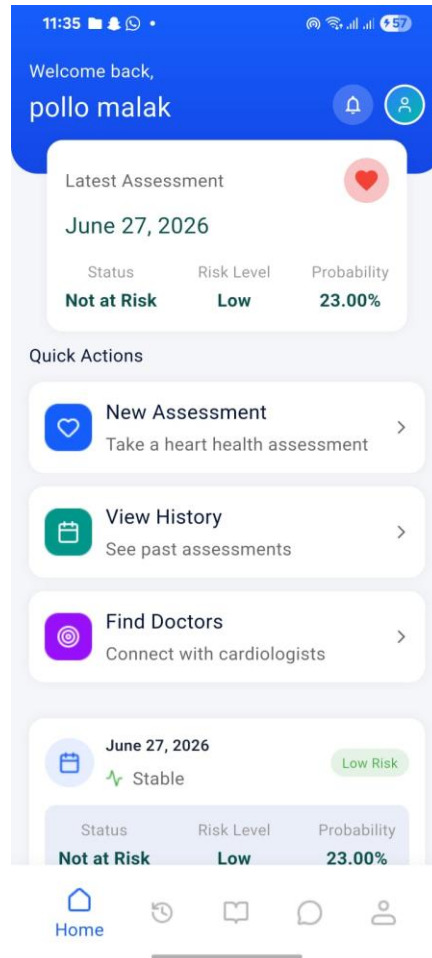


Figure: Home Screen screen

History Screen: All Assessments

The All Assessments tab of the History screen lets the patient start a new assessment and lists previous assessments with their date, stability trend, and risk level, each linking to its full report.

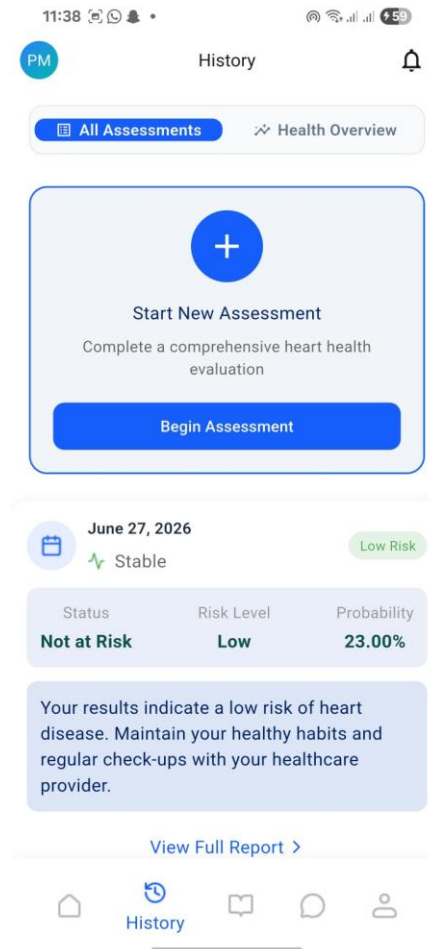


Figure: History Screen: All Assessments screen

History Screen: Health Overview (Dashboard)



Figure: History Screen: Health Overview (Dashboard) screen

PDF Export: Health Status Report

From the Health Overview dashboard, the patient can export a shareable PDF report that captures the same risk trend, BMI, and blood-pressure charts, the full assessment history table, and the aggregated health insights (recurring risk factors, persistent warning signs, and recommended actions).

Health Status Report

Patient: pollo malak
Report Date: 2026-07-04



Latest Health Assessment Summary

2026-06-27	120/80 mmHg	80 mg/dL	150 mg/dL	23.0%
Date	Blood Pressure	Blood Sugar	Cholesterol	Risk %

Heart Disease Risk Trend

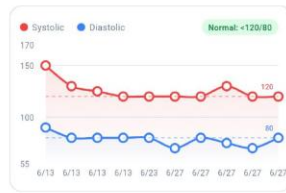


BMI Over Time



Blood Pressure Over Time

Figure: Exported PDF report — summary and trend charts



All Assessments History (10)

Date	Blood Pressure	Blood Sugar	Cholesterol	Risk %
2026-06-13	150/90 mmHg	150 mg/dL	250 mg/dL	95.0%
2026-06-13	130/80 mmHg	125 mg/dL	225 mg/dL	92.0%
2026-06-13	125/80 mmHg	110 mg/dL	210 mg/dL	92.0%
2026-06-13	120/80 mmHg	100 mg/dL	200 mg/dL	85.0%
2026-06-23	120/80 mmHg	90 mg/dL	190 mg/dL	82.0%
2026-06-27	120/70 mmHg	80 mg/dL	170 mg/dL	23.0%
2026-06-27	120/80 mmHg	80 mg/dL	170 mg/dL	82.0%
2026-06-27	130/75 mmHg	80 mg/dL	150 mg/dL	32.0%
2026-06-27	120/70 mmHg	120 mg/dL	108 mg/dL	23.0%
2026-06-27	120/80 mmHg	80 mg/dL	150 mg/dL	23.0%

AI Health Insights

Based on 10 assessments

Most Recurring Risk Factors

- smoking	5/10
- diabetes	4/10
- history of myocardial infarction	3/10
- high BMI	2/10
- lack of physical activity	2/10

Persistent Warning Signs

- shortness of breath	6x
- chest pain or discomfort	4x

Figure: Exported PDF report — assessment history table and health insights

Result Screen (Part 1)



Figure: Result Screen (Part 1) screen

Result Screen (Part 2)

Scrolling down, the Result screen lists the identified risk factors and positives, personalized recommendations and lifestyle tips, warning signs to watch for, and a suggested follow-up timeframe, ending with shortcuts to find a nearby doctor or view assessment history.



Figure: Result Screen (Part 2) screen

Articles Screen

The Articles screen offers a scrollable feed of educational cardiac-health content (categorized as Prevention, Nutrition, etc.) with estimated reading time, a short summary, and a link to read the full article.

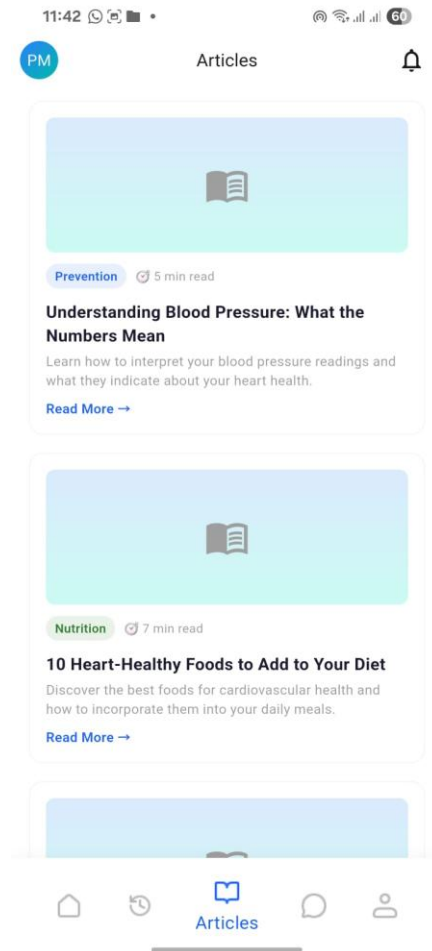


Figure: Articles Screen screen

Article Details Screen

The Article Details screen displays the full content of a selected article, such as an explanation of blood-pressure readings and key takeaways for heart health.

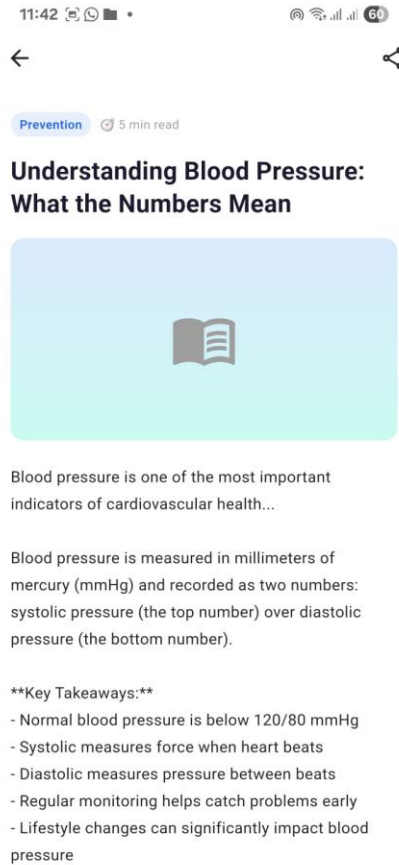


Figure: Article Details Screen screen

Chat Assistant Screen

The Chat screen connects the patient to a Heart Health Assistant that answers questions about heart health, prevention, and lifestyle, offers quick-question suggestions, and supports sending medical files for analysis.

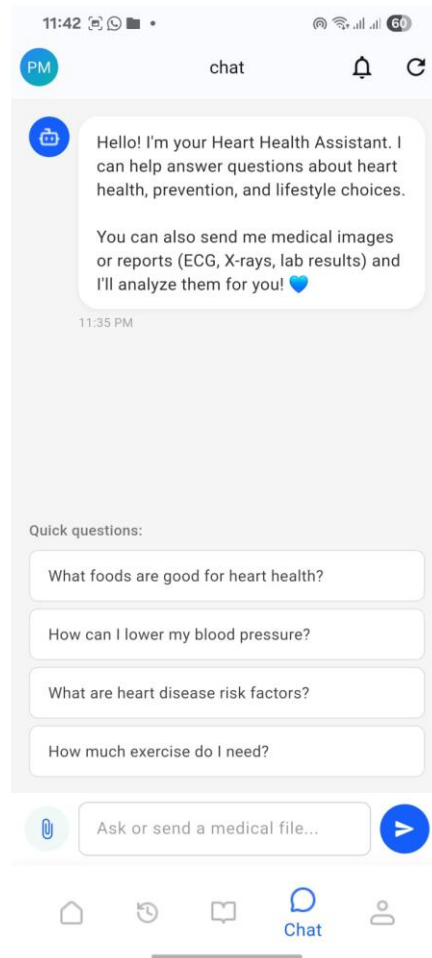


Figure: Chat Assistant Screen screen

Chat Screen: Attach Medical File

Tapping the attachment icon in the chat opens an action sheet allowing the patient to take a photo, choose an image from the gallery, or upload a PDF/image file (ECG, X-ray, or lab results) for analysis, via the `image_picker` and `file_picker` packages.

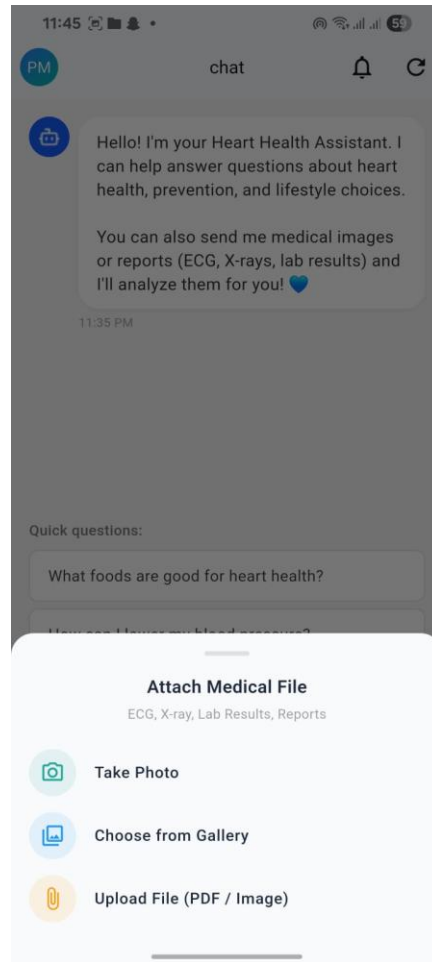


Figure: Chat Screen: Attach Medical File screen

Profile Screen

The Profile screen shows the patient's basic account information and join date, an Edit Profile action, App Settings (notifications, language), and Security & Privacy options including change password, terms & privacy, and log out.

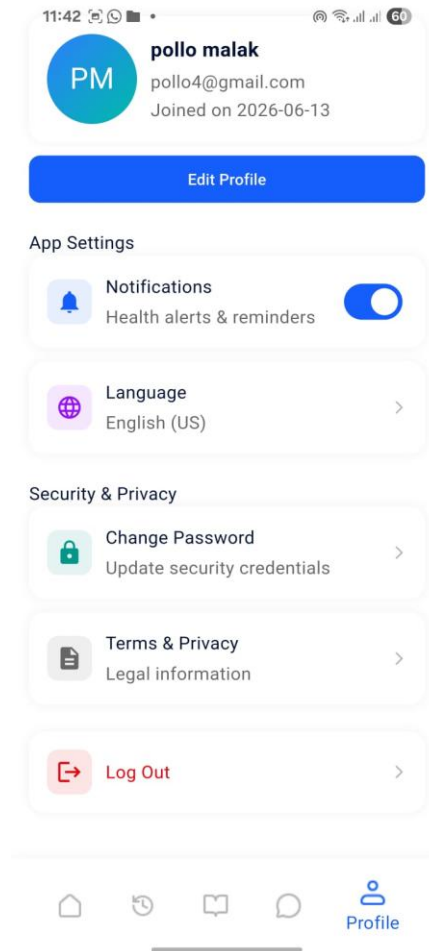


Figure: Profile Screen screen

Edit Profile Screen

The Edit Profile bottom sheet lets the patient update their first name, last name, and email, then save or cancel the changes; on save, the shared MainBloc invalidates the cached profile so the updated data is reflected immediately across the app.

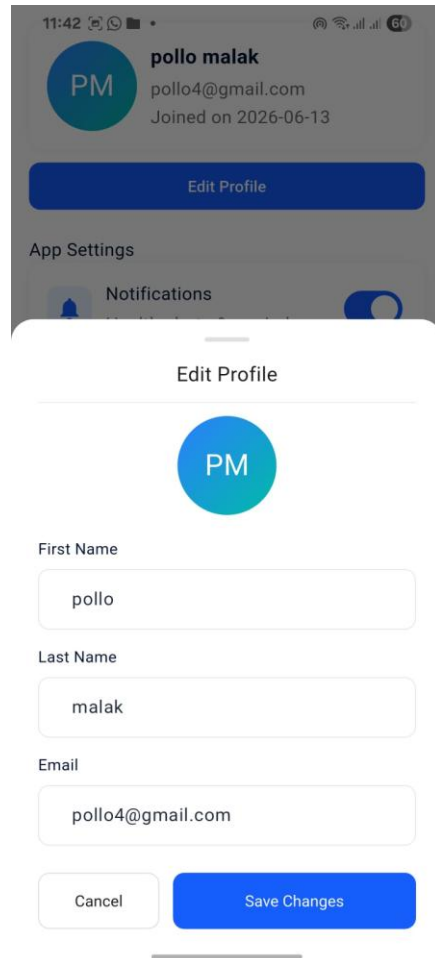


Figure: Edit Profile Screen screen

Language Settings Screen

11:42

60



Language

Suggested

☐ Arabic

☒ English

Figure: Language Settings Screen screen

7. Features of HeartGuard

HeartGuard, built with Flutter, offers a focused set of features to support patients in monitoring their heart health:

- **Account & Profile:** Patients create an account and manage their profile (name, email) and app settings such as notifications and language, with changes reflected instantly across the app.
- **Heart Health Assessment:** A guided flow for entering vital signs (blood pressure, blood sugar, cholesterol) and lifestyle information, submitted to the backend for risk evaluation.
- **ML-Generated Risk Result:** Displays the calculated risk probability and level, AHA-aligned vital sign status, risk factors, recommendations, lifestyle tips, and warning signs to watch for.
- **History & Analytics Dashboard:** Tracks all past assessments with animated `fl_chart` line charts for risk probability, BMI, and blood pressure over time, plus recurring risk factor and warning sign insights.
- **PDF Report Export:** Generates a shareable PDF health status report summarizing trends, assessment history, and health insights, useful for discussing results with a physician.
- **AI Chat Assistant:** Answers heart-health, prevention, and lifestyle questions, and can analyze attached medical files (images, PDFs) such as ECGs, X-rays, or lab results.
- **Educational Articles:** A library of short articles on prevention and nutrition topics, such as understanding blood pressure readings and heart-healthy foods.
- **Multi-Layered Caching:** In-memory and persistent local caching keep the app responsive and usable offline, refreshing from the server only when data changes.

Together, these features make HeartGuard an approachable, informative companion for everyday heart-health awareness, powered by Flutter.

8. Implementation and Responsive Design

8.1 Overview

To deliver a consistent experience across different phone screen sizes, HeartGuard's interface was built with layout flexibility in mind from the start. The goal was to keep dashboards, forms, and the chat interface legible and well-proportioned regardless of the specific device, without relying on a single fixed design.

8.2 Implementation Approach

The interface relies on Flutter's standard layout widgets combined with scrollable and constraint-aware containers, so that content reflows naturally instead of overflowing or being clipped on smaller screens.

8.3 Widgets and Features Used

- **ListView:** used for scrollable content such as the assessment history list and the articles feed, keeping long lists accessible on smaller screens.
- **Column and Row:** used throughout to arrange form fields, vital-sign cards, and dashboard summary tiles flexibly.
- **Padding and SizedBox:** used to keep consistent spacing between cards, charts, and buttons across screens.
- **Text with theme-based styles:** font sizes and weights are defined through the app's theme so headings and body text remain legible and consistent.
- **Icons:** used throughout the bottom navigation bar, quick actions, and settings list to reinforce meaning alongside text.
- **GestureDetector / InkWell:** used for tap interactions such as toggling between the All Assessments and Health Overview tabs, and opening the medical file attachment sheet.
- **fl_chart with responsive sizing:** the risk-trend, BMI, and blood-pressure line charts resize to the available width so they remain readable on different devices.
- **IndexedStack:** used to preserve state across the Home, History, Chat, Articles, and Profile tabs while switching between them through the bottom navigation bar.

8.4 Outcome

Through consistent use of flexible, constraint-aware widgets, HeartGuard achieved an interface that adapts cleanly across different phone sizes, keeping vital health information — charts, risk indicators, and recommendations — clear and easy to read wherever the app is used.

9. Integration with API in HeartGuard

HeartGuard relies on API integration to submit assessments, retrieve profile and history data, and power the chat assistant. This section outlines how the Flutter application communicates with the backend to keep patient data accurate and up to date.

9.1 API Overview

HeartGuard communicates with a RESTful API built with Node.js and Express, backed by MongoDB, and deployed as the app's single backend service. The API handles authentication, patient profile management, health assessment submission and retrieval, and chat messaging.

9.2 Implementation Details

- **HTTP Client:** Dio is the primary tool for API requests due to its interceptors and robust error handling; the http package is used for a small number of simpler calls.
- **Data Handling:** JSON responses are parsed into Dart model classes (e.g., ProfileModel, AssessmentModel) with null-safe parsing to gracefully handle older or incomplete records.
- **Authentication:** the login/register endpoints return a session token that is attached to subsequent requests to access protected routes such as the profile and assessment endpoints.
- **Caching Layer:** successful responses are cached in-memory and in SharedPreferences, so the app can display the latest known data instantly and only refetches when data is stale or explicitly changed.

9.3 API Endpoints

Key endpoints used by the Flutter client are summarized below (base URL omitted for brevity):

Method	Endpoint	Description
POST	/api/auth/login	Authenticates an existing patient and returns a session token.
POST	/api/auth/register	Registers a new patient account.
GET / PUT	/api/patient/profile	Retrieves or updates the signed-in patient's profile information.
POST	/api/health-data/submit	Submits a new heart-health assessment for risk evaluation.
POST	/api/chat/message	Sends a message (optionally with an attached medical file) to the chat assistant.

Method	Endpoint	Description
	<pre>static const String baseUrl = "https://heart-disease-backend- pink.vercel.app"; static const String login = "/api/auth/login"; static const String signUp = "/api/auth/register"; static const String profile = "/api/patient/profile"; static const String submitAssessment = "/api/health- data/submit"; static const String chat = "/api/chat/message";</pre>	

Code: API endpoint constants used by the Dio-based network layer

9.4 Challenges and Solutions

- **Server Availability:** if the backend is unreachable, the app surfaces a clear, user-friendly error message via the `ErrorMessageHandler` utility instead of a raw exception.
- **Data Consistency:** after a profile update, the local cache is invalidated before re-fetching so the UI never shows stale data alongside a successful update.
- **Field Naming Mismatches:** differences between the backend's JSON field naming and the Dart models were resolved with explicit, tolerant parsing in each model's `fromJson` factory.

9.5 Impact on HeartGuard

- Assessments are submitted and evaluated in real time, with results available immediately on the Result screen.
- Profile changes are reflected consistently across the Home, History, and Profile tabs.
- The chat assistant can respond to both text questions and attached medical files through the same messaging endpoint.

9.6 Conclusion

Using Dio as the main HTTP client, HeartGuard connects to a Node.js and MongoDB backend to submit assessments, manage patient profiles, and power the chat assistant, while a multi-layered caching strategy keeps the app fast and usable even under imperfect network conditions.

10. Bloc Management

State management in HeartGuard is implemented using the BLoC (Business Logic Component) pattern. This approach separates the user interface from the business logic, making the application more testable, maintainable, and modular.

10.1 Why BLoC?

- Flutter applications are reactive by nature, and BLoC provides a structured way to manage UI responses to data changes or user interactions.
- BLoC relies on Streams to handle asynchronous data flows, which suits an app like HeartGuard that depends on real-time assessment submissions, chat responses, and history updates.
- A single MainBloc instance, registered once in GetIt, can be shared across the Home, History, and Profile tabs, keeping state consistent while avoiding duplicate network calls.

10.2 BLoC Pattern

The BLoC pattern in HeartGuard follows a consistent flow across all modules:

- **Event:** a user action triggers an event, such as SubmitAssessmentEvent or LoadProfileEvent.
- **BLoC Logic:** the corresponding Bloc receives the event, calls the appropriate repository (which handles caching and API calls), and determines the appropriate response.
- **State Emission:** the Bloc emits a new state, such as ProfileSuccessState or AssessmentResultLoadedState, which BlocBuilder uses to rebuild the UI.

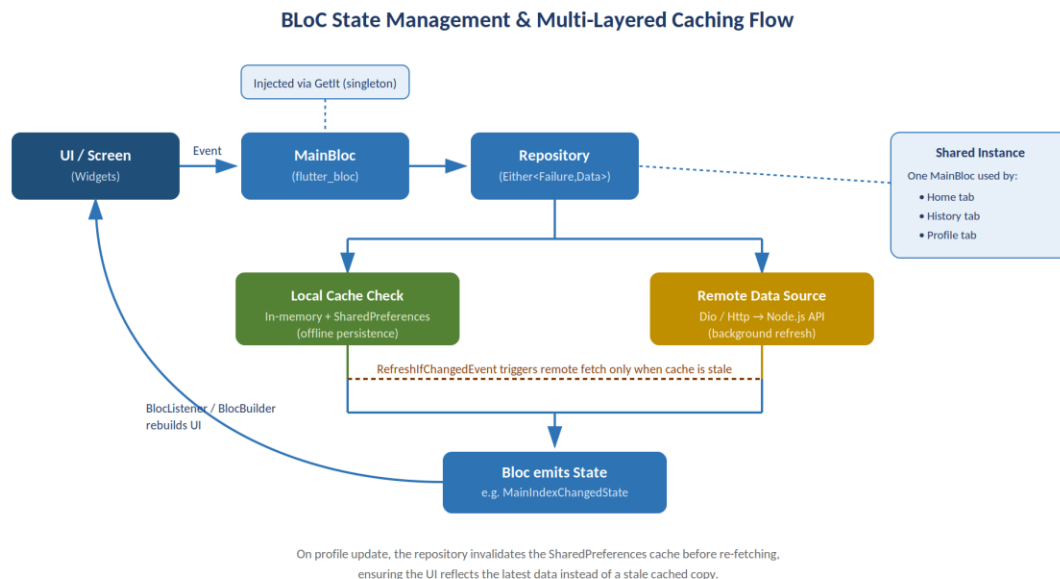


Figure: BLoC state management and multi-layered caching flow in HeartGuard

10.3 Benefits of Using BLoC

- Scalability: new features can be added with minimal changes to existing UI code.
- Debuggability: state transitions are explicit and well-defined, simplifying issue tracking.
- Testing: each Bloc is independent and can be tested without relying on the UI.
- Granular Rebuilds: BlocBuilder's buildWhen lets a widget rebuild only for the states it cares about, keeping shared screens performant.

The snippet below shows how the Home tab's profile section listens only to the profile-related states of the shared MainBloc:

```
return BlocBuilder<MainBloc, MainState>(  
  buildWhen: (previous, current) =>  
    current is ProfileLoadingState ||  
    current is ProfileSuccessState ||  
    current is ProfileErrorState,  
  builder: (context, state) {  
    if (state is ProfileLoadingState) {  
      return const Center(child: MyLoadingWidget());  
    }  
  
    if (state is ProfileErrorState) {  
      return Center(child: Text(state.error));  
    }  
  
    if (state is ProfileSuccessState) {  
      return _HomeContent(state: state);  
    }  
  
    return const SizedBox();  
  },  
);
```

Code: BlocBuilder with buildWhen scoping rebuilds to profile-related states

11. ML Integration for Heart Disease Risk Analysis

11.1 Overview

The core intelligence behind HeartGuard's assessment feature is a dedicated Machine Learning service that analyzes the patient's submitted health data — vital signs, lifestyle habits, and relevant history — and returns a structured heart-disease risk report. This enables the app to turn raw form input into a clear, actionable summary for the patient.

11.2 Problem Statement

Turning structured health data into a trustworthy, easy-to-read risk report required solving a few specific problems:

- Producing a consistent, well-defined output format so the Flutter app can reliably render the same sections (summary, risk factors, recommendations) for every assessment.
- Classifying vital signs such as blood pressure, blood glucose, and cholesterol against recognized medical thresholds rather than leaving them as raw numbers.
- Keeping the risk report understandable and actionable for a non-medical audience.

11.3 Solution

When a patient submits an assessment, the Flutter app sends the vital signs and lifestyle data to the backend's `/api/health-data/submit` endpoint. The backend forwards this data to the Machine Learning service, which returns the predicted risk probability, an overall risk level, and a structured analysis object containing a summary, identified risk factors, positives, recommendations, lifestyle tips, warning signs, and a suggested follow-up timeframe. The backend stores this response in MongoDB and returns it to the Flutter app, which parses it into the assessment result model and renders it on the Result screen.

11.4 Implementation Details

- **Vital Sign Classification:** blood pressure, blood glucose, and cholesterol values are classified against AHA-aligned threshold ranges (e.g., Normal, Elevated, High), and a corresponding level field is always included in the structured response so the UI can render consistent status badges.
- **Structured Output:** the response follows a fixed JSON schema (prediction, probability, risk_level, and an analysis section with summary, risk_factors, recommendations, lifestyle_tips, warning_signs, and follow_up), which the Flutter app maps directly onto the Result screen's sections.
- **Cross-Assessment Insights:** the same structured fields are aggregated across a patient's full assessment history to power the History dashboard's recurring risk factor and persistent warning sign insights.

11.5 Technical Requirements Handled

- Null-Safe Parsing: older assessment records that predate certain fields are parsed gracefully without crashing the Result or History screens.
- Consistent Field Casing: differences between the backend's field naming and the Dart models were resolved in the model parsing layer.
- Threshold Reliability: classification thresholds were refined to remove ambiguity around borderline vital-sign values (see Challenges and Solutions).

11.6 Outcome

The ML integration gives patients an immediate, structured breakdown of their heart-disease risk right after submitting an assessment, and the same structured data powers longer-term trend analysis in the History dashboard — turning a single assessment into an ongoing, informative picture of the patient's heart health.

12. How To Use App

12.1 Users

HeartGuard is designed for a single type of user — the patient — who uses the app to assess their heart-disease risk, track it over time, and learn more about heart health.

12.2 Patients can perform various tasks, including:

- Account Management: creating an account or signing in, and updating profile information and app settings (language, notifications).
- Assessment Submission: entering vital signs and lifestyle information to receive an ML-generated risk result.
- Result Review: reviewing the risk percentage, vital sign status, risk factors, recommendations, and warning signs.
- History Tracking: browsing past assessments and viewing trend charts on the Health Overview dashboard.
- Report Sharing: exporting a PDF health status report to share with a healthcare provider.
- Chat Assistance: asking the chat assistant heart-health questions or sending medical files for analysis.
- Learning: reading educational articles on prevention and nutrition.

The main goal of the app is to make heart-health awareness simple, informative, and encouraging for the patient.

12.3 The Flow for the App is:

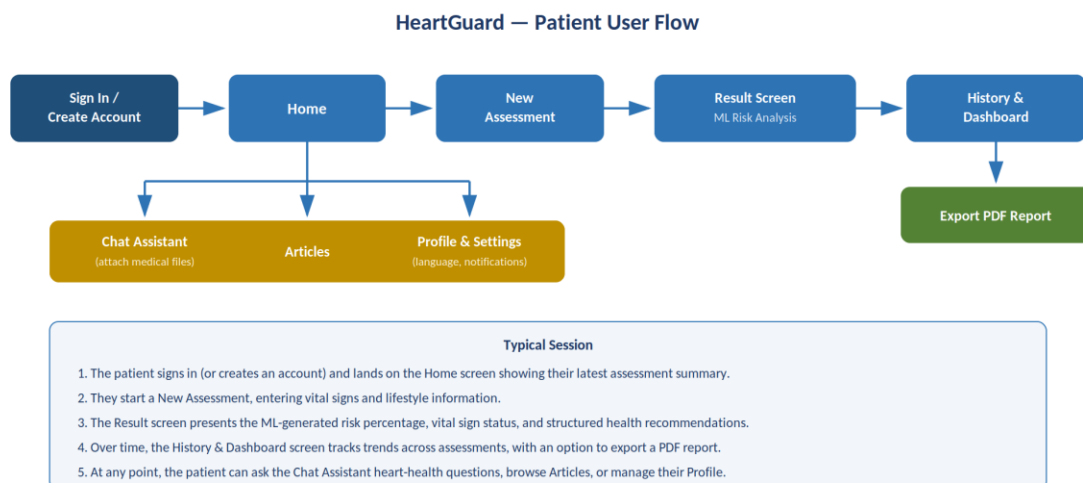


Figure: HeartGuard patient user flow

Chapter 5: Machine Learning Model

Part 1 : Dataset

Dataset Overview

The dataset used in this project is a structured tabular clinical dataset, in which each row represents a single patient and each column represents a measured or recorded attribute for that patient. It brings together demographic information, lifestyle factors, medical history, vital signs, and laboratory measurements, giving a broad clinical picture of each patient rather than relying on any single category of information.

Because each patient record is labeled with a known outcome, indicating whether cardiovascular disease is present or not, this project is framed as a Supervised Machine Learning problem. More specifically, since the outcome takes one of exactly two possible values, the task is a Binary Classification problem, and the dataset described in this chapter forms the basis on which the HeartGuard model is trained and evaluated.

Dataset Source

The dataset was obtained from the official CDC NHANES (National Health and Nutrition Examination Survey) database, accessible at <https://wwwn.cdc.gov/nchs/nhanes/continuousnhanes/>.

NHANES is maintained by the Centers for Disease Control and Prevention (CDC) and is one of the most established public health data collection programs in the United States. Unlike synthetically generated datasets, NHANES contains real-world clinical data gathered through actual physical examinations, laboratory testing, and structured interviews with participants, which gives the underlying measurements a level of clinical credibility that is difficult to replicate with artificial data. For this reason, NHANES is widely used in medical and epidemiological research, including studies specifically focused on cardiovascular disease risk.

To build a dataset that is both larger and more representative of recent patient populations, this project combines the two most recent complete NHANES survey cycles available prior to the latest release, and data from 2017 to 2020 (pre pandemic) and From 2021 to 2023. Combining these two cycles increases the number of available patient records while preserving the reliability that comes with using an official, government-maintained data source.

Dataset Characteristics

Table 2.1 summarizes the key structural characteristics of the dataset used in this project.

Property	Value
Dataset Type	Structured Tabular Dataset
Learning Type	Supervised Learning
Problem Type	Binary Classification
Number of Records	27,493
Number of Input Features	22
Target Variable	HeartDisease

Table 2.1: Summary of dataset structure and problem formulation.

This type of dataset, structured, tabular, and composed of a mix of numerical and categorical clinical features, is well suited to Gradient Boosting. Gradient Boosting builds its predictions from decision trees, which naturally handle tabular data with mixed feature types and do not require the features to follow any particular distribution. This makes it a practical choice for a dataset such as this one, where clinical measurements like blood pressure and cholesterol sit alongside categorical indicators such as medical history and lifestyle habits.

Feature Description

The dataset's 21 input features span five broad categories: demographic information, lifestyle factors, medical history, vital signs, and laboratory measurements. Table 2.2 lists the confirmed clinical features central to cardiovascular risk assessment, including the blood pressure, blood glucose, and cholesterol measurements used by the model.

Feature Name	Description	Data Type
BMI	Body Mass Index, calculated from the patient's height and weight.	Numerical (Continuous)

Feature Name	Description	Data Type
Smoking	Indicates whether the patient has a history of smoking.	Categorical (Binary: 0 = No, 1 = Yes)
AlcoholDrinking	Encoded indicator of the patient's alcohol consumption, following NHANES-style numeric coding (with values 777 and 999 marking "Refused" and "Don't Know" responses).	Categorical (Coded)
BrainStroke	Indicates whether the patient has a history of stroke.	Categorical (Binary: 0 = No, 1 = Yes)
DiffWalking	Indicates whether the patient reports serious difficulty walking or climbing stairs.	Categorical (Binary: 0 = No, 1 = Yes)
Sex	Patient's biological sex, using NHANES coding (1 = Male, 2 = Female).	Categorical (Binary, Coded)
AgeCategory	Patient's age, recorded as a numeric value.	Numerical (Discrete)
Race	Patient's race/ethnicity, using NHANES-coded categories.	Categorical (Nominal, Coded)
Diabetic	Indicates whether the patient has been diagnosed with diabetes.	Categorical (Binary: 0 = No, 1 = Yes)
SleepTime	Average number of hours of sleep per day.	Numerical (Continuous)

Feature Name	Description	Data Type
Asthma	Indicates whether the patient has a history of asthma.	Categorical (Binary: 0 = No, 1 = Yes)
KidneyDisease	Indicates whether the patient has a history of kidney disease.	Categorical (Binary: 0 = No, 1 = Yes)
CancerHistory	Indicates whether the patient has a history of cancer.	Categorical (Binary: 0 = No, 1 = Yes)
ChronicHypertension	Indicates whether the patient has been diagnosed with chronic high blood pressure.	Categorical (Binary: 0 = No, 1 = Yes)
LiverDisease	Indicates whether the patient has a history of liver disease.	Categorical (Binary: 0 = No, 1 = Yes)
ImmunologicalDiseases	Indicates whether the patient has a history of immunological disease.	Categorical (Binary: 0 = No, 1 = Yes)
MyocardialInfarctionInHeart	Indicates whether the patient has a prior history of myocardial infarction (heart attack).	Categorical (Binary: 0 = No, 1 = Yes)
SystolicBP	Systolic blood pressure, measuring the pressure in the arteries while the heart contracts.	Numerical (Continuous)
DiastolicBP	Diastolic blood pressure, measuring the pressure in the arteries between heartbeats.	Numerical (Continuous)
BloodSugar	Instant (point-in-time) blood glucose measurement taken at the time of examination.	Numerical (Continuous)

Feature Name	Description	Data Type
Cholesterol	Total blood cholesterol level.	Numerical (Continuous)
HbA1c	Glycated hemoglobin level, reflecting the average blood glucose concentration over approximately the preceding three months.	Numerical (Continuous)

Target Variable

The target variable for this project is HeartDisease, a binary label indicating each patient's cardiovascular disease status. A value of 0 indicates that the patient does not have heart disease, while a value of 1 indicates that the patient has been identified as having heart disease.

Because HeartDisease can only take one of these two values, the task of predicting it falls under Binary Classification. The model's objective is not to estimate a continuous quantity, but to assign each patient to one of two distinct classes based on their clinical profile, which directly shapes the choice of model architecture, evaluation metrics, and preprocessing strategy used throughout this project.

Dataset Quality

During initial data exploration, the dataset was examined for common data quality issues. This examination identified the presence of duplicate patient records as well as missing values across a number of features. Both issues were addressed as part of the preprocessing stage described in the following chapter; the specific techniques used to handle them are discussed there rather than in this overview.

Why This Dataset?

The NHANES dataset was selected as the foundation for HeartGuard for several reasons. It is built from real clinical data rather than simulated or synthetic records, and it is collected under the oversight of the CDC, giving it a level of methodological rigor that is difficult to match with smaller or less formally governed datasets. NHANES is regarded as one of the world's most trusted public health datasets, and it includes many of the clinical measurements that are recognized as important cardiovascular risk factors, such as blood pressure, cholesterol, and blood glucose.

Before any preprocessing or feature engineering is applied to the HeartGuard dataset, a preliminary exploratory data analysis (EDA) was carried out. It is important to clarify the scope and purpose of this stage from the outset, since it differs somewhat from the exhaustive, open-ended exploration that is often associated with the term EDA. Because the dataset used in this project is NHANES, a professionally collected, real-world medical dataset maintained under the oversight of the CDC, rather than a synthetic or loosely curated dataset, it does not carry the same degree of structural unpredictability that would justify an extensive statistical investigation. NHANES data is gathered through standardized physical examinations, laboratory protocols, and structured interviews, which already imposes a considerable amount of consistency and quality control before the data ever reaches this project.

For this reason, the EDA performed here is intentionally concise. Its purpose is not to uncover deep statistical relationships between variables or to characterize the epidemiological structure of the population, but to validate the quality, structure, and reliability of the dataset before it is handed over to the preprocessing pipeline described in the next chapter. In other words, this chapter functions as a data quality checkpoint rather than a research-oriented exploration, and the findings it produces are used to inform, rather than replace, the more rigorous preprocessing steps that follow.

Dataset Validation

The first step of the EDA process was to verify the overall integrity of the dataset after it was loaded, confirming that the number of records and columns matched expectations and that no structural corruption had occurred during the initial data collection or export process. This included checking that every expected feature described in Chapter 2 was present and correctly labeled.

Alongside this structural check, the data type of each feature was reviewed to confirm that numerical columns, such as BMI, blood pressure, and cholesterol, were represented as numeric types, and that categorical columns, such as Sex and Race, retained their expected coded values. The target variable, HeartDisease, was also examined to confirm that it was distributed across only the two expected classes, with no unexpected or invalid labels present. Together, these checks were used to confirm the overall consistency of the dataset before any preprocessing decisions were made.

Missing Values

As part of the EDA process, the dataset was reviewed for the presence of missing values across its features. This review identified gaps in a number of columns, most notably among the optional laboratory measurements and, in isolated cases, within the AgeCategory feature. Consistent with the concise scope of this chapter, the missing values identified here were not resolved during EDA itself; rather, they were recorded as an observation to be addressed later, during the preprocessing stage described in Chapter 4, where they are handled using an IterativeImputer together with clinically informed default values for the optional laboratory features.

Duplicate Records

The dataset was also checked for the presence of duplicate patient records prior to preprocessing. Identifying duplicates at this stage is important because repeated records can artificially inflate the apparent size of the dataset and bias the model toward patterns that are overrepresented simply due to duplication rather than genuine prevalence in the patient population. Any duplicate records identified during this check were flagged for removal as part of the preprocessing stage, ensuring that the data used for training and evaluation reflects distinct patient observations.

Class Distribution

The distribution of the target variable, HeartDisease, was examined to understand the balance between patients with and without cardiovascular disease. As is common in real-world clinical datasets, the classes were found to be imbalanced, with a smaller proportion of records representing patients diagnosed with heart disease. This observation was made during EDA specifically to inform the model development process, since class imbalance has a direct effect on how the model learns and how its performance should later be evaluated. The specific resampling strategy used to address this imbalance is discussed in Chapter 5.

Summary Statistics

Basic descriptive statistics, including the mean, minimum, maximum, and standard deviation, were reviewed for the dataset's numerical features. This step was used to understand the general ranges of values such as BMI, blood pressure, blood sugar, and cholesterol, and to confirm that these ranges were broadly consistent with clinically plausible values. This review also served as an early indicator of the out-of-range AgeCategory values later addressed in the preprocessing stage, rather than as an attempt to draw detailed statistical conclusions about the patient population.

Part 2 : Data Processing

Cleaning, Feature Engineering, and Preparation of Clinical Data for Model Training

Overview

Before any model can be trained on the HeartGuard dataset, the raw clinical records must be transformed into a clean, consistent, and numerically well-behaved form. Data preprocessing is the stage where this transformation happens, and it is essential for two related reasons. First, machine learning algorithms such as IterativeImputer and Gradient Boosting expect complete, correctly typed, and consistently encoded input, so raw values with missing entries, corrupted records, or mixed representations cannot be fed into the model as they are. Second, in a medical dataset such as this one, careless handling of missing values or outliers carries a clinical cost: an unrealistic or biased value passed into the model can distort the resulting risk estimate in ways that would not be acceptable in a healthcare context.

For these reasons, preprocessing in HeartGuard is implemented as a sequence of fit/transform steps built on top of standard Scikit-learn conventions, rather than as a set of one-off manual operations. Every parameter that the preprocessing stage learns, such as clipping bounds, imputation coefficients, and scaling statistics, is learned exclusively from the training data during `fit()`, then applied identically during `transform()` to validation data, test data, and, ultimately, to new patient records submitted through the Flask API in `app.py`. This chapter describes that pipeline in the order in which it is actually implemented.

Data Inspection

Before designing the preprocessing pipeline, the dataset was inspected for common data quality issues, including missing values, duplicate records, data types, and overall consistency across features. These findings were already summarized in Chapter 2 and are not repeated in detail here; what matters for this chapter is that this inspection directly shaped the preprocessing decisions described in the following sections, in particular the decision to treat the optional laboratory features differently from the core numerical features, and to apply a dedicated data-quality guard to the `AgeCategory` column before any imputation or scaling takes place.

Feature Classification

To apply the correct treatment to each column, the dataset's input features are organized into four functional groups. This classification, defined in `heartguard_transformers.py`, is used consistently throughout the preprocessing pipeline and determines how each column is imputed, scaled, or encoded.

Feature Group	Features	Why It Is Treated Differently
Core Numerical Features	AgeCategory, BMI, SleepTime, AlcoholDrinking	Continuous measurements that are always expected in the input; standardized and imputed jointly with the optional laboratory features.
Optional Laboratory Features	SystolicBP, DiastolicBP, BloodSugar, Cholesterol, HbA1c	Continuous lab results that may be legitimately absent when a test was not performed; require missing-flagging and clinical default filling before scaling.
Binary Features	Sex, Smoking, BrainStroke, DiffWalking, Diabetic, Asthma, KidneyDisease, CancerHistory, ChronicHypertension, LiverDisease, ImmunologicalDiseases, MyocardialInfarctionInHeart	Already encoded as 0/1 indicators; passed through without scaling or additional encoding.
Nominal Features (Race)	Race (5 categories)	An unordered categorical field; requires One-Hot Encoding rather than a single numeric column.

Table 4.1: Feature classification groups used throughout the preprocessing pipeline.

Each group requires a different form of preprocessing because each represents a different kind of information. Continuous numerical values benefit from imputing any gaps and standardizing their scale so that no single measurement dominates the model purely because of its numeric range. Binary features are already in a form the classifier can use directly, so no additional encoding is required. The nominal Race feature, on the other hand, cannot be left as a single numeric column, since doing so would imply an artificial ordering between categories that does not actually exist.

Clinical Feature Engineering

The first step of the deployed pipeline is a custom transformer, `ClinicalFeatureEngineer`, implemented in `heartguard_transformers.py` as a Scikit-learn-compatible class built on `BaseEstimator` and `TransformerMixin`. It replaces what the project's earlier notebooks performed as separate, manual steps outside the pipeline, bringing the Sex remapping, outlier clipping, an

AgeCategory data-quality guard, and the optional-lab missing-flag logic together into a single fit/transform component.

During `fit()`, the transformer learns clipping boundaries for each numerical column from the training data only. During `transform()`, it applies the following operations, in order:

- Cleaning AgeCategory values: any value falling outside the clinically plausible range of 1 to 80 years, including the corrupted export values around $5.4e-79$ identified in Chapter 2, is set to NaN and left for a later imputation step rather than being clipped or treated as a literal near-zero age.
- Creating Missing Flag features: for each of the five optional laboratory tests, a companion `Missing_Flag_<column>` indicator is recorded before any value is filled in, capturing whether that specific test was actually performed for the patient.
- Filling missing laboratory values: once flagged, a skipped laboratory test is filled with a fixed clinically normal reference value defined in `NORMAL_RANGE_DEFAULTS`, rather than a value estimated from the training data.
- Preserving information about missing tests: because the `Missing_Flag_*` columns are appended to the feature set and carried through the rest of the pipeline, the model always has access to whether a laboratory value it sees was an observed measurement or an assumed default.
- Preparing features for the pipeline: the transformer also remaps the dataset's raw Sex codes (1 = Male, 2 = Female) to the model's internal 0/1 encoding, and defensively reindexes incoming columns to the order the pipeline was fitted on, so that features arriving in a different order, such as JSON payloads sent to the Flask API, are still processed correctly.

Encapsulating this logic in a dedicated transformer improves code organization by keeping every custom clinical rule in one place, instead of scattering them across notebooks and application code. Just as importantly, because `ClinicalFeatureEngineer` follows the standard fit/transform interface, it plugs directly into a Scikit-learn (and imblearn) Pipeline object. This means its learned clipping bounds are fitted only once, on training data, and then travel automatically inside the serialized pipeline artifact (`HeartGuard_Final_Model.pkl`), so the exact same feature engineering logic is reapplied, without any risk of drift, whether the pipeline is scoring the test set or a single patient record submitted to `/predict`.

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from heartguard.transformers import (
    NUMERICAL_COLS, BINARY_CATEGORICAL_COLS, MISSING_FLAG_COLS,
    RACE_COL, RACE_CATEGORIES, get_engineered_columns,
)
engineered_cols = get_engineered_columns(X_train.columns)

num_idx = [engineered_cols.index(c) for c in NUMERICAL_COLS]
cat_idx = [engineered_cols.index(c) for c in BINARY_CATEGORICAL_COLS]
missing_flag_idx = [engineered_cols.index(c) for c in MISSING_FLAG_COLS]
race_idx = [engineered_cols.index(RACE_COL)]

column_scaler = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(), num_idx),
        ("cat", "passthrough", cat_idx),
        ("missing_flag", "passthrough", missing_flag_idx),
        ("race", OneHotEncoder(categories=[RACE_CATEGORIES], sparse_output=False,
                                   handle_unknown="ignore"), race_idx),
    ],
    remainder="drop",
)
print(f"Numerical block size: {len(num_idx)}, Binary categorical block size: {len(cat_idx)}, Missing-flag block size: {len(missing_flag_idx)}, Race one-hot block size: {len(race_idx)}")

... Numerical block size: 9, Binary categorical block size: 12, Missing-flag block size: 5, Race one-hot block size: 1
```

Integrating the transformer inside the pipeline guarantees that every prediction follows exactly the same preprocessing sequence used during model training, eliminating inconsistencies between training and deployment.

Handling Missing Values

Missing values are handled differently depending on which feature group they belong to. For the five optional laboratory tests, a skipped measurement is filled with a fixed clinical default rather than a value learned from the data:

Laboratory Test	Clinical Default Value	Clinical Basis
SystolicBP	120.0 mmHg	AHA normal systolic blood pressure
DiastolicBP	80.0 mmHg	AHA normal diastolic blood pressure

Laboratory Test	Clinical Default Value	Clinical Basis
BloodSugar	90.0 mg/dL	Normal fasting glucose level
Cholesterol	180.0 mg/dL	Desirable total cholesterol level
HbA1c	5.4 %	Normal (non-diabetic) HbA1c level

Table 4.2: Clinical default values used to fill skipped laboratory tests.

Each filled value is accompanied by its corresponding `Missing_Flag_*` feature, so the classifier is always able to distinguish an assumed clinical baseline from an actual patient measurement. Any missingness that remains in the core numerical features after this step, most notably in `AgeCategory` once out-of-range values have been set to `NaN`, is handled later in the pipeline by the imputer step, an `IterativeImputer` using a `BayesianRidge` estimator, which jointly models the numerical columns to estimate the remaining gaps.

Clinically normal reference values were preferred over simple statistical imputation for the optional laboratory tests because a skipped test provides no statistical information about what that specific patient's result would have been. Estimating it from the training data's distribution risks anchoring the assumption to the profile of patients who did take the test, which is not necessarily representative of a patient who did not. Filling with an established clinical normal value instead keeps the assumption medically defensible on its own terms, while the paired `Missing_Flag_*` feature lets the model learn how much weight to place on a value it knows to be an assumption rather than an observation.

Handling Outliers

Outlier handling is performed inside the same `ClinicalFeatureEngineer` transformer described in Section 4.4. During `fit()`, clipping boundaries for every numerical column are computed as the 1st and 99th percentiles of the genuinely observed training values, using pandas' `NaN`-skipping `quantile()` function so that rows still missing an optional lab result at this point do not distort the learned bounds. These boundaries are stored as a fitted attribute, `clip_bounds_`, on the transformer.

During `transform()`, each numerical column is clipped to its corresponding learned boundaries using NumPy's `clip()` function. Because the boundaries are computed only once, from the training data, inside `fit()`, and are then reused unchanged whenever `transform()` is called, on the validation set, the test set, or a single record submitted to the API, no information from outside the training

data ever influences how outliers are handled. This prevents Data Leakage and guarantees that every patient record, regardless of when it is processed, is clipped in exactly the same way.

Feature Encoding

Feature encoding is handled inside the `column_scaler` `ColumnTransformer`, which is built to always emit its output columns in a fixed order: the numerical block, followed by the binary categorical block, followed by the `Missing_Flag` block, followed by the Race one-hot block.

- Binary variables (Sex, after remapping, together with the eleven other binary categorical features and the five `Missing_Flag_*` columns) remain encoded as 0 and 1. They are passed through the `ColumnTransformer` unchanged, using the "passthrough" transformer, since they are already in the numeric binary form the classifier expects.
- *Race is transformed using a One-Hot Encoder configured with the fixed category list [1.0, 2.0, 3.0, 4.0, 5.0], producing five separate columns, `Race_1` through `Race_5`, one per category.*

One-Hot Encoding is appropriate for Race because it is a nominal variable: its categories have no natural order or numeric relationship to one another. Leaving Race as a single numeric column would implicitly suggest that the categories can be compared or ranked, for example that category 4 is in some sense "more" than category 2, which a tree-based model such as Gradient Boosting could exploit as a spurious split. Representing each category as its own independent binary column removes this artificial ordering entirely.

One further encoding-related step occurs later in the pipeline: because the `SMOTEENN` resampling step used during training interpolates between neighboring samples, it can leave the binary, `Missing_Flag`, and Race one-hot columns with non-binary intermediate values. A dedicated `CategoricalRounder` step is applied immediately afterward to round these columns back to strict 0/1 values, preserving the categorical encoding described above.

Feature Scaling

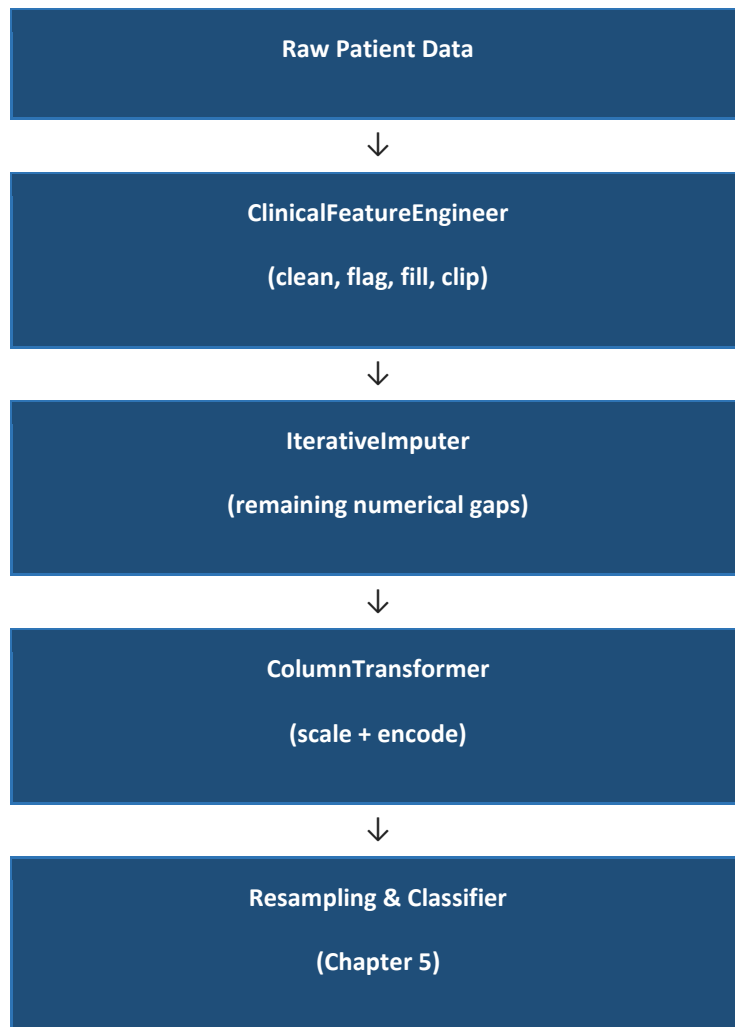
The numerical block, made up of the four core numerical features together with the five optional laboratory features, is standardized using Scikit-learn's `StandardScaler` inside the "num" branch of the `column_scaler` `ColumnTransformer`. As with every other learned preprocessing parameter, the scaler's mean and standard deviation are fitted exclusively on the training data and then applied consistently to any data passed through the pipeline afterward.

Gradient Boosting, the classifier used in this project, does not strictly require feature scaling, since its decision trees split on individual feature thresholds rather than on distances or magnitudes between features. Standardization was nonetheless included as part of the preprocessing pipeline to keep the pipeline consistent end-to-end and to preserve compatibility with other machine

learning models that are sensitive to feature scale, should the classifier be changed in the future without requiring changes to the preprocessing stage itself.

Preprocessing Pipeline Summary

The preprocessing operations described in this chapter are executed as the first stages of a single, unified `imblearn` Pipeline object, in the following order:



Step	Pipeline Component	Operation Performed
1	feature_engineer (ClinicalFeatureEngineer)	Cleans AgeCategory, remaps Sex, creates and fills Missing_Flag_* features for optional labs using clinical defaults, and clips numerical outliers using training-derived bounds.
2	imputer (IterativeImputer)	Jointly imputes any remaining missing values across the numerical columns using a BayesianRidge estimator.
3	column_scaler (ColumnTransformer)	Standardizes numerical features with StandardScaler, passes through binary and Missing_Flag columns, and One-Hot Encodes the Race column.

Table 4.3: Order of operations in the HeartGuard preprocessing pipeline.

Every parameter learned in these three steps, the clipping bounds, the IterativeImputer's fitted estimator, and the StandardScaler's statistics, is fitted exclusively on the training data and stored inside the same pipeline object. Once training is complete, this pipeline is saved as a single artifact (HeartGuard_Final_Model.pkl), so the identical sequence of preprocessing operations described in this chapter is applied automatically to every new patient record before it reaches the classifier, whether during evaluation or through the live /predict endpoint in app.py. The steps that follow preprocessing, class-imbalance resampling and model training, are covered in the next chapter.

Part 3 : Model Selection and Hyperparameter configuration

Comparing Candidate Algorithms and Configuring Gradient Boosting for Clinical Screening

With a clean, fully preprocessed dataset available from the previous chapter, attention turns to the question of which machine learning algorithm is best suited to the HeartGuard prediction task. Rather than committing to a single algorithm from the outset, this project evaluated a set of established classification models under identical experimental conditions, so that the final choice of algorithm could be justified by measured performance rather than by assumption. This chapter documents that comparison, the configuration used for each candidate model, and the reasoning behind the final selection of Gradient Boosting as the classifier used in HeartGuard.

Candidate Machine Learning Models

Six candidate models were selected for evaluation, spanning a range of algorithmic families from simple linear classifiers to ensemble-based tree methods. This range was chosen deliberately, so that the comparison would not be biased toward a single modeling philosophy: Logistic Regression represents a simple, interpretable linear baseline; Random Forest and the three boosting algorithms, Gradient Boosting, XGBoost, and LightGBM, represent tree-based ensemble methods with different training strategies; and the Support Vector Machine represents a margin-based approach capable of modeling non-linear decision boundaries through kernel functions. Table 5.1 summarizes each candidate model, the reason it was considered for this task, and its main advantages and limitations in the context of heart disease classification.

Model	Learning Type	Purpose	Advantages	Limitations
Logistic Regression	Supervised (Linear)	Establish a simple, interpretable baseline for binary classification.	Fast to train; coefficients are easy to interpret; performs well when relationships are approximately linear.	Struggles to capture non-linear interactions between clinical features.
Random Forest	Supervised (Ensemble, Bagging)	Capture non-linear relationships using an	Robust to outliers and mixed feature types; less prone to overfitting	Can be less precise than boosting methods on structured tabular data; larger model size.

Model	Learning Type	Purpose	Advantages	Limitations
		ensemble of decision trees.	than a single decision tree.	
Support Vector Machine (SVM)	Supervised (Margin-Based)	Evaluate a margin-based classifier capable of modeling complex decision boundaries.	Effective in high-dimensional feature spaces; kernel functions allow non-linear separation.	Computationally expensive on larger datasets; less naturally interpretable; sensitive to feature scaling.
Gradient Boosting	Supervised (Ensemble, Boosting)	Sequentially correct prediction errors to improve accuracy on structured clinical data.	Strong predictive performance on tabular data; handles mixed feature types without requiring a specific data distribution.	Training is sequential and can be slower than bagging methods; sensitive to noisy data if left unchecked.
XGBoost	Supervised (Ensemble, Boosting)	Compare against a highly optimized, regularized implementation of gradient boosting.	Efficient training; built-in regularization can reduce overfitting; widely used in tabular classification tasks.	Larger number of internal settings than simpler models; behavior can be less transparent.
LightGBM	Supervised (Ensemble, Boosting)	Compare against a boosting implementation optimized for training speed on large tabular datasets.	Fast training on large datasets; efficient memory usage; competitive predictive performance.	Can be more sensitive to small or noisy datasets; leaf-wise growth can overfit if not controlled.

Table 5.1: Candidate machine learning models considered for heart disease classification.

Each of these models is well established for tabular, structured classification problems of the kind presented by the HeartGuard dataset, which made them reasonable candidates for a systematic comparison rather than a purely theoretical selection.

Model Comparison Strategy

To ensure that the comparison between candidate models reflected genuine differences in algorithmic performance rather than differences in experimental setup, every model was trained and evaluated under exactly the same conditions. Specifically, all six candidates used:

- The same NHANES-derived dataset described in Chapter 2, with no differences in the underlying patient records between models.
- The same preprocessing pipeline described in Chapter 4, so that every model received input that had been cleaned, imputed, encoded, and scaled in an identical way.
- The same train/test split, ensuring that no model gained an advantage from seeing a more favorable partition of the data.
- The same evaluation metrics, including Accuracy, Recall, and ROC-AUC, so that performance could be compared on a like-for-like basis.

Holding these factors constant isolates the algorithm itself as the only varying element in the comparison. This guarantees that any observed difference in performance between candidate models can be attributed to the modeling approach rather than to inconsistencies in how the data was prepared or evaluated, which is essential for drawing a defensible conclusion about which algorithm is best suited to the task.

Hyperparameter Configuration

Before describing how each candidate model was configured, it is useful to distinguish between model parameters and hyperparameters, since the two are often confused. Model parameters, such as the coefficients of a Logistic Regression model or the split thresholds inside a decision tree, are values that the model learns automatically from the training data. Hyperparameters, by contrast, are configuration settings that are chosen before training begins, such as the number of trees in an ensemble or the maximum depth allowed for each tree, and they are not learned from the data but instead control how the learning process itself behaves.

In this project, each candidate model was trained using the predefined set of hyperparameters provided by its corresponding machine learning implementation. These default configurations represent reliable baseline settings that are commonly used in practice and have already been tuned by the library authors to perform reasonably well across a wide range of typical datasets. Using these baseline configurations was sufficient for the purpose of this phase, since the objective at this stage was to compare candidate algorithms against one another under fair, consistent conditions, rather than to exhaustively optimize every hyperparameter of every model before a single algorithm had even been chosen.

This decision also reinforces the fairness of the comparison described in Section 5.2: because every model was left at its respective library-provided baseline configuration, no single candidate was given an advantage through additional, model-specific tuning that was not equally applied to the others. Once the

most suitable algorithm was identified through this comparison, further optimization effort could then be focused specifically on that model, which is the approach followed in the remainder of this chapter and in the results discussed later in this documentation.

Final Model Selection

After training all six candidate models under the identical experimental conditions described in Section 5.2, their performance was compared using the actual results obtained from the trained models, rather than assumed from theory. Table 5.2 reports Accuracy, Precision, Recall, F1-Score, and ROC-AUC for all six candidates at their default classification threshold, together with a Voting Ensemble of the six models included as an additional point of reference.

Final Evaluation Summary (New Results Only)						
Sorted by Recall → F1 Score → Accuracy (descending).						
Rank	Model	Accuracy	Precision	Recall	F1 Score	ROC AUC
1	Logistic Regression	0.8873	0.2898	0.8423	0.4312	0.9527
2	Random Forest	0.9133	0.3443	0.7849	0.4787	0.9540
3	SVC	0.9094	0.3333	0.7849	0.4679	0.9386
4	Voting Ensemble	0.9434	0.4630	0.7168	0.5626	0.9552
5	Gradient Boosting	0.9527	0.5259	0.6918	0.5975	0.9567
6	XGBoost	0.9553	0.5487	0.6667	0.6019	0.9526
7	LightGBM	0.9565	0.5610	0.6595	0.6063	0.9519

Although Logistic Regression achieved the highest Recall (0.8423), it was not selected as the final production model because model selection was based on overall clinical performance rather than on a single evaluation metric.

Since HeartGuard is a clinical screening system rather than a diagnostic tool, its primary design objective is to minimize False Negatives, that is, to miss as few genuinely at-risk patients as possible. At this stage, however, model selection is concerned with identifying which of the six candidate algorithms provides the strongest foundation for that objective once deployed, rather than with fixing the final Recall/Precision trade-off itself; the specific operating point used to prioritize Recall in production is determined separately, through the threshold optimization process described in Chapter 7.

Gradient Boosting was selected as the final classifier for HeartGuard because it combined one of the strongest overall discriminative performances among all six candidates with a well-balanced profile across the remaining metrics and probability outputs well suited to later threshold optimization. Gradient Boosting achieved an Accuracy of 95.27%, a Precision of 52.59%, a Recall of 69.18%, an F1-Score of 59.75%, and the highest ROC-AUC (95.67%) among all evaluated models, confirming excellent discrimination between patients with and without heart disease independent of any single decision threshold. Because ROC-AUC reflects a classifier's ranking quality across every possible threshold, this result also indicated that Gradient Boosting produced well-calibrated probability estimates, meaning it could support a deliberately lowered, clinically motivated decision threshold rather than being reliable only at the conventional 0.5 cutoff. Although LightGBM and XGBoost achieved marginally higher Accuracy and F1-Score at their respective default thresholds, a dedicated comparison against XGBoost, the strongest remaining boosting candidate, across a sweep of thresholds (presented in Chapter 7) showed that Gradient Boosting consistently achieved higher Recall at every operating point evaluated. Since Recall is the metric most directly tied to HeartGuard's screening objective of minimizing false negatives, this consistent advantage, rather than performance at any single default threshold, was the deciding factor in carrying Gradient Boosting forward into threshold optimization and deployment.

Furthermore, Gradient Boosting was the classifier used throughout the complete HeartGuard pipeline, including feature importance analysis, SHAP-based explainability, model serialization, threshold optimization, and deployment within the Flask application. Maintaining the same classifier across training, interpretation, and deployment ensured methodological consistency while avoiding unnecessary engineering changes for only marginal performance gains.

This reasoning is examined further in the Results chapter, where the deployed decision threshold and the trade-off between Recall and Precision are discussed in the context of the final production system.

Transition to Model Training

With Gradient Boosting selected as the final classifier, the next stage of the project moves from comparing candidate algorithms to working with this single chosen model in depth. This involves training the final Gradient Boosting classifier on the preprocessed data, generating probability predictions for each patient, and, subsequently, optimizing the clinical decision threshold applied to those probabilities. These steps, along with the detailed evaluation results that motivated the threshold optimization strategy, are described in the following chapter.

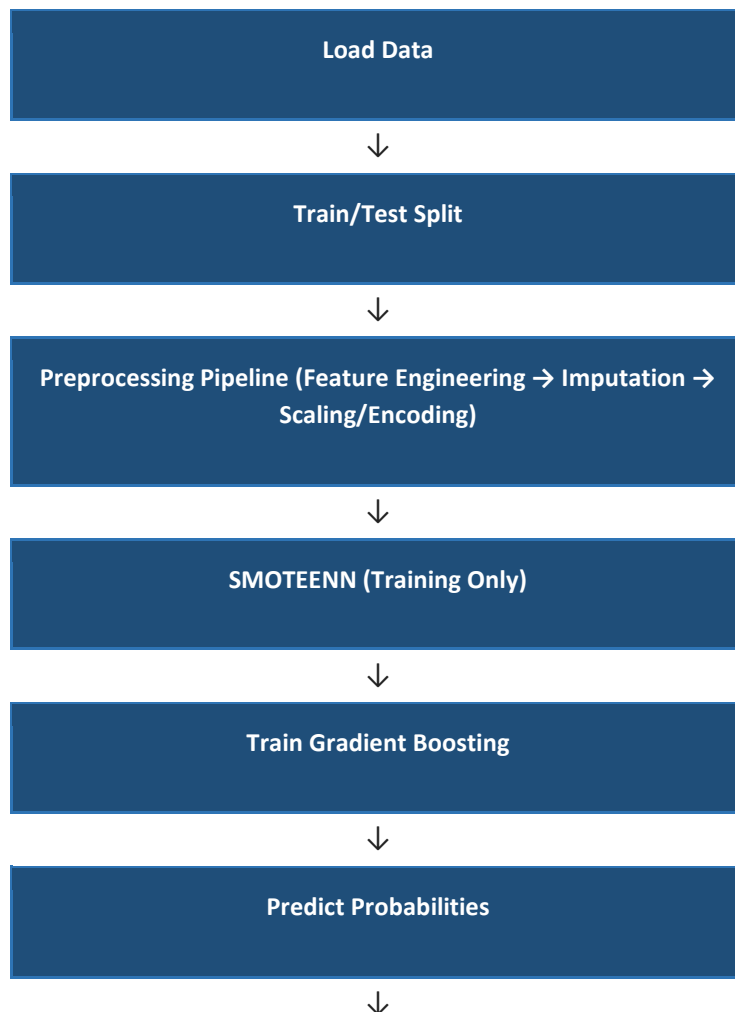
Part 4 : Model Training

Fitting the Unified Pipeline and the Final Gradient Boosting Classifier

This chapter describes how the final HeartGuard classifier is trained, following exactly the sequence of operations implemented in the project's model-training notebook. Every step described below corresponds directly to a cell in that notebook; no additional experiments, parameters, or preprocessing steps beyond those actually present in the code are introduced here.

Training Workflow

Model training in HeartGuard is carried out as a single, ordered sequence of operations, beginning with the raw dataset and ending with a fully evaluated classifier. Figure 6.1 summarizes this workflow in the exact order in which it is executed in the notebook.



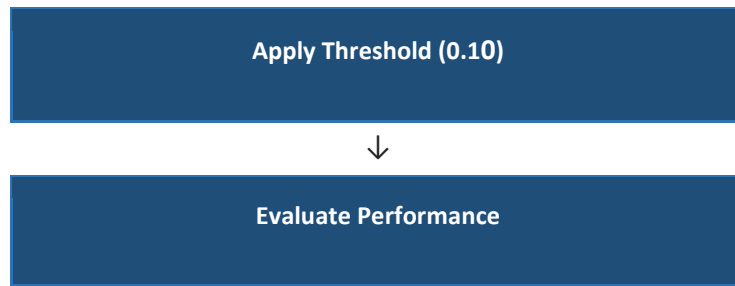


Figure 6.1: Model training workflow, in the exact order implemented in the notebook.

An important detail of this workflow is that preprocessing, class balancing, and classifier training are not implemented as separate, manually sequenced steps. Instead, they are combined into a single `imblearn Pipeline` object, `final_pipeline`, which is fitted once, on the training data only, using `final_pipeline.fit(X_train, y_train)`. Each stage of this workflow is described in the sections that follow.

Data Loading

Training begins by loading `HeartGuardDatasetFinal.csv`, which does not itself contain the `HbA1c` laboratory feature. The `HbA1c` column is instead read from a second source file, `HeartGuard_Final_23_Features_Dataset.csv`, and attached to the main dataframe. Before this merge is trusted, the notebook verifies that both files contain the same number of rows and that several shared fields, including `BMI`, `SystolicBP`, and `AgeCategory/Age`, agree exactly between the two files, confirming that the two files are row-aligned before the `HbA1c` values are copied across.

Immediately after the `HbA1c` column is attached, a data-quality guard is applied to `AgeCategory`: because a small number of rows contain a corrupted, denormalized value (of the order of $5.4e-79$) introduced by an earlier export step, any `AgeCategory` value outside the plausible range of 1 to 80 is replaced with `NaN`. This guard is applied at this stage so that the corrupted values are treated as missing data and resolved later by the `IterativeImputer` inside the training pipeline, rather than being allowed to distort the model as extreme numeric outliers.

Train-Test Split

Once the `HeartDisease` target column has been separated from the feature set, the data is split into a training set and a testing set using Scikit-learn's `train_test_split`, with `test_size=0.2`, `stratify=y`, and `random_state=42`. This produces a training set of 21,994 patient records and a held-out test set of 5,499 records, with the same proportion of at-risk and not-at-risk patients preserved in both subsets through stratification.

The test set is set aside at this point and is not touched again until the final evaluation step described in Section 6.8. This is essential because it allows the model's performance to be measured on patient records it has never encountered during training, imputation, scaling, or

class balancing, which is the only way to obtain a realistic estimate of how HeartGuard would behave on new patients in practice.

Training Pipeline

All preprocessing operations are implemented as stages of a single `imblearn Pipeline` object, `final_pipeline`, which is fitted once on the training data. The pipeline executes its stages in the following fixed order:

- `feature_engineer` — the `ClinicalFeatureEngineer` transformer described in Chapter 4, which cleans `AgeCategory`, remaps `Sex`, creates the `Missing_Flag_*` indicator columns, fills the optional laboratory features with clinical default values, and clips numerical outliers.
- `imputer` — an `IterativeImputer` using a `BayesianRidge` estimator, with `max_iter=50` and `random_state=42`, which resolves any remaining missing values across the numerical columns, including the `AgeCategory` values set to `NaN` in Section 6.2.
- `column_scaler` — a `ColumnTransformer` that applies a `StandardScaler` to the 9-column numerical block (the 4 core numerical features together with the 5 optional laboratory features), passes the 12-column binary categorical block and the 5-column `Missing_Flag_*` block through unchanged, and applies a `OneHotEncoder` to the `Race` column, producing a 5-column one-hot block.
- `smoteenn` — a `SMOTEENN` resampler, with `sampling_strategy=0.5` and `random_state=42`, applied at this point in the pipeline so that it acts only on the training data, combining synthetic oversampling of the minority `HeartDisease` class with an editing step that removes ambiguous samples.
- `rounder` — a `CategoricalRounder` transformer that restores the binary and one-hot encoded columns to valid 0/1 values after `SMOTEENN`, which can otherwise produce non-binary interpolated values for these columns.
- *classifier* — the final `GradientBoostingClassifier`, configured with `n_estimators=300`, `max_depth=5`, and `random_state=42`.

Because `SMOTEENN` is included as a stage inside this pipeline, it is applied automatically only when the pipeline is fitted on the training data; it plays no role when the fitted pipeline is later used to transform the test set or a new patient record, both of which are only ever transformed, never resampled.

To ensure consistency between model training and deployment, all preprocessing, feature engineering, data balancing, and classification steps were integrated into a single self-contained `imblearn Pipeline`. This guarantees that the exact same transformations applied during training are automatically applied during inference inside the deployed Flask application.


```
from sklearn.experimental import enable_iterative_imputer
from sklearn.impute import IterativeImputer
from sklearn.linear_model import BayesianRidge
from sklearn.ensemble import GradientBoostingClassifier
from imblearn.pipeline import Pipeline as ImbPipeline
from imblearn.combine import SMOTEENN
from heartguard.transformers import ClinicalFeatureEngineer, CategoricalRounder

final_pipeline = ImbPipeline([
    ("feature_engineer", ClinicalFeatureEngineer(numerical_cols=NUMERICAL_COLS)),
    ("imputer", IterativeImputer(estimator=BayesianRidge(), max_iter=50, random_state=42)),
    ("column_scaler", column_scaler),
    ("smoteenn", SMOTEENN(sampling_strategy=0.5, random_state=42)),
    ("rounder", CategoricalRounder(n_numerical=len(num_idx),
                                  n_categorical=len(cat_idx) + len(missing_flag_idx) +
                                  len(missing_flag_idx))),
    ("classifier", GradientBoostingClassifier(n_estimators=300, max_depth=5, random_state=42))
])

print("Training the unified, self-contained pipeline...")
final_pipeline.fit(X_train, y_train)
print("Pipeline trained successfully.")

... Training the unified, self-contained pipeline...
Pipeline trained successfully.
```

The final pipeline performs the following operations sequentially:

- Clinical feature engineering.
- Missing value imputation using Iterative Imputer.
- Numerical scaling and categorical preprocessing.
- Class balancing using SMOTEENN.
- Restoration of categorical values after resampling.
- Gradient Boosting classification.

Exporting the entire pipeline as a single object ensures that both training and deployment use identical preprocessing logic, preventing inconsistencies between development and production.

Model Training

The classifier itself is a GradientBoostingClassifier with `n_estimators=300`, `max_depth=5`, and `random_state=42`, positioned as the final stage of `final_pipeline`. Calling `final_pipeline.fit(X_train,`

y_train) fits every preceding stage, feature engineering, imputation, scaling and encoding, and SMOTEENN, on the training data, and then trains this classifier on the resulting, class-balanced, fully numeric representation of the training set.

During this stage, the model builds its 300 decision trees sequentially, with each tree learning to correct the errors made by the trees trained before it, allowing the classifier to learn the combinations of demographic, lifestyle, medical history, vital sign, and laboratory features that are associated with cardiovascular disease in the training population.

Probability Prediction

After training, the fitted final_pipeline is used to generate predictions on the held-out test set via final_pipeline.predict_proba(X_test)[:, 1], which returns, for every test patient, a probability that they belong to the at-risk HeartDisease class, rather than an immediate 0/1 label.

Working with probabilities at this stage, rather than allowing the classifier to assign a hard label internally, is what makes it possible to apply the clinical decision threshold described in Section 6.7 as a separate, explicit step. This keeps the trained model itself unchanged while allowing the point at which a patient is flagged as at risk to be set deliberately, according to the clinical priorities of a screening tool, rather than left at whatever default boundary the classifier would otherwise use.

Applying the Clinical Threshold

The trained Gradient Boosting classifier produces, for each patient in the test set, a predicted probability of cardiovascular disease. These probabilities are converted into final class labels by comparing each probability with a predefined decision threshold using:

```
y_pred = (y_prob >= threshold).astype(int)
```

For the final deployed version of HeartGuard, the threshold is set to 0.10, meaning that any patient with a predicted probability of cardiovascular disease greater than or equal to 0.10 is classified as at risk.

The threshold value is stored together with the trained pipeline inside the exported `model_package` object and saved as `HeartGuard_Final_Model.pkl` using `joblib.dump()`. Packaging the classifier and its decision threshold within a single deployment artifact guarantees that the deployed application applies exactly the same decision rule used during the final evaluation, ensuring consistency between offline testing and real-world inference.

The rationale for selecting the 0.10 threshold is intentionally deferred to Chapter 7, where multiple operating thresholds are compared as part of the model evaluation process

Model Evaluation

After applying the deployment threshold of 0.10, the classifier is evaluated on the independent held-out test set described in Section 6.3. This dataset was not used during feature engineering, missing-value imputation, class balancing, or model training, providing an unbiased estimate of the model's predictive performance.

Evaluation is performed using the true labels (`y_test`), the predicted probabilities (`y_prob`), and the final predicted class labels (`y_pred`) generated using the deployed threshold. Performance is quantified using Accuracy, Precision, Recall, F1-Score, ROC-AUC, the Confusion Matrix, and the ROC Curve.

This chapter has described the complete training workflow of the final Gradient Boosting pipeline, from data preprocessing through probability estimation and the generation of final predictions using the deployment threshold. The resulting performance metrics, threshold analysis, and clinical interpretation are presented in the following chapter.

Part 5 : Evaluation Metrics and Results

Introduction

The HeartDisease target variable in this project is imbalanced, as established during the exploratory data analysis in Chapter 3: not-at-risk patients considerably outnumber at-risk patients. In an imbalanced binary classification problem, a single metric such as Accuracy can be misleading, since a model can score well on Accuracy while still failing to identify most genuinely at-risk patients. For this reason, HeartGuard is evaluated using multiple complementary metrics rather than any one figure in isolation. Consistent with the medical screening purpose of this project, Recall was treated as the primary optimization objective, since failing to flag a genuinely at-risk patient is clinically more costly than raising an unnecessary follow-up check.

Evaluation Metrics

Table 7.1 reports the five evaluation metrics obtained for the final Gradient Boosting classifier, at the 0.10 decision threshold used in the deployed HeartGuard system.

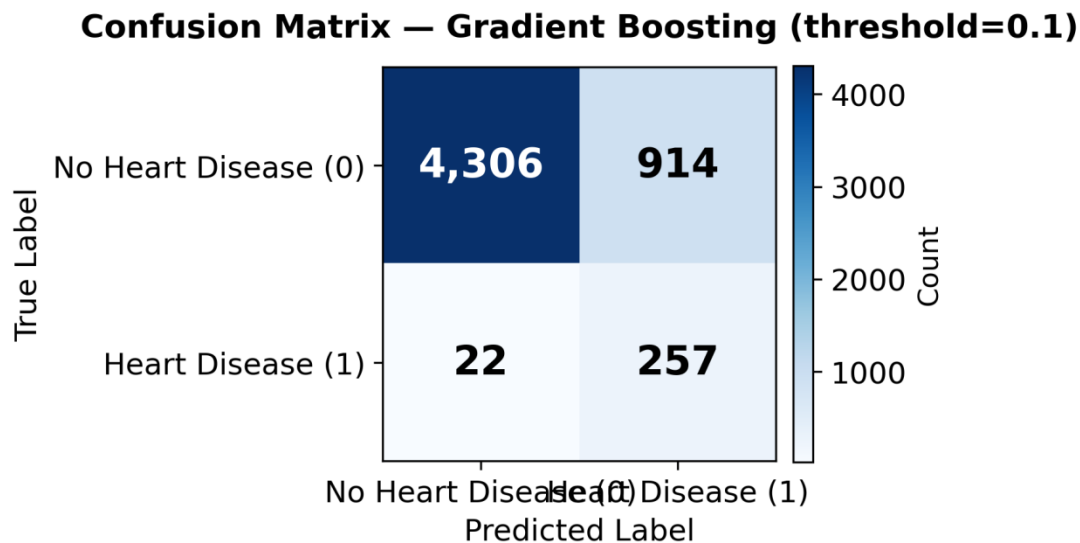
Metric	Value
Accuracy	82.98%
Precision	21.95%
Recall	92.11%
F1-Score	35.45%
ROC-AUC	95.67%

Accuracy measures the overall proportion of patients correctly classified as either at-risk or not at-risk. Precision measures the proportion of patients predicted as at-risk who are truly at risk. Recall measures the proportion of genuinely at-risk patients that the model successfully identifies. F1-Score represents the harmonic mean of Precision and Recall, providing a single measure of their balance. ROC-AUC evaluates the model's ability to distinguish between the two classes across all possible decision thresholds, making it independent of the specific threshold used during deployment. Because HeartGuard is designed as a clinical decision-support system for early cardiovascular risk screening, Recall was treated as the primary ranking criterion to

reduce the likelihood of false negatives, where an at-risk patient is incorrectly classified as healthy, with F1-Score, Accuracy, and ROC-AUC considered as secondary criteria to keep the classifier clinically reliable rather than optimizing Recall in isolation. At the deployed 0.10 threshold, this priority is reflected directly in the final results: Recall reaches 92.11%, while Accuracy (82.98%) and Precision (21.95%) are comparatively modest. This is an intentional, accepted trade-off rather than a weakness of the classifier: in a screening context, correctly flagging as many at-risk patients as possible is considered more valuable than minimizing the number of healthy patients referred for an unnecessary follow-up check. Accuracy alone is insufficient because of the dataset's class imbalance; a model biased toward the majority class can still achieve a high Accuracy while failing to detect a substantial number of true heart disease cases. The ROC-AUC of 95.67%, by contrast, is unaffected by this threshold choice: it summarizes the model's discriminative ability across every possible operating point and confirms that the underlying classifier remains excellent independent of where the decision boundary is set for deployment.

Results

Confusion Matrix

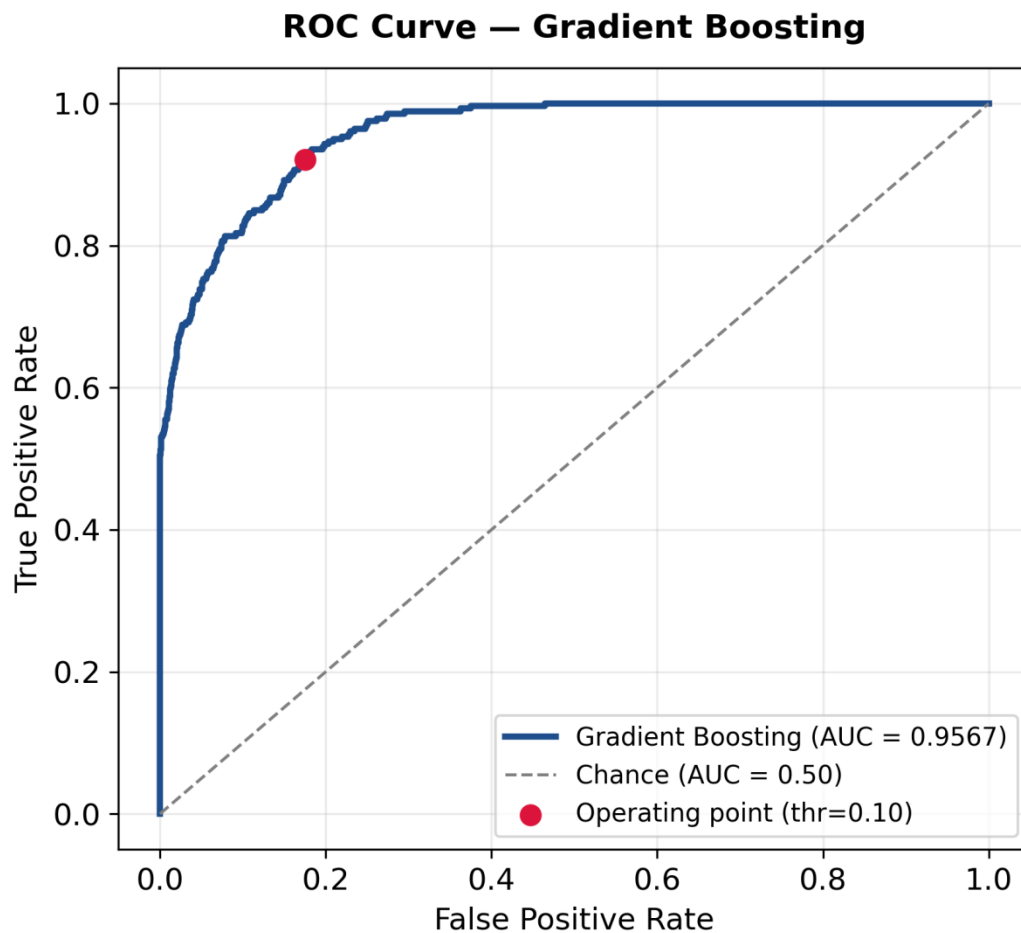


presents the confusion matrix of the final Gradient Boosting classifier at the deployed decision threshold (0.10). At this operating point, the model correctly identified 257 patients with heart disease (True Positives) while missing only 22 positive cases (False Negatives), demonstrating a high sensitivity for cardiovascular risk screening. Of the remaining not-at-risk patients, 4,306 were correctly classified as healthy (True Negatives), while 914 were incorrectly flagged as at-risk (False

Positives). This improvement in Recall was achieved by accepting a larger number of false positives, an intentional trade-off considered clinically appropriate for an early screening system, where recommending an additional medical examination is generally less harmful than failing to identify an at-risk patient.

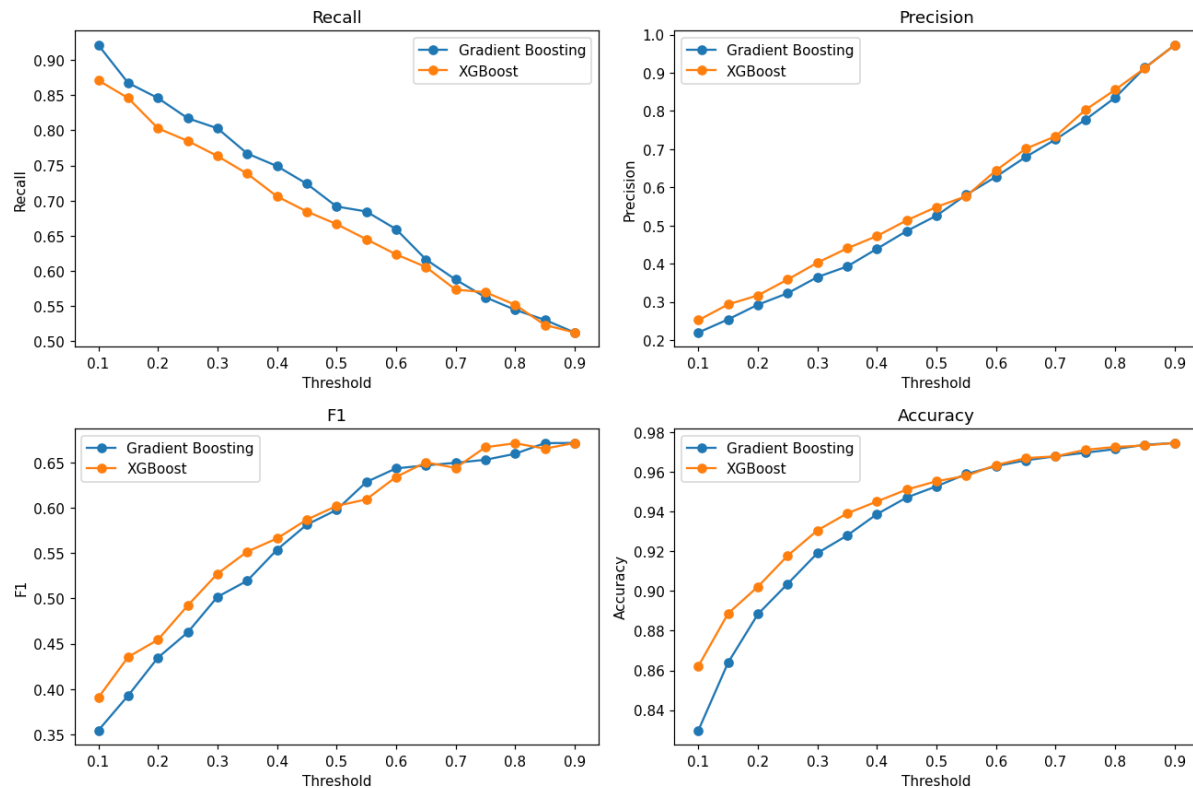
ROC Curve

shows the Receiver Operating Characteristic (ROC) curve of the final Gradient Boosting classifier. The model achieved an ROC-AUC of 0.9567, indicating excellent discrimination between patients with and without heart disease across all possible decision thresholds. The highlighted operating point corresponds to the deployed threshold (0.10), selected after threshold optimization to prioritize Recall while maintaining strong overall discrimination. This confirms that the classifier provides reliable probability estimates, allowing the deployment threshold to be adjusted according to the clinical objective of minimizing missed heart disease cases.



Model Selection and Threshold Justification

As established in Chapter 5, Gradient Boosting was retained as the final classifier because it combined one of the strongest overall discriminative performances of all six candidates (the highest ROC-AUC observed) with a well-balanced profile across the remaining metrics, and because its probability outputs were well suited to the deliberate threshold adjustment a screening tool like HeartGuard requires. Figure 7.2 supports this choice further by comparing Gradient Boosting directly against XGBoost, the strongest of the remaining boosting-based candidates, across a sweep of decision thresholds from 0.10 to 0.90.



Across every threshold evaluated in this analysis, Gradient Boosting consistently achieved higher Recall than XGBoost, indicating that it identified a larger proportion of genuinely at-risk patients at every operating point. This behavior directly supports the selection of Gradient Boosting as the final classifier, as the primary objective of HeartGuard is to minimize false negatives in a clinical screening setting.

The threshold analysis also guided the selection of the deployed 0.10 decision threshold. As expected, lowering the threshold increased Recall while reducing Precision, F1-Score, and Accuracy. Among all thresholds evaluated, 0.10 achieved the highest Recall (92.11%), correctly identifying 257 of the 279 at-risk patients in the test set, at the cost of a lower Accuracy (82.98%) and Precision (21.95%) than higher thresholds would have produced. Since HeartGuard is intended as an early screening and decision-support system rather than a definitive diagnostic tool, this trade-off was considered clinically appropriate: missing a genuinely at-risk patient (a false negative) is considered substantially more costly than referring a healthy patient for an

additional, ultimately unnecessary medical examination (a false positive). On this basis, Gradient Boosting evaluated at the 0.10 threshold was adopted as the final deployed configuration of HeartGuard.

Taken together, the evaluation metrics, confusion matrix, and threshold analysis demonstrate that the final Gradient Boosting classifier provides excellent discrimination ability (ROC-AUC = 0.9567) while achieving the highest sensitivity at the deployed threshold (0.10). This operating point best aligns with the project's objective of maximizing the detection of at-risk patients while accepting an appropriate increase in false-positive referrals.

Key Findings

- The final Gradient Boosting classifier achieves a Recall of 0.9211 at the deployed 0.10 threshold, correctly identifying 257 of the 279 genuinely at-risk patients in the test set.
- Its ROC-AUC of 0.9567 confirms strong discrimination ability between at-risk and not-at-risk patients, independent of the specific threshold used in deployment.
- The confusion matrix shows the expected trade-off of a low decision threshold: few False Negatives (22) alongside a larger number of False Positives (914), consistent with prioritizing Recall over Precision in a screening context.
- Accuracy (0.8298) and Precision (0.2195) are lower than they would be at a higher threshold, which is an accepted and deliberate consequence of optimizing for Recall rather than a weakness in the model itself.
- Across a full sweep of thresholds, Gradient Boosting maintains higher Recall than XGBoost at every point tested, reinforcing the model selection made in Chapter 5.
- The 0.10 threshold was chosen because it achieved the highest Recall of all thresholds evaluated in the sweep, maximizing the detection of at-risk patients; the resulting reduction in Precision and Accuracy was accepted as a deliberate, clinically motivated trade-off consistent with HeartGuard's role as an early screening tool rather than a definitive diagnostic system.

Part 6 : Feature Importance

Global Interpretation of the Final Gradient Boosting Classifier

Introduction

With the final Gradient Boosting classifier trained, evaluated, and exported in Chapters 6 and 7, attention now turns to a different question: not how well the model performs, but which of its input features drive that performance. This chapter presents a global feature importance analysis computed directly from the trained classifier itself, rather than from any external estimation method. Because the model selected for HeartGuard, a scikit-learn `GradientBoostingClassifier`, is a tree-based ensemble, it natively exposes a `feature_importances_` attribute once fitted, making this analysis a direct inspection of the deployed model rather than a separate approximation of it.

This chapter is intentionally limited to global, model-level interpretation. The feature importance ranking presented here is later reused by the application's rule-based clinical interpretation layer to prioritize which patient-specific risk and protective factors are presented first. The ranking itself does not generate explanations; rather, it provides a global measure of each feature's relative contribution to the trained classifier and serves as a priority list for the rule-based interpretation module described in the system implementation chapter.

Computing Feature Importance from the Trained Classifier

Feature importance values were obtained by accessing the `feature_importances_` attribute of the classifier step inside the fitted `final_pipeline` (`final_pipeline.named_steps["classifier"]`), the same fitted object that is evaluated in Chapter 7 and exported to `HeartGuard_Final_Model.pkl` in Chapter 6. No separate model was trained for this analysis.

For a `GradientBoostingClassifier`, this attribute reports, for each input feature, the total reduction in the splitting criterion (Gini impurity) that the feature is responsible for across every split in every one of the ensemble's 300 decision trees, normalized so that the importances of all features sum to 1. A higher value indicates that the feature was chosen more often, and produced larger impurity reductions, when the trees decided how to partition patients into more and less homogeneous risk groups.

Because the classifier is the final stage of an imblearn Pipeline, it does not see the 22 raw input columns directly; it sees the 31-column array produced by the `column_scaler` `ColumnTransformer` described in Chapter 4, ordered as the 9 numerical features, followed by the 12 binary categorical

features, the 5 Missing_Flag_* indicators, and finally the 5 one-hot encoded Race columns. This fixed layout is documented as FEATURE_ORDER in heartguard_transformers.py, the same reference used to:

This fixed layout is documented as FEATURE_ORDER in heartguard_transformers.py, allowing each importance value returned by the classifier to be correctly matched to its corresponding engineered feature rather than to an anonymous array position.

output in the notebook, and it is what allows each of the 31 raw importance values returned by the classifier to be correctly matched back to a named, interpretable feature rather than an anonymous array position.

Top 10 Most Important Features

Figure 9.1 shows the ten features with the highest importance values out of the full set of 31 engineered features seen by the classifier. Table 9.1 reports the corresponding numeric values

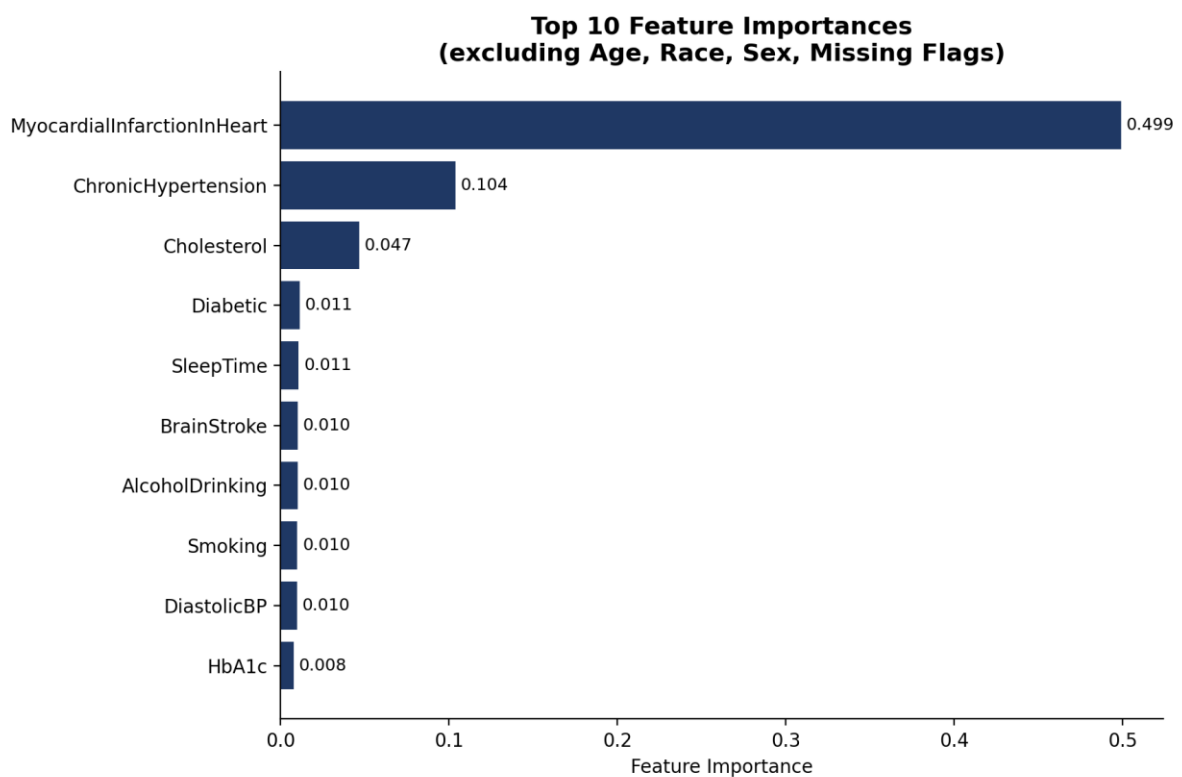


Figure 9.1: Top 10 global feature importances obtained directly from the trained Gradient Boosting classifier. These rankings are later reused by the rule-based clinical interpretation layer to prioritize patient-specific explanations.

Rank	Feature	Relative Importance
1	MyocardialInfarctionInHeart	0.4994
2	AgeCategory	0.2192
3	ChronicHypertension	0.1039
4	Cholesterol	0.0466
5	Race_3	0.0217
6	Diabetic	0.0113
7	SleepTime	0.0108
8	BrainStroke	0.0102
9	AlcoholDrinking	0.0102
10	Smoking	0.0100

Table 9.1: Top 10 features by Gini-based importance, as reported by the trained Gradient Boosting classifier.

Interpretation and Clinical Relevance

The ranking in Table 9.1 is dominated by a small number of features. *MyocardialInfarctionInHeart* alone accounts for approximately half of the classifier's total importance (0.4994). This is consistent with its clinical meaning: a documented prior heart attack is one of the strongest and most direct indicators of cardiovascular disease available in the dataset, and it is unsurprising that a model built to detect *HeartDisease* relies heavily on it. Because this feature is so closely related to the target by definition, its dominant ranking should be read as a reflection of that clinical relationship rather than evidence that the remaining features are unimportant to real-world risk.

AgeCategory (0.2192) and ChronicHypertension (0.1039) form the second tier of influence. Both are well-established cardiovascular risk factors in the clinical literature: cardiovascular risk rises steadily with age as a result of cumulative vascular changes, and chronic hypertension is a recognized direct contributor to heart disease through its sustained mechanical burden on the heart and blood vessels. Together with the first-ranked feature, these three variables account for the large majority of the model's total importance, which indicates that the classifier's predictions are driven primarily by a compact set of clinically recognizable risk indicators rather than being spread diffusely across many weak signals.

Cholesterol (0.0466) is the highest-ranked of the optional laboratory features and is likewise consistent with established cardiovascular risk knowledge, given the well-documented relationship between elevated cholesterol and atherosclerotic disease. Race_3 (0.0217) appears among the top 10 as a demographic factor; because Race is one-hot encoded, its individual categories can carry different importance values, and this result should be read descriptively, as a pattern the model found in this dataset, rather than as a causal claim about race and cardiovascular risk. The remaining features in the top 10, Diabetic, SleepTime, BrainStroke, AlcoholDrinking, and Smoking, contribute smaller but non-negligible shares of importance, and each is independently recognized as a cardiovascular risk factor or health behavior in the clinical literature, which lends face validity to the model's learned ranking even at these lower magnitudes.

Part 7 : Model Saving and Interface Pipeline

From a Trained Pipeline to a Live Prediction in the Flask Application

Introduction

Chapters 6 through 9 described how the final Gradient Boosting classifier is trained, evaluated, and interpreted. None of that work is useful to a patient or clinician, however, until it can be reused without retraining, every time a new prediction is requested. This chapter describes how the trained artifacts produced by the model-optimization notebook are persisted to disk, how the Flask application (`app.py`) loads them, and the sequence of steps that takes a single patient's data from an incoming request through to a completed JSON response.

Model Serialization

Training the `final_pipeline` object, together with the Heart Age regression model, is a computationally expensive process: it involves fitting an `IterativeImputer`, resampling the training data with `SMOTEENN`, and growing 300 sequential decision trees, among other steps. Model serialization is the process of saving this already-fitted object, with every learned parameter it contains, to a file on disk so that it can be loaded back into memory and reused for inference, without repeating any of the training computation. This separation between training time and serving time is what allows the Flask application to answer a prediction request in a fraction of a second rather than retraining a model for every patient.

The project uses `joblib`, rather than Python's built-in `pickle` module, to perform this serialization. Both model files, `HeartGuard_Final_Model.pkl` and `HeartGuard_Heart_Age_Model.pkl`, are written with `joblib.dump()` in the model-optimization notebook and read back with `joblib.load()` in `app.py`. `joblib` is the serialization approach recommended in scikit-learn's own documentation for persisting fitted estimators, because it is built to handle objects that internally hold large NumPy arrays, such as a fitted `StandardScaler`'s learned statistics or a tree ensemble's split thresholds, more efficiently than the general-purpose `pickle` module. A further, project-specific reason is visible directly in `app.py`: reconstructing the saved objects at load time depends on several classes and modules being importable in the loading environment before `joblib.load()` is called, including scikit-learn's `IterativeImputer` (gated behind an `experimental-enable` flag), the `imblearn` package (needed to resolve the pipeline's `Pipeline` and `SMOTEENN` objects), and the project's own `ClinicalFeatureEngineer` and `CategoricalRounder` classes from `heartguard_transformers.py`. `app.py` performs these imports at the top of the file, before either `joblib.load()` call, precisely so that this reconstruction can succeed.

Saved Artifacts

HeartGuard_Final_Model.pkl

This artifact stores a dictionary with three keys: "model", the fully fitted final_pipeline object (covering every preprocessing step from Chapter 4 together with the trained Gradient Boosting classifier); "threshold", the fixed decision threshold of 0.15 established in Chapter 6; and "feature_columns", the ordered list of the 22 raw input column names that the pipeline was fit on. It is saved as a single bundle, rather than as three separate files, so that the model, its decision threshold, and its expected input schema can never drift out of sync with one another once deployed. In app.py, this file is loaded once at startup: "model" is used directly for model.predict_proba() on new patient data, since every preprocessing step already lives inside the pipeline; "threshold" is compared against the predicted probability to produce a binary prediction; and "feature_columns" is used to reindex each incoming record into the exact column order the pipeline expects, regardless of the order in which fields arrive in a request.

HeartGuard_Heart_Age_Model.pkl

This artifact stores a dictionary with two keys: "model", the RandomForestRegressor trained in Chapter 6 on rows where HeartDisease is 0, learning the relationship between a healthy patient's clinical numerical features and their age; and "features", the list of eight numerical columns (all of NUMERICAL_COLS except AgeCategory) the regressor was fit on. It is saved separately from HeartGuard_Final_Model.pkl because it represents a distinct predictive task, estimating a patient's "Heart Age" rather than classifying cardiovascular risk, trained on a different subset of rows and with a different target variable. In app.py, it is loaded once alongside the main model, and at inference time a patient's numerical features, after passing through the same fitted feature_engineer used by the classifier, are restricted to the saved "features" list and passed to heart_age_model.predict() to produce the patient's estimated Heart Age.

Model Loading

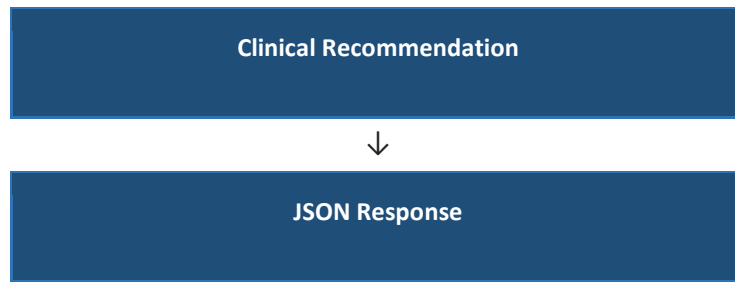
Both artifacts are loaded exactly once, at the top of app.py, when the Flask application starts, rather than being loaded inside the /predict route itself. HeartGuard_Final_Model.pkl is loaded first, and its "model", "threshold", and "feature_columns" values are unpacked into module-level variables; HeartGuard_Heart_Age_Model.pkl is loaded immediately afterward in the same way. A SHAP TreeExplainer is also constructed once at this point, directly from the already-loaded classifier (model.named_steps["classifier"]), rather than being saved to or loaded from its own pickle file, since TreeExplainer objects are inexpensive to build and constructing one fresh at startup avoids any fragility that could arise from unpickling a SHAP explainer created with a different SHAP library version.

Loading these artifacts once at startup, instead of on every incoming request, is what makes the application's inference latency practical. Deserializing a pipeline of this size involves file I/O and reconstructing every fitted object it contains; performing that work once, when the server starts, means every subsequent call to `/predict` reuses the same in-memory model, threshold, feature order, Heart Age model, and SHAP explainer, rather than repeating that cost for every patient. This design also fails safely: if either `.pkl` file is missing or cannot be unpickled, the corresponding `joblib.load()` call raises an exception while the application is starting up, which stops the server from ever accepting requests with a broken or mismatched model, rather than allowing it to serve incorrect predictions silently.

Inference Pipeline

Once the application is running, a single call to the `/predict` endpoint moves a patient's data through a fixed sequence of stages before a response is returned. Figure 10.1 summarizes this sequence; each stage is described in turn below.





Patient Data

The pipeline begins when a client sends an HTTP POST request to `/predict` with a JSON body. This body may contain either a single patient record or a list of records, allowing the same endpoint to score one patient or a batch in one request.

Input Validation

The `_prepare_input()` function first converts any categorical `AgeCategory` range label (for example, "45-49") into its numeric midpoint, since the pipeline was fit on a numeric `AgeCategory` value; an unrecognized category raises a `ValueError`. It then checks that every required column, meaning every one of `feature_columns` except the five optional laboratory columns, is present in the incoming data, raising a `ValueError` listing any missing fields. Optional laboratory columns (`SystolicBP`, `DiastolicBP`, `BloodSugar`, `Cholesterol`, and `HbA1c`) may be omitted or sent as null if the corresponding test was not performed; any missing optional column is added as an all-null column rather than causing a validation failure. Any `ValueError` raised during this stage is caught by the `/predict` route and returned to the client as an HTTP 400 response containing the error message, rather than being allowed to reach the model.

Feature Preparation

Once validated, the record is assembled into a `DataFrame` and reindexed to `feature_columns`, the exact column order the pipeline was fit on. From this point, the `DataFrame` is ready to enter `final_pipeline` directly: the `ClinicalFeatureEngineer`, `IterativeImputer`, and `column_scaler` stages described in Chapter 4 execute automatically as the first stages of the loaded pipeline whenever it is called, so no separate preprocessing code needs to be repeated inside `app.py`.

Prediction

The prepared row is passed to `model.predict_proba()`, returning the predicted probability of the positive class. This probability is compared against the loaded threshold (0.15) to produce a binary prediction, and is also passed to `get_risk_band()` (defined in `heartguard_transformers.py`) to obtain a Low, Moderate, or High risk-band label.

Clinical Recommendation

The capped, SHAP-ranked feature list is used to look up a specialist referral corresponding to the patient's single leading risk factor, defaulting to a General Practitioner if no significant modifiable

factor remains. The same feature list, together with the patient's current probability, their previous stored probability, and the resulting trend, is used to generate a clinical action plan: an AHA-aligned baseline recommendation that may optionally be rewritten for a more natural tone, falling back automatically to the baseline text if that rewriting step is unavailable.

JSON Response

Finally, the probability, binary prediction, threshold, risk band, Heart Age, trend information, the SHAP-derived top features (each annotated with its impact percentage), the action plan, the specialist recommendation, and the list of any skipped laboratory tests are assembled into a single JSON object and returned to the client via `jsonify()`. For a batch request, a list of such objects is returned, one per patient record. Two further routes, `/history/<patient_id>` and `/health`, exist alongside `/predict` to retrieve a patient's past assessments and to report basic service status respectively; their underlying storage is outside the scope of this chapter.

Chapter Summary

This chapter described how the artifacts produced by the model-optimization notebook are carried into production. The trained pipeline and Heart Age model are serialized with `joblib` into two self-contained `.pkl` files, each bundling everything needed to use the corresponding model at inference time. The Flask application loads both files, and builds a SHAP explainer from the loaded classifier, once at startup, so that this cost is paid only once rather than on every request. Each call to `/predict` then moves a patient's data through a fixed sequence of validation, feature preparation, prediction, Heart Age estimation, SHAP explanation, and clinical recommendation stages before a single JSON response is returned, connecting the modeling work of the preceding chapters to a live, patient-facing prediction.

Chapter 6: Backend API

The system is designed with security as a foundational concern, implementing multiple layers of protection to safeguard sensitive patient health data while maintaining reliability and performance.

Purpose

- Allow patients to register and manage their health profiles securely
- Accept detailed health data and forward it to a Flask ML model for prediction
- Return risk predictions, blood sugar levels, cholesterol levels, and AI analysis
- Provide a conversational heart-health AI assistant powered by Groq and OpenRouter

6.1 Technology Stack

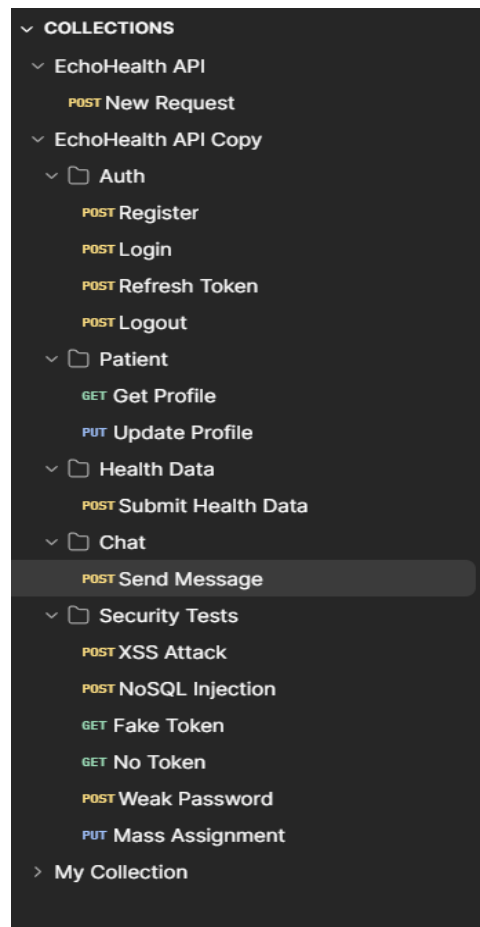
Component	Technology
Runtime	Node.js
Framework	Express.js
Database	MongoDB (via Mongoose)
Authentication	JWT (jsonwebtoken) + bcryptjs
ML Integration	Flask REST API (Python)
AI Chat	Groq API (LLaMA 3) + OpenRouter (Vision)
Logging	Winston (file + console)
Validation	express-validator

6.2 Project Structure

The project follows a standard MVC-style architecture with clear separation between routes, controllers, models, middleware, validators, and utilities. Each layer has a single responsibility, making the codebase easy to maintain, test, and extend.

```
echohealth-backend/
├── config/db.js           MongoDB connection
├── controllers/
│   ├── authController.js   Register, Login, Refresh, Logout
│   ├── patientController.js Get & Update profile
│   ├── healthDataController1.js Submit health data → Flask ML
│   └── chatController.js   AI heart-health assistant
├── middleware/
│   ├── authMiddleware.js   JWT verification
│   ├── security.js         Rate limiters + slow-down
│   └── validateRequest.js   express-validator error handler
├── models/
│   ├── Patient.js         Patient schema + lockout fields
│   └── HealthData.js       Health data + prediction results
├── routes/
│   ├── authRoutes.js
│   ├── patientRoutes.js
│   ├── healthDataRoutes.js
│   └── chatRoutes.js
├── utils/
│   ├── logger.js          Winston logger
│   └── responseHandler.js   Unified response format
├── validators/
│   ├── authValidator.js
│   ├── patientValidator.js
│   └── healthDataValidator.js
├── server.js              Entry point
└── .env                   Environment variables (never committed)
```

Base URL: http://localhost:5001. All endpoints return JSON. Protected endpoints require a valid Bearer Token in the Authorization header. The Postman collection screenshot below shows all available requests organized by category.



6.3 Authentication Endpoints

Method	Endpoint	Auth	Description
POST	/api/auth/register	No	Register a new patient account
POST	/api/auth/login	No	Login and receive access + refresh tokens
POST	/api/auth/refresh	No	Get a new access token using a refresh token
POST	/api/auth/logout	No	Invalidate the refresh token (server-side logout)

POST /api/auth/register

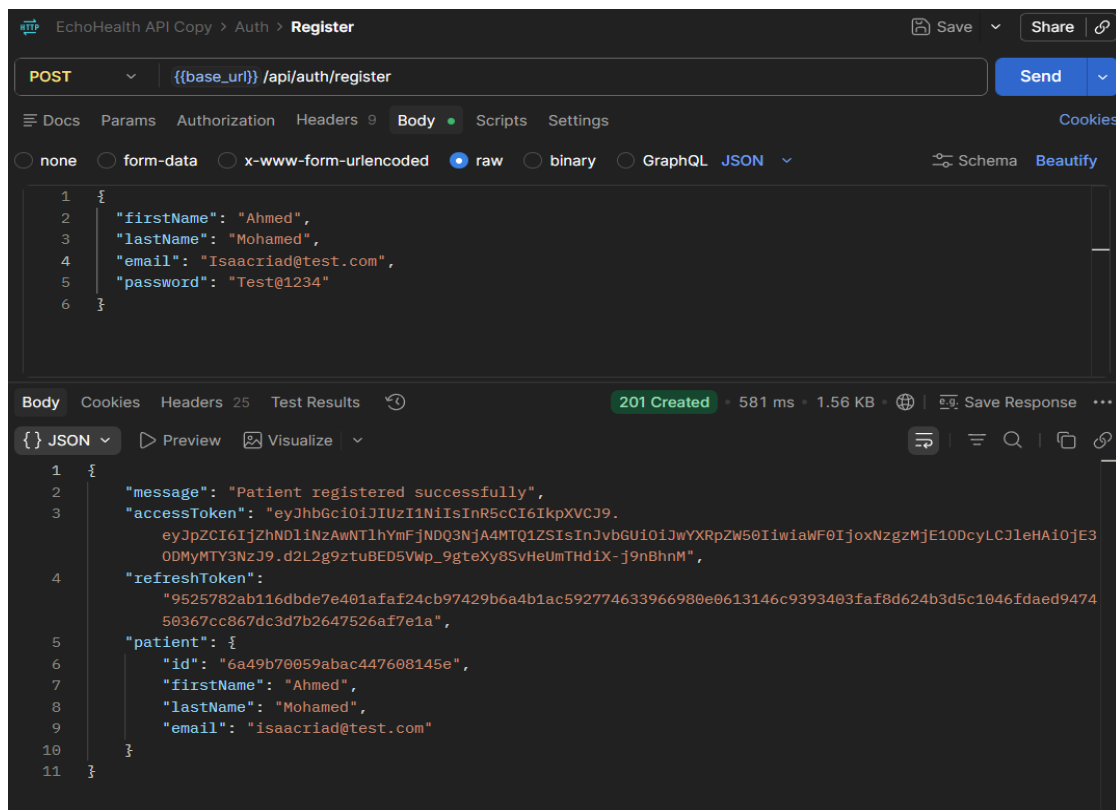
Registers a new patient. All fields are validated before saving. The password is hashed with bcrypt at cost factor 12 before being stored — the plain-text password never touches the database.

Request Body:

```
{ "firstName": "Ahmed", "lastName": "Mohamed", "email": "ahmed@test.com",  
  "password": "Test@1234" }
```

Response 201: Returns accessToken, refreshToken, and the patient object (without password field).

Validation rules: firstName and lastName max 50 characters, letters only. Password minimum 8 characters and must contain at least one uppercase letter, one lowercase letter, one digit, and one special character.



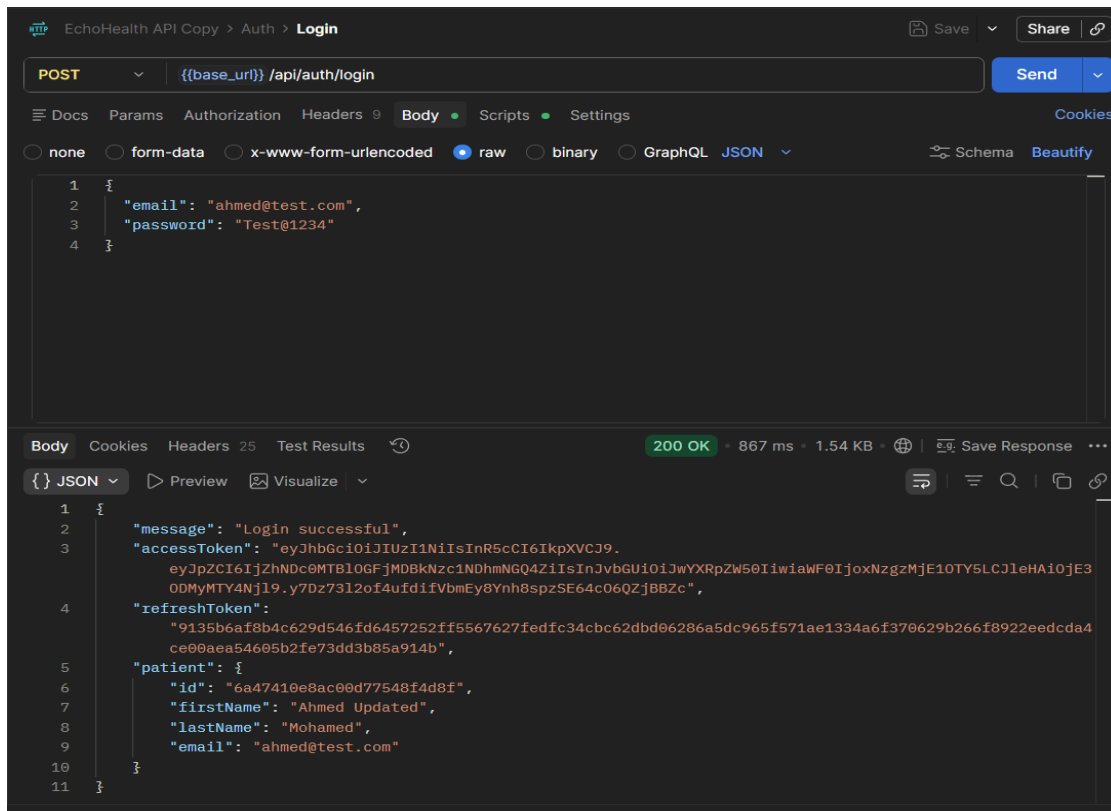
POST /api/auth/login

Authenticates a patient and issues a short-lived access token (15 minutes) and a long-lived refresh token (7 days). The account is locked for 15 minutes after 5 consecutive failed attempts. A timing-safe dummy bcrypt comparison runs even when the email is not found, preventing user-enumeration timing attacks.

Request Body:

```
{ "email": "ahmed@test.com", "password": "Test@1234" }
```

Response 200: Same shape as register. Returns 423 Locked if the account is currently locked.



POST /api/auth/refresh

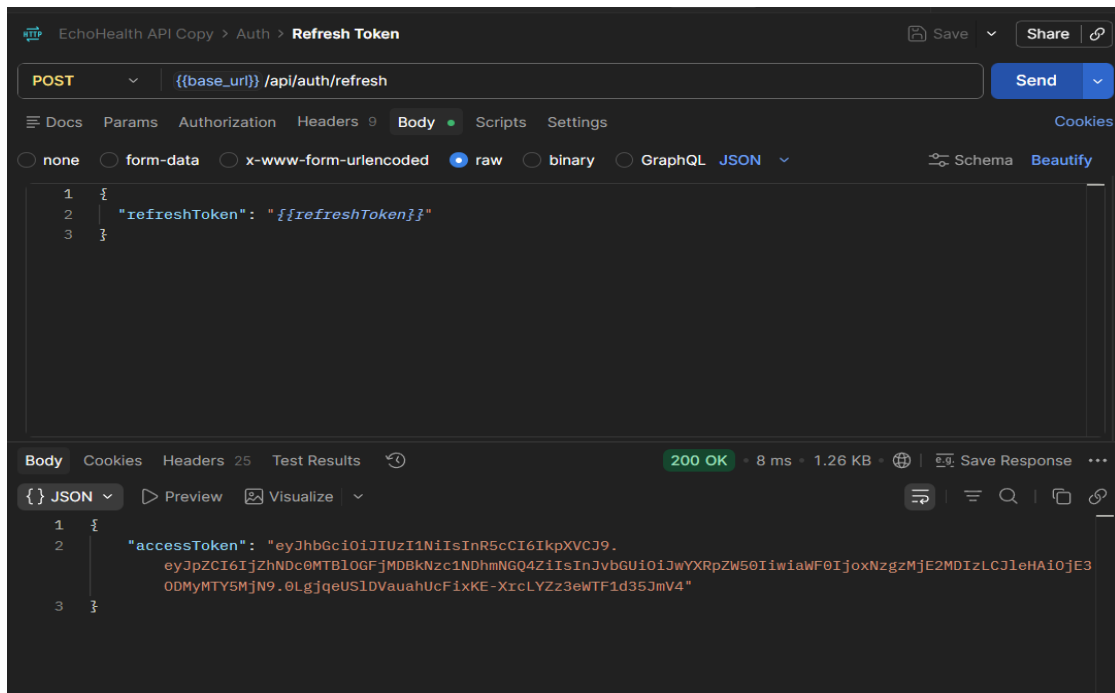
Issues a new short-lived access token using a valid, unexpired refresh token, without requiring the patient to log in again. The refresh token is validated against the value stored in the database — tokens that have been logged out are rejected even if they have not yet expired cryptographically.

Request Body:

```
{ "refreshToken": "<your refresh token>" }
```

Response 200:

```
{ "accessToken": "eyJ..." }
```



POST /api/auth/logout

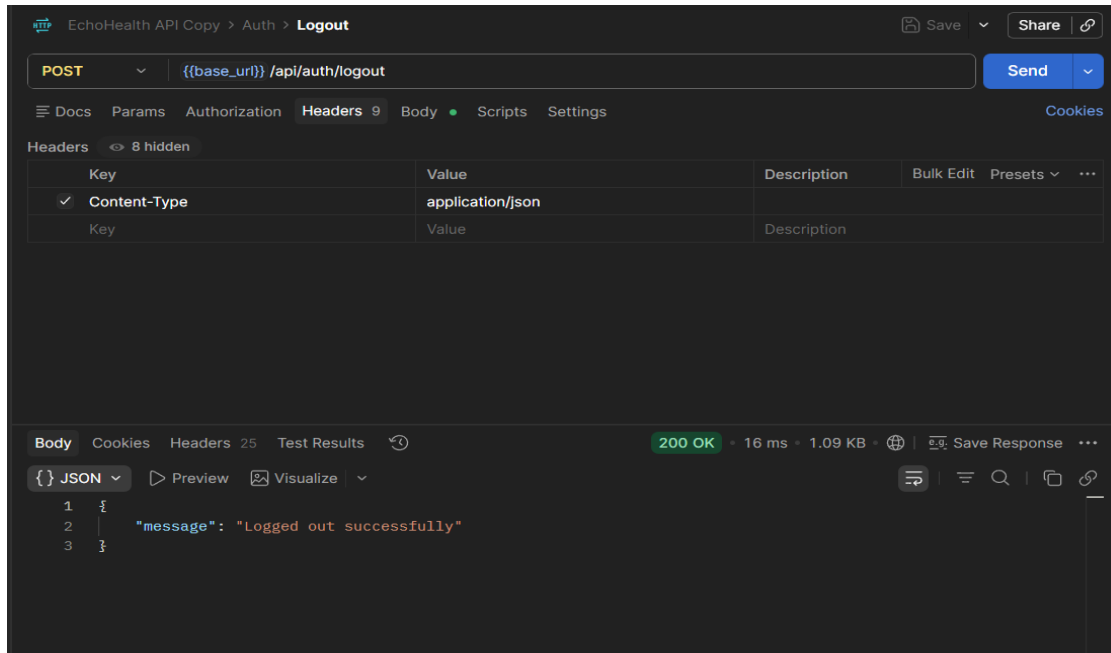
Clears the refresh token from the database, preventing it from being used to issue new access tokens. This is a true server-side logout — the token is invalidated immediately regardless of its remaining cryptographic expiry.

Request Body:

```
{ "refreshToken": "<your refresh token>" }
```

Response 200:

```
{ "message": "Logged out successfully" }
```

Patient Endpoints

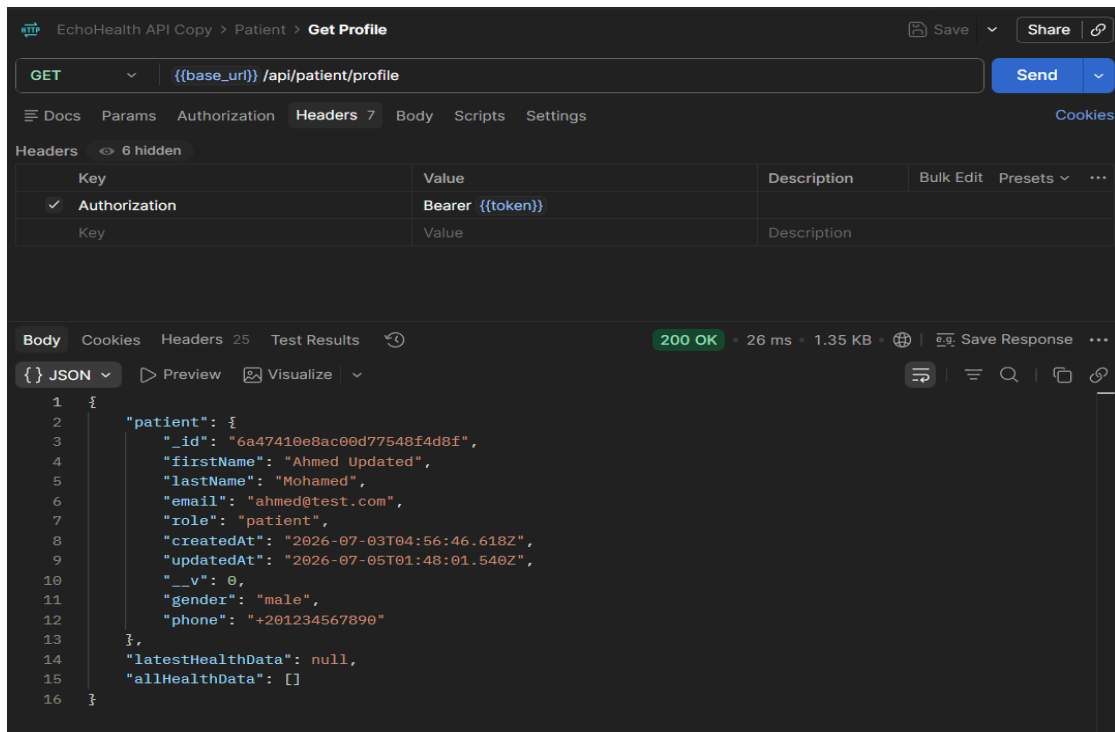
Method	Endpoint	Auth	Description
GET	/api/patient/profile	Yes	Get patient profile + all health records
PUT	/api/patient/profile	Yes	Update patient profile (whitelisted fields only)

GET /api/patient/profile

Returns the authenticated patient's full profile along with their most recent health data record and their complete health history. The password field is excluded from all queries at the model level using `select: false`.

Response 200:

```
{ "patient": { ... }, "latestHealthData": { ... }, "allHealthData": [ ... ] }
```



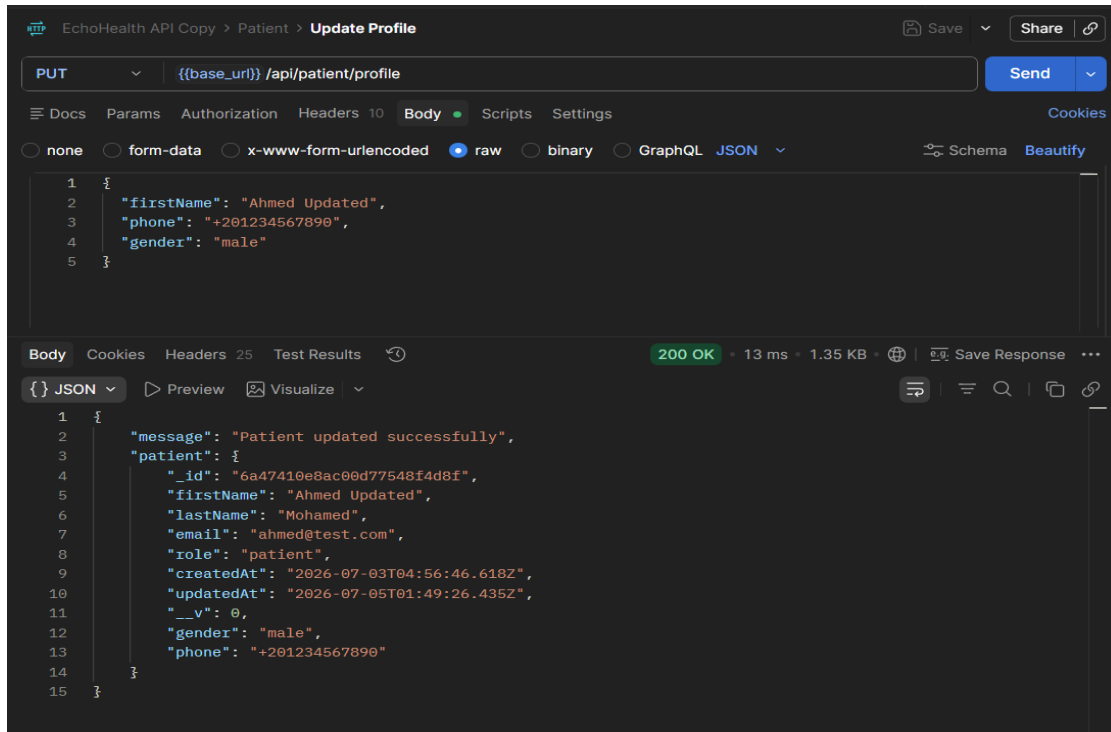
PUT /api/patient/profile

Updates patient profile information. Only five fields are accepted: firstName, lastName, phone, gender, and dob. Any attempt to change email, role, password, or any other field is silently ignored through a server-side whitelist — even if the client sends those fields explicitly.

Request Body:

```
{ "firstName": "Ahmed Updated", "phone": "+201234567890", "gender": "male", "dob": "1990-05-15" }
```

Response 200: Updated patient object without the password field.



Health Data Endpoints

Method	Endpoint	Auth	Description
POST	/api/health-data/submit	Yes	Submit health metrics → ML prediction → save result

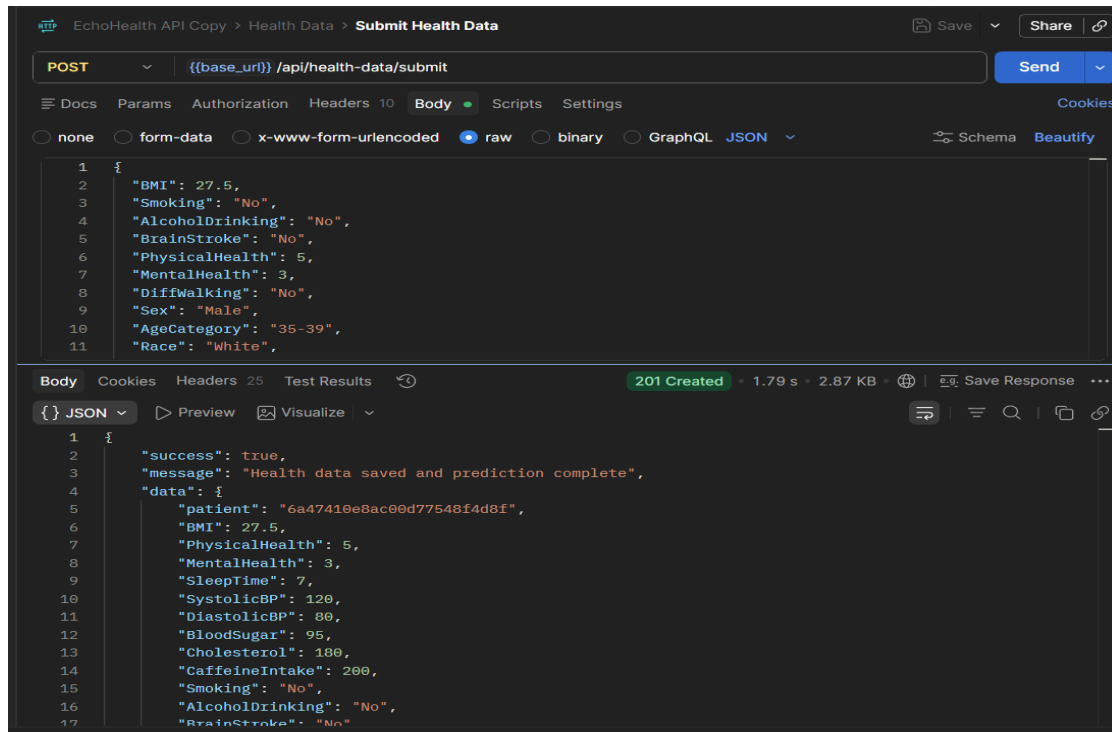
POST /api/health-data/submit

Accepts 25 validated health metrics, forwards them through a field whitelist to the Flask ML service for prediction, and saves the complete result to MongoDB. Only the 25 approved fields are ever sent to Flask or stored in the database — additional fields in the request body are silently dropped.

Required fields: BMI (10–80), Smoking, AlcoholDrinking, BrainStroke, PhysicalHealth (0–30), MentalHealth (0–30), DiffWalking, Sex, AgeCategory, Race, Diabetic, PhysicalActivity, GenHealth, SleepTime (1–24), Asthma, KidneyDisease, CancerHistory, ChronicHypertension, LiverDisease, ImmunologicalDiseases, MyocardialInfarctionInHeart, SystolicBP (50–300), DiastolicBP (30–200), BloodSugar (0–1000), Cholesterol (0–1000), CaffeineIntake (0–5000).

Response 201: Returns predictionResult (At Risk / Not at Risk), probability, riskLevel (Low / Medium / High), blood pressure level, blood sugar level, cholesterol level, and a full AI-generated

analysis including summary, risk factors, recommendations, and lifestyle tips. Returns 503 if the Flask ML service is not running.



Chat Endpoint

Method	Endpoint	Auth	Description
POST	/api/chat/message	Yes	Send a message to the heart-health AI assistant

POST /api/chat/message

Supports two operating modes. In text mode, the patient sends a plain-text question and the Groq API with LLaMA 3 responds. In media mode, the patient uploads a medical image (ECG, X-ray, lab results) or PDF report as base64, and OpenRouter's vision model provides a structured medical analysis. The assistant is constrained by a system prompt to heart-health topics only.

Enforced limits: message maximum 2000 characters, file maximum 5MB, accepted file types JPEG/PNG/WebP/PDF, conversation history trimmed to the last 20 turns to prevent token abuse.

Text Request Body:

```
{ "message": "What foods are good for heart health?", "conversationHistory": [] }
```

Media Request Body:

```
{ "message": "Analyze this ECG", "fileData": "<base64>", "fileType": "image/jpeg",  
  "conversationHistory": [] }
```

Response 200:

```
{ "success": true, "message": "Heart-healthy foods include..." }
```

6.4 Data Models

Patient Model

Stores patient account information along with built-in security fields for account lockout and session management. The password field carries select: false at the schema level so it is never accidentally returned in any query.

Field	Type	Required	Notes
firstName	String	Yes	Max 50 chars, letters only
lastName	String	Yes	Max 50 chars, letters only
email	String	Yes	Unique, stored as lowercase
password	String	Yes	bcrypt hashed at cost 12, never returned in queries
phone	String	No	Format validated (+20...)
gender	String	No	Enum: male or female
dob	Date	No	ISO 8601 format
role	String	Auto	Enum: patient or admin. Default: patient
loginAttempts	Number	Auto	Lockout counter, hidden from queries
lockUntil	Date	Auto	Lock expiry timestamp, hidden from queries
refreshToken	String	Auto	Stored hashed, enables server-side logout
passwordChangedAt	Date	Auto	Invalidates tokens issued before password change

HealthData Model

Stores the complete health submission per patient including all 25 validated input metrics and the full prediction output from the ML model and AI analysis engine.

- All numeric fields carry database-level min/max constraints (e.g. BMI: 10–80, SleepTime: 1–24) as a second line of defence after input validation.
- All binary disease-history fields use a strict Yes/No enum at the schema level.
- The aiAnalysis embedded document stores: summary, risk_factors array, positive_factors array, recommendations array, lifestyle_tips array, warning_signs array, and a follow_up text field.

- A compound index on { patient, createdAt } ensures efficient retrieval of a patient's complete history sorted by date without a full collection scan.

ENVIRONMENT VARIABLES

All secrets and configuration values are stored in a `.env` file in the project root. This file must never be committed to version control — it is listed in `.gitignore`. A `.env.example` file is provided as a safe, value-free template for developers setting up the project.

Variable	Required	Description
PORT	No	Server port. Default: 5001
NODE_ENV	No	development or production
MONGO_URI	Yes	MongoDB connection string
JWT_SECRET	Yes	Minimum 32 random characters, used to sign access tokens
ALLOWED_ORIGINS	No	Comma-separated frontend origins for CORS validation
GEMINI_API_KEY	Yes	Google Gemini API key for AI health analysis
GROQ_API_KEY	Yes	Groq API key for the chat assistant (LLaMA 3 model)
OPENROUTER_API_KEY	Yes	OpenRouter key for vision and image analysis
FLASK_API_URL	Yes	URL of Flask ML service. Default: <code>http://127.0.0.1:5000</code>

6.5 HTTP status code

The API uses standard HTTP status codes consistently across all endpoints. The table below describes every code the API can return and the exact conditions that trigger it.

Code	Status	When it Occurs
200	OK	Login, profile retrieval, token refresh, logout
201	Created	Successful register, successful health data submission
400	Bad Request	Validation failed, email already in use, missing required fields
401	Unauthorized	No token, malformed token, invalid/expired token, alg:none token
415	Unsupported Media Type	Request Content-Type is not application/json on a POST/PUT/PATCH
423	Locked	Account locked after 5 consecutive failed login attempts
429	Too Many Requests	Rate limit exceeded: 10/15min on auth endpoints, 20/min on chat
500	Internal Server Error	Unexpected server-side error — safe message returned, no stack trace
503	Service Unavailable	Flask ML service is not running or connection was refused

6.6 Running the project

Prerequisites

- Node.js version 18 or higher
 - MongoDB running locally or a MongoDB Atlas connection string
 - Flask ML service running on port 5000 (required for health data submission)
 - API keys for Groq, OpenRouter, and Gemini
-

Installation

```
git clone <repository-url>
cd heart_disease_Backend-main
npm install
cp .env.example .env
npm run dev
```

Verify it is Running

Once started, the terminal should display: "Server running on port 5001 [development]" and "MongoDB connected successfully". Verify by visiting:

```
GET http://localhost:5001/
Response: { "status": "EchoHealth API is running" }
```

Log Files

Winston automatically creates two log files in the logs/ directory:

- logs/security.log — all warnings and above: failed logins, locked accounts, invalid tokens, and security events
- logs/combined.log — all events including informational level messages

Every API request passes through the following middleware chain in a fixed sequence before the controller logic executes. Each layer can independently reject the request and return an error response, preventing it from reaching subsequent layers.

Step	Middleware	Action
1	Request ID	Assigns a unique UUID to every request for log correlation
2	Helmet	Applies 15+ security response headers
3	CORS	Validates origin against ALLOWED_ORIGINS whitelist
4	Body Parser	Parses JSON request body (max 20MB)
5	Content-Type Check	Rejects non-JSON bodies on POST/PUT/PATCH with 415
6	MongoSanitize	Strips MongoDB operators from req.body
7	XSS Clean	Escapes HTML characters from all request data
8	HPP	Deduplicates query parameters
9	Rate Limiter	100 req/15min globally; 10 req/15min on auth routes
10	Auth Slow-Down	Progressive delay on auth routes after 3 attempts
11	authMiddleware	Verifies JWT signature, algorithm, and expiry (protected routes only)
12	Input Validator	Validates and sanitises all input fields against defined rules
13	Controller	Business logic executes and response is returned to client

Chapter 7: Security

The EchoHealth backend implements 16 distinct security layers covering input validation, authentication hardening, rate limiting, account protection, data sanitization, and HTTP-level defences. Every request passes through multiple layers in a fixed sequence before the controller logic is allowed to execute.

The table below summarises all layers. Each one is then examined in detail — covering what the vulnerability is, what a real attack looks like, and exactly how EchoHealth defends against it.

Security Layer	Package / Tool	Protection Provided
Helmet HTTP Headers	helmet	15+ secure response headers: CSP, HSTS, X-Frame-Options, X-Content-Type-Options
CORS Restriction	cors	Only allows origins listed in ALLOWED_ORIGINS env variable
Content-Type Enforcement	Custom middleware	Rejects POST/PUT/PATCH requests that are not application/json (returns 415)
NoSQL Injection Protection	express-mongo-sanitize	Strips MongoDB operators (\$gt, \$where, \$regex) from req.body
XSS Sanitization	xss-filters	Escapes HTML/script characters from all request inputs
HTTP Parameter Pollution	hpp	Deduplicates query parameters to prevent override attacks
Auth Rate Limiting	express-rate-limit	10 requests per 15 minutes on all /api/auth/* endpoints
Auth Slow-Down	express-slow-down	+300ms delay per attempt after 3 on auth routes, max 5 seconds
API Rate Limiting	express-rate-limit	100 requests per 15 minutes globally; 20 per minute on /api/chat/*
JWT Algorithm Enforcement	jsonwebtoken	Algorithm locked to HS256 — the alg:none bypass is explicitly disabled
Access Token Expiry	jsonwebtoken	Access tokens expire in 15 minutes; refresh tokens in 7 days

Security Layer	Package / Tool	Protection Provided
Account Lockout	authController.js	Account locked for 15 minutes after 5 consecutive failed login attempts
Input Validation	express-validator	Strict rules: length limits, regex, enums, numeric ranges, date sanity
Mass Assignment Prevention	patientController.js	Whitelist of 5 updatable fields — role and password cannot be changed via API
Timing Attack Protection	bcryptjs	Dummy bcrypt compare runs even when email is not found in the database
Security Logging	winston	All auth events, failed logins, and lockouts written to logs/security.log

7.1 XSS — Cross-Site Scripting

The Problem

Cross-Site Scripting (XSS) is one of the most widespread and dangerous injection vulnerabilities in web applications. It occurs when an attacker is able to inject malicious HTML or JavaScript into data that is later rendered by a browser as if it were trusted content. Unlike server-side attacks that target the backend directly, XSS weaponises the browser itself — turning it against the user who is viewing the page.

In the context of a healthcare platform like EchoHealth, the consequences are severe. Patient names, health summaries, and AI analysis results are stored in the database and later displayed in a patient-facing or admin-facing interface. If a malicious script were stored inside a field such as `firstName`, it would execute silently in the browser of every user who views that patient's record — including clinicians and administrators. This could allow an attacker to steal session tokens and hijack accounts, redirect users to fake login pages to harvest credentials, exfiltrate sensitive medical data to an external server, or silently modify displayed health information to mislead patients or staff.

Real-World Example

An attacker sends a registration request where the `firstName` field contains a JavaScript payload rather than a real name:

```
{ "firstName": "<script>alert('hack')</script>", "lastName": "Ali", "email": "xss@test.com", "password": "Test@1234" }
```

If accepted and stored, this script would execute in the browser of any staff member or patient who later views that profile. In a real attack the payload would not be a harmless alert — it would

silently POST the victim's session token to an attacker-controlled server, giving the attacker full control of that account and access to all medical records visible to it.

How We Fixed It in EchoHealth

EchoHealth defends against XSS at two independent layers, so that bypassing one still leaves the other intact.

The first layer is the `xss-filters` middleware applied globally in `server.js`, which sanitizes `req.body`, `req.query`, and `req.params` before any controller code runs — stripping or escaping HTML tags and JavaScript event handlers from every input field automatically.

The second layer is the `express-validator` regex in `authValidator.js`, which enforces that `firstName` and `lastName` may only contain letters (including Arabic Unicode), spaces, hyphens, and apostrophes. A string containing angle brackets, script tags, or any other HTML character fails this rule and is rejected immediately with a 400 Bad Request before reaching the sanitizer, the controller, or the database.

As the Postman screenshot confirms, the attack request is stopped at the validation layer with a 400 response and the message "First name contains invalid characters" — the payload never touches MongoDB.

```
41
42 function clean(data = '') {
43   let isObject = false;
44   if (typeof data === 'object' && data !== null) {
45     data = JSON.stringify(data);
46     isObject = true;
47   }
48
49   data = xssFilters.inHTMLData(data).trim();
50   if (isObject) {
51     data = JSON.parse(data);
52   }
53
54   return data;
55 }
56
57 const xssClean = (req, res, next) => {
58   if (req.body) {
59     req.body = clean(req.body);
60   }
61   if (req.query) {
62     for (const key in req.query) {
63       if (Object.prototype.hasOwnProperty.call(req.query, key)) {
64         try {
65           req.query[key] = clean(req.query[key]);
66         } catch (e) {
67           //
68         }
69       }
70     }
71   }
72 }
```

EchoHealth API Copy > Security Tests > XSS Attack

POST {{base_url}} /api/auth/register Send

Docs Params Authorization Headers 9 Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
1 {
2   "firstName": "<script>alert('hack')</script>",
3   "lastName": "Ali",
4   "email": "xss@test.com",
5   "password": "Test@1234"
6 }
```

Body Cookies Headers 25 Test Results 400 Bad Request 10 ms 1.17 KB Save Response

{ } JSON Preview Debug with AI

```
1 {
2   "message": "Validation failed",
3   "errors": [
4     {
5       "field": "firstName",
6       "message": "First name contains invalid characters"
7     }
8   ]
9 }
```

7.2 NoSQL Injection

The Problem

NoSQL Injection is the MongoDB equivalent of the SQL injection attack that has plagued relational databases for decades. MongoDB queries are expressed as JavaScript objects, and if user-supplied data is inserted directly into those objects without sanitisation, an attacker can inject MongoDB query operators — such as `$gt` (greater than), `$ne` (not equal), `$where`, and `$regex` — to alter the logic of the query entirely.

The most dangerous application of this attack is authentication bypass. If a login endpoint builds its Mongoose query directly from `req.body`, an attacker can craft a request that makes MongoDB match any document in the collection, regardless of the actual stored password. This gives the attacker full, authenticated access to another patient's account — including their complete medical history, prediction results, and all personally identifiable information — without ever knowing that patient's real credentials.

Real-World Example

An attacker sends a login request where both email and password are replaced with MongoDB comparison operators:

```
{ "email": { "$gt": "" }, "password": { "$gt": "" } }
```

The operator `$gt: ""` means "any value greater than an empty string" — which matches every non-empty string in the database. Without protection, MongoDB would find the first patient in the collection and return their data as if the attacker had provided the correct credentials. The attacker now has a valid access token for a real patient account.

How We Fixed It in EchoHealth

The `express-mongo-sanitize` middleware is applied globally in `server.js` immediately after the body parser. It recursively walks every key in `req.body` and removes any key that begins with a dollar sign (\$) or contains a period (.), which are the characters MongoDB uses to interpret operator expressions. This means the injected operators are stripped out before the request reaches the controller — the Mongoose query never sees them.

As a second independent defence, the `express-validator` in `loginValidation` requires email to be a valid email string, not an object. If the value is an object (as in this attack), the validator rejects the request with a 400 Bad Request and the message "Valid email is required" — the sanitizer has not even had to act yet.

The Postman screenshot confirms that the attack is stopped at the validation layer with a 400 response — the query never reaches MongoDB.

```
79 app.use((req, res, next) => {
80   if (req.body) mongoSanitize.sanitize(req.body, { allowDots: true });
81   if (req.query) mongoSanitize.sanitize(req.query, { allowDots: true });
82   if (req.params) mongoSanitize.sanitize(req.params, { allowDots: true });
83   next();
84 });
85 app.use(xssClean);
```

The screenshot shows a REST client interface for a test named "NoSQL Injection" on the "EchoHealth API Copy". The request is a POST to "{base_url}/api/auth/login". The body is raw JSON:

```
{
  "email": { "$gt": "" OR 1=1 --" },
  "password": { "$gt": "" }
}
```

. The response is a "400 Bad Request" (6 ms, 1.15 KB). The JSON response body is:

```
{
  "message": "Validation failed",
  "errors": [
    {
      "field": "email",
      "message": "Valid email is required"
    }
  ]
}
```


7.3 Fake / Forged Token

The Problem

JSON Web Tokens (JWTs) are the mechanism that separates authenticated requests from anonymous ones. A JWT is a cryptographically signed string consisting of three base64-encoded parts: a header declaring the algorithm, a payload containing the user's identity and expiry time, and a signature generated by the server using its private JWT_SECRET. The signature is what makes the token trustworthy — only the server that knows the secret can produce a valid one.

If an API blindly trusts the contents of the token payload without verifying the signature — or if it accepts an unsigned token (a known vulnerability called the alg:none attack) — then any attacker can craft a token claiming to be any user in the system, including administrators, and gain full access to their data without any credentials. On a medical platform, this would expose every patient's health records, prediction history, and personally identifiable information to an unauthenticated third party.

Real-World Example

An attacker sends a GET request to the protected /api/patient/profile endpoint with an Authorization header containing a made-up, non-JWT string:

```
Authorization: Bearer faketoken123abc
```

A more sophisticated version of this attack uses the alg:none exploit — the attacker crafts a real-looking JWT with a header that declares the algorithm as "none", a payload claiming to be an admin user, and an empty signature. Libraries that do not explicitly reject unsigned tokens would accept this as valid.

How We Fixed It in EchoHealth

The authMiddleware performs three checks before allowing any request to reach a protected controller. First, it verifies that the Authorization header is present and correctly formatted (Bearer <token>). Second, it checks that the token string has exactly three dot-separated segments — malformed strings like faketoken123abc are rejected immediately with "Malformed token" before any cryptographic operation runs.

Third, and most importantly, jwt.verify() is called with the algorithms option explicitly set to ['HS256']. This locks the server to accept only tokens signed with the HMAC-SHA256 algorithm and the server's own JWT_SECRET. Tokens using alg:none, RS256, or any other algorithm are rejected outright — the alg:none bypass that affected early JWT libraries is explicitly disabled.

The Postman screenshot shows the result: any request carrying an invalid or forged token is rejected with 401 Unauthorized before any patient data is touched.

EchoHealth API Copy > Security Tests > Fake Token

SaveShare

GET{{base_url}} /api/patient/profileSend

DocsParamsAuthorizationHeaders7BodyScriptsSettingsCookies

Headers6 hidden

Key	Value	Description	Bulk Edit	Presets	
✓ Authorization	{{bearer_token_0gsv}}				
Key	Value	Description			

BodyCookiesHeaders25Test Results401 Unauthorized6 ms1.09 KBSave Response

{ } JSONPreviewPass the correct auth credentials

```
1 {
2   "message": "Malformed token"
3 }
```

7.4 Missing Authorization Token

The Problem

The most fundamental access-control requirement is that protected routes must be completely inaccessible without valid credentials. If a server allows even a single unauthenticated request through to a protected controller — even as an edge case or oversight — it effectively makes that endpoint public. For a healthcare API, a single unguarded endpoint exposing patient records would constitute a data breach.

This is not a sophisticated attack. It requires no special tools, no crafted payloads, and no knowledge of the target's internal structure. Any person with the endpoint URL and a basic HTTP client (browser, curl, Postman) can send an unauthenticated request. The absence of a credential check is one of the most common API vulnerabilities in real-world systems and appears regularly in bug bounty reports and security audits.

Real-World Example

An attacker sends a completely normal GET request to `/api/patient/profile` with no Authorization header whatsoever:

```
GET http://localhost:5001/api/patient/profile
```

There is no token, no credentials, no session cookie — just a bare HTTP request. If the middleware were absent or applied incorrectly, the server would run the controller and return the patient's profile, health data history, and all linked records to an anonymous caller.

How We Fixed It in EchoHealth

The `protect` middleware (`authMiddleware.js`) is applied explicitly to every protected route before the controller in the route definition. The very first check inside the middleware is whether an Authorization header exists and begins with `"Bearer "`. If either condition fails, the middleware calls `res.status(401).json({ message: "No token provided" })` and terminates the request chain — `next()` is never called, the controller never runs, and no data is returned.

Because the middleware is applied at the route level (not opt-out but opt-in for public routes), there is no risk of accidentally forgetting to protect a new endpoint — public routes must be explicitly left without the middleware.

```

25   async function generateAndSaveRefreshToken(patient) {
26       const refreshToken = crypto.randomBytes(64).toString('hex');
27       patient.refreshToken = refreshToken;
28       await patient.save({ validateBeforeSave: false });
29       return refreshToken;
30   }

```

EchoHealth API Copy > Security Tests > No Token

GET {{base_url}} /api/patient/profile Send

Docs Params Authorization Headers 6 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body Cookies Headers 25 Test Results 401 Unauthorized • 4 ms • 1.09 KB Save Response ...

{ } JSON Preview Pass the correct auth credentials

```

1  {
2  |   "message": "No token provided"
3  }

```

```

if (refreshToken) {
    await Patient.findOneAndUpdate(
        { refreshToken },
        { refreshToken: null }
    );
}

```

7.5 Weak Password Rejection

The Problem

Weak passwords are the single biggest enabler of account takeover across all categories of web application. A password like "123456" or "password" can be cracked in milliseconds by automated tools using credential stuffing (trying passwords from real-world breach databases), dictionary attacks (trying common words and patterns), or brute force (trying every possible combination up to a certain length).

For a healthcare platform, the consequences extend far beyond the compromised account. A patient's EchoHealth account contains their full medical history, heart disease risk predictions, blood pressure readings, cholesterol levels, and AI-generated health analysis — data that is deeply personal and, in many jurisdictions, legally protected. An attacker who takes over an account through a weak password gains access to all of this data instantly, with a fully valid JWT that will pass every authentication check.

The problem is compounded by password reuse — most users recycle the same password across multiple services. When one service suffers a breach, attackers immediately test those leaked credentials against other services (credential stuffing). If EchoHealth accepts weak passwords, it also accepts passwords that have already been leaked elsewhere.

Real-World Example

An attacker (or an inadvertent patient) attempts to register with the password "123456" — a six-character, numeric-only string that appears in virtually every credential breach database ever published:

```
{ "firstName": "Test", "lastName": "User", "email": "weak@test.com", "password":  
  "123456" }
```

Without a password policy, this account would be created and immediately vulnerable to any automated attack tool. The attacker would obtain a valid JWT in seconds.

How We Fixed It in EchoHealth

The express-validator in authValidator.js enforces a multi-rule password policy on the registration endpoint. The password must be at least 8 characters long, must contain at least one uppercase letter, at least one lowercase letter, at least one digit, and at least one special character from the set !@#\$%^&*(). All five rules are evaluated simultaneously.

The password "123456" fails four of the five rules at once. The validator collects all failures and returns a 400 Bad Request with an array of specific error messages — the registration is rejected entirely and no database document is created. The policy means that even the shortest compliant

password (e.g. "Abc@1234") requires a brute-force attacker to search a space of at least 200 trillion combinations — compared to under 1 million for a 6-digit numeric password.

The Postman screenshot shows all four simultaneous validation failures returned to the client.

```
33     if (patient.passwordChangedAt) {
34         const changedAtSeconds = Math.floor(patient.passwordChangedAt.getTime() / 1000);
35         if (decoded.iat < changedAtSeconds) {
36             logger.warn('Token used after password change – rejected', {
37                 requestId: req.requestId,
38                 userId: patient._id,
39                 ip: req.ip,
40                 tokenIssuedAt: decoded.iat,
41                 passwordChangedAt: changedAtSeconds,
42             });
43             return res.status(401).json({
44                 message: 'Password was recently changed. Please log in again.',
45             });
46         }
47     }
48
49     const dummyHash = '$2a$12$invalidhashfortimingprotectiononly00000000000000000000';
50     const isMatch = patient
51       ? await bcrypt.compare(password, patient.password)
52       : await bcrypt.compare(password, dummyHash);
53
54     if (!patient || !isMatch) {
55       // — Increment lockout counter if patient exists —————
56       if (patient) {
57         const newAttempts = (patient.loginAttempts || 0) + 1;
58         const shouldLock = newAttempts >= MAX_LOGIN_ATTEMPTS;
59
60         await Patient.findByIdAndUpdate(patient._id, {
61           loginAttempts: newAttempts,
62           ...(shouldLock && { lockUntil: new Date(Date.now() + LOCK_DURATION_MS) }),
63         });
64
65         logger.warn('Failed login attempt', {
66             requestId: req.requestId,
67             email,
68             ip: req.ip,
69             attempts: newAttempts,
70             accountLocked: shouldLock,
71         });
72       }
73     }
74   }
75 }
```

EchoHealth API Copy > Security Tests > Weak Password

POST {{base_url}} /api/auth/register Send

Docs Params Authorization Headers 9 Body Scripts Settings Ctrl+⇧ bookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
1 {
2   "firstName": "Test",
3   "lastName": "User",
4   "email": "weak@test.com",
5   "password": "123456"
6 }
```

Body Cookies Headers 25 Test Results 400 Bad Request 8 ms 1.42 KB Save Response

{ } JSON Preview Debug with AI

```
1 {
2   "message": "Validation failed",
3   "errors": [
4     {
5       "field": "password",
6       "message": "Password must be at least 8 characters"
7     },
8     {
9       "field": "password",
10      "message": "Password must contain at least one uppercase letter"
11     },
12     {
13       "field": "password",
14       "message": "Password must contain at least one lowercase letter"
15     },
16     {
17       "field": "password",
18       "message": "Password must contain at least one special character"
19     }
20   ]
21 }
```

7.6 Mass Assignment / Privilege Escalation

The Problem

Mass assignment is a classic API vulnerability that arises when a server-side update operation blindly applies every field sent by the client to the database document, without checking whether the client is authorised to modify those fields. It is sometimes called "over-posting" or "parameter binding" in different frameworks.

The danger is that every MongoDB document contains fields that must never be modifiable by the user: the role field that determines access level (patient vs admin), the password hash, the `_id`, and any security-critical fields such as `loginAttempts` or `lockUntil`. If the profile update endpoint passes `req.body` directly to `findByIdAndUpdate()`, a logged-in patient can include any of these fields in their request body and the server will silently apply them — effectively allowing any patient to promote themselves to admin, overwrite their own password hash, or reset their lockout status, all through a normal profile update.

This vulnerability has appeared in real-world systems across every tech stack and framework. It does not require any special tooling — any HTTP client can send the extra fields. The exploit is invisible to standard logging if the server does not specifically track which fields were updated.

Real-World Example

A logged-in patient sends a PUT request to `/api/patient/profile`, including a legitimate `firstName` update alongside two hidden escalation fields:

```
{ "firstName": "Hacker", "role": "admin", "password": "mynewpassword" }
```

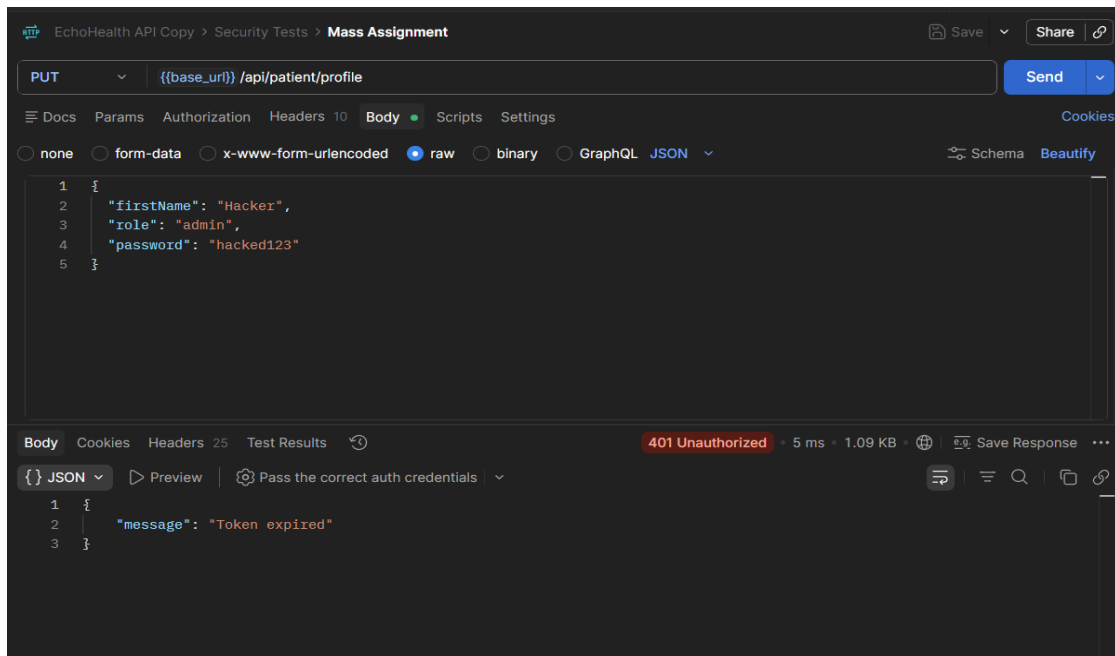
On a vulnerable server, all three fields would be written to the database. The patient's role is now "admin", giving them access to any admin-only endpoints. Their password has also been overwritten with a known value, bypassing `bcrypt` entirely. The patient has fully compromised their own account's privilege level with a single API call.

How We Fixed It in EchoHealth

The `patientController.js` update handler never passes `req.body` directly to Mongoose. Instead, it defines an explicit whitelist of the five fields a patient is permitted to update: `firstName`, `lastName`, `phone`, `gender`, and `dob`. The handler iterates over this whitelist and builds a new object containing only those keys — any other key in `req.body`, including `role`, `password`, `_id`, `loginAttempts`, or `lockUntil`, is silently omitted and never reaches the database.

The `patientValidator.js` adds a second layer: it explicitly checks for the presence of an email field and returns a 400 error ("Email cannot be changed here") if the client attempts to change email through this endpoint, making the restriction explicit rather than silent.

In the Postman screenshot the request was additionally stopped by an expired token (401 "Token expired"), but even with a valid token the role and password fields would never have been written — the whitelist would have silently dropped them and only firstName would have been updated.



Chapter 8: Challenges & Conclusion

Challenges Faced

A. Stale Cached Data After Profile Updates

Problem: After a patient updated their profile, the previously cached data in `SharedPreferences` persisted, and the UI kept showing the old profile information even though the update had succeeded on the server.

Solution: The repository's change-detection logic was extended to include the patient's name and email fields, and the cached profile entry in `SharedPreferences` is now explicitly invalidated before re-fetching after a successful update, ensuring the UI always reflects the latest saved data.

B. JSON Field Naming Mismatches

Problem: The backend's JSON responses used a different field-naming convention (`snake_case`) than the Dart models expected (`camelCase`) in certain endpoints, causing some fields to parse as null instead of raising a visible error.

Solution: The affected model's `fromJson` factory was updated to explicitly map both naming conventions, resolving the mismatch without requiring backend changes.

C. Vital Sign Classification Threshold Ambiguity

Problem: The initial classification rules used by the ML service's response for vital signs (blood pressure, blood glucose, cholesterol) were too vague, which occasionally led to borderline values being misclassified, and the level field (e.g., Normal, Elevated) did not consistently appear across every structured response.

Solution: Explicit, AHA-aligned numeric threshold ranges were defined for each vital sign, and the level field was standardized to always appear in the structured output, improving the reliability of both the Result screen's status badges and the History dashboard's cross-assessment insights.

D. Android Build and Platform Compatibility Issues

Problem: Several Android-specific build issues surfaced during development: Gradle and Java version incompatibilities that prevented successful builds, compile errors from an unconditionally imported `dart:html` (a web-only library), and a malformed intent-filter entry in `AndroidManifest.xml` that caused a crash on launch.

Solution: Gradle and Java versions were aligned according to Flutter's compatibility requirements, the web-only import was wrapped behind a conditional import so it only executes on supported platforms, and

the AndroidManifest.xml intent-filter entry was corrected, restoring a stable launch on all target Android devices.

Solutions Implemented

All major features were implemented successfully:

- Assessments are submitted and evaluated with a consistent, reliable structured response from the ML service.
- Profile updates are reflected immediately and consistently across the Home, History, and Profile tabs.
- The History dashboard aggregates recurring risk factors and warning signs reliably across a patient's full assessment history.
- The app builds and runs reliably across the target Android devices used during testing.

Conclusion

The development of HeartGuard using Flutter demonstrated the framework's strengths for building a responsive, data-driven health application from a single codebase. Despite challenges such as caching consistency, JSON field mismatches, vital-sign classification reliability, and Android build compatibility, Flutter's tooling and ecosystem — combined with the BLoC pattern for state management — enabled the team to resolve these issues effectively. The result is a consistent, informative experience across devices that helps patients understand their heart-disease risk, track it over time, and take proactive steps toward better heart health.

1.1 Background

Cardiovascular disease refers to a group of conditions affecting the heart and blood vessels, including coronary artery disease, heart failure, and stroke. It remains one of the leading causes of death worldwide, and a large share of that burden falls on cases that are diagnosed only after symptoms have already become serious. Many of the risk factors behind these conditions, such as high blood pressure, elevated cholesterol, and abnormal blood sugar levels, develop gradually and can go unnoticed for years before a cardiac event occurs.

Early prediction changes this picture significantly. When a patient's risk level is identified before symptoms appear, healthcare providers have the opportunity to recommend lifestyle changes, closer monitoring, or preventive treatment, all of which are far more effective and less costly than emergency intervention. The challenge, however, is that cardiovascular risk depends on the

combined effect of many clinical factors at once, which makes it difficult to assess reliably using simple rule-based checks alone.

This is where artificial intelligence and machine learning have started to play a growing role in healthcare. Machine learning models are increasingly used to support tasks such as disease screening, risk stratification, and clinical decision support, because they can process large numbers of patient variables together and identify patterns that are not always obvious through manual review. HeartGuard is built on this same principle, applying a machine learning approach to the specific problem of cardiovascular risk prediction.